



ORACLE  
NetSuite

# SuiteTalk REST Web Services Records Guide

---

2025.2

January 7, 2026



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

### **Documentation Accessibility**

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

### **Access to Oracle Support**

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

### **Sample Code**

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at [www.netsuite.com/tos](http://www.netsuite.com/tos).

Oracle may modify or remove sample code at any time without notice.

### **No Excessive Use of the Service**

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

### **Beta Features**

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

# Table of Contents

|                                           |    |
|-------------------------------------------|----|
| REST Web Services Supported Records ..... | 1  |
| Account .....                             | 5  |
| Accounting Period .....                   | 6  |
| Advanced Intercompany Journal Entry ..... | 7  |
| Analytical Impact .....                   | 9  |
| Assembly Build .....                      | 9  |
| Assembly Item .....                       | 10 |
| Assembly Unbuild .....                    | 12 |
| Billing Account .....                     | 13 |
| Billing Revenue Event .....               | 14 |
| Billing Schedule .....                    | 15 |
| Bin .....                                 | 16 |
| Bin Putaway Worksheet .....               | 16 |
| Bin Transfer .....                        | 17 |
| Blanket Purchase Order .....              | 17 |
| BOM .....                                 | 18 |
| BOM Revision .....                        | 20 |
| Budget Category .....                     | 21 |
| Campaign .....                            | 22 |
| Campaign Response .....                   | 24 |
| Cash Refund .....                         | 25 |
| Cash Sale .....                           | 26 |
| Change Order .....                        | 28 |
| Charge .....                              | 29 |
| Check .....                               | 29 |
| Class .....                               | 31 |
| Commerce Category .....                   | 31 |
| Competitor .....                          | 33 |
| Consolidated Exchange Rate .....          | 34 |
| Contact .....                             | 35 |
| Contact Category .....                    | 36 |
| Contact Role .....                        | 37 |
| Cost Category .....                       | 37 |
| Coupon Code .....                         | 39 |
| Credit Card Charge .....                  | 40 |
| Credit Card Refund .....                  | 41 |
| Credit Memo .....                         | 43 |
| Currency .....                            | 44 |
| Currency Rate .....                       | 45 |
| Customer .....                            | 46 |
| Customer Category .....                   | 48 |
| Customer Deposit .....                    | 48 |
| Customer Message .....                    | 49 |
| Customer Payment .....                    | 50 |
| Customer Refund .....                     | 52 |
| Customer Status .....                     | 53 |
| Customer Subsidiary Relationship .....    | 54 |
| Department .....                          | 55 |
| Deposit .....                             | 56 |
| Deposit Application .....                 | 58 |
| Description Item .....                    | 59 |
| Discount Item .....                       | 60 |
| Download Item .....                       | 61 |

|                                      |     |
|--------------------------------------|-----|
| Email Template .....                 | 62  |
| Employee .....                       | 63  |
| Estimate .....                       | 67  |
| Event .....                          | 69  |
| Expense Category .....               | 71  |
| Expense Report .....                 | 72  |
| Fair Value Price .....               | 73  |
| Fulfillment Request .....            | 74  |
| Gift Certificate .....               | 75  |
| Gift Certificate Item .....          | 76  |
| Global Account Mapping .....         | 77  |
| HCM Job .....                        | 77  |
| Impact Subcategory .....             | 79  |
| Inbound Shipment .....               | 80  |
| Intercompany Journal Entry .....     | 81  |
| Intercompany Transfer Order .....    | 82  |
| Inventory Adjustment .....           | 82  |
| Inventory Cost Revaluation .....     | 83  |
| Inventory Count .....                | 85  |
| Inventory Item .....                 | 86  |
| Inventory Number .....               | 87  |
| Inventory Transfer .....             | 88  |
| Invoice .....                        | 89  |
| Issue .....                          | 91  |
| Item Account Mapping .....           | 92  |
| Item Fulfillment .....               | 93  |
| Item Group .....                     | 93  |
| Item Receipt .....                   | 94  |
| Item Revision .....                  | 95  |
| Job .....                            | 96  |
| Job Status .....                     | 98  |
| Job Type .....                       | 98  |
| Journal Entry .....                  | 99  |
| Kit Item .....                       | 100 |
| Location .....                       | 101 |
| Manufacturing Cost Template .....    | 102 |
| Manufacturing Operation Task .....   | 103 |
| Manufacturing Routing .....          | 104 |
| Markup Item .....                    | 106 |
| Message .....                        | 107 |
| Merchandise Hierarchy Level .....    | 109 |
| Merchandise Hierarchy Node .....     | 110 |
| Merchandise Hierarchy Version .....  | 111 |
| Nexus .....                          | 111 |
| Non-Inventory Purchase Item .....    | 112 |
| Non-Inventory Resale Item .....      | 113 |
| Non-Inventory Sale Item .....        | 115 |
| Note Type .....                      | 116 |
| Opportunity .....                    | 117 |
| Other Charge for Purchase Item ..... | 119 |
| Other Charge for Resale Item .....   | 120 |
| Other Charge for Sale Item .....     | 121 |
| Other Name .....                     | 122 |
| Other Name Category .....            | 123 |
| Partner .....                        | 124 |

|                                      |     |
|--------------------------------------|-----|
| Paycheck .....                       | 126 |
| Payment Item .....                   | 130 |
| Payment Method .....                 | 131 |
| Payroll Item .....                   | 131 |
| Period End Journal Entry .....       | 134 |
| Phone Call .....                     | 135 |
| Price Book .....                     | 137 |
| Price Level .....                    | 138 |
| Price Plan .....                     | 139 |
| Pricing Group .....                  | 139 |
| Project Task .....                   | 141 |
| Promotion Code .....                 | 142 |
| Purchase Contract .....              | 144 |
| Purchase Order .....                 | 146 |
| Return Authorization .....           | 147 |
| Requisition .....                    | 149 |
| Revenue Recognition Schedule .....   | 150 |
| Revenue Recognition Template .....   | 151 |
| Sales Order .....                    | 152 |
| Sales Role .....                     | 154 |
| Sales Tax Item .....                 | 155 |
| Service Purchase Item .....          | 156 |
| Service Resale Item .....            | 156 |
| Service Sale Item .....              | 157 |
| Ship Item .....                      | 158 |
| Subscription .....                   | 159 |
| Subscription Line .....              | 160 |
| Subscription Plan .....              | 161 |
| Subscription Term .....              | 162 |
| Statistical Journal Entry .....      | 162 |
| Subsidiary .....                     | 163 |
| Subtotal Item .....                  | 165 |
| Support Case .....                   | 165 |
| Task .....                           | 167 |
| Tax Type .....                       | 169 |
| Term .....                           | 170 |
| Time Bill (Track Time) .....         | 172 |
| Topic .....                          | 172 |
| Transfer Order .....                 | 174 |
| Unit of Measure .....                | 175 |
| Usage .....                          | 176 |
| Vendor .....                         | 176 |
| Vendor Bill .....                    | 177 |
| Vendor Category .....                | 178 |
| Vendor Credit .....                  | 179 |
| Vendor Payment .....                 | 179 |
| Vendor Prepayment .....              | 180 |
| Vendor Prepayment Application .....  | 181 |
| Vendor Return Authorization .....    | 182 |
| Vendor Subsidiary Relationship ..... | 183 |
| Website .....                        | 185 |
| Weekly Timesheet .....               | 186 |
| Work Order .....                     | 187 |
| Work Order Close .....               | 189 |
| Work Order Completion .....          | 190 |

|                                                                                                       |     |
|-------------------------------------------------------------------------------------------------------|-----|
| Work Order Issue .....                                                                                | 191 |
| REST Web Services Tutorials .....                                                                     | 192 |
| Sales Order Use Cases .....                                                                           | 192 |
| Use Case For Creating Your Sales Order .....                                                          | 193 |
| Use Case For Applying a Promotion to Your Sales Order .....                                           | 194 |
| Use Case For Retrieving Your Sales Order .....                                                        | 195 |
| Use Case For Updating Your Sales Order .....                                                          | 198 |
| Use Case For Approving Your Sales Order .....                                                         | 200 |
| Use Case For Fulfilling Your Sales Order .....                                                        | 201 |
| Use Case For Creating Invoices or Cash Sales from Your Sales Order .....                              | 202 |
| Use Case For Creating a Progress Sales Order .....                                                    | 203 |
| Use Case For Deleting a Sales Order .....                                                             | 204 |
| SuiteBilling Use Cases .....                                                                          | 204 |
| Use Case For Managing Your Subscription Catalog .....                                                 | 204 |
| Use Case For Managing Your Subscription Sales .....                                                   | 213 |
| Inventory Item Use Cases .....                                                                        | 216 |
| Use Case for Changing the Location of an Inventory Item .....                                         | 216 |
| Use Case for Adding a Vendor to an Inventory Item .....                                               | 217 |
| Use Case for Finding Items that are Assigned a Pricing Group .....                                    | 217 |
| Use Case for Deleting an Inventory Item .....                                                         | 218 |
| Customer Use Cases .....                                                                              | 218 |
| Use Case for Finding Customers with Shipping Carrier Set to UPS .....                                 | 218 |
| Use Case for Updating Contact Details of a Customer .....                                             | 219 |
| Use Case for Creating a Customer from an External ID .....                                            | 219 |
| Use Case for Fetching a List of All Customers with Pagination .....                                   | 220 |
| Use Case for Creating an Invoice from a Customer .....                                                | 221 |
| Employee Use Cases .....                                                                              | 221 |
| Use Case for Adding a Supervisor to an Employee .....                                                 | 222 |
| Use Case for Adding Employee Address .....                                                            | 222 |
| Use Case for Adding Subscriptions to an Employee .....                                                | 222 |
| Use Case for Creating an Expense Report from an Employee Record .....                                 | 222 |
| Use Case for Finding All Employee Records Under a Supervisor .....                                    | 223 |
| Use Case for Retrieving Permissions Assigned to a Role .....                                          | 223 |
| Use Case for Finding IDs of Roles Assigned to an Employee .....                                       | 225 |
| Use Case for Finding the Name of a Role .....                                                         | 225 |
| Invoice Use Cases .....                                                                               | 226 |
| Use Case for Creating an Invoice .....                                                                | 226 |
| Use Cases for Updating an Invoice .....                                                               | 227 |
| Use Case or Retrieving an Invoice .....                                                               | 227 |
| Use Case for Fetching Invoices from a Lead Source .....                                               | 230 |
| Use Case for Deleting an Invoice .....                                                                | 231 |
| Journal Entry Use Cases .....                                                                         | 231 |
| Use Case for Creating a Journal Entry .....                                                           | 231 |
| Use Cases for Approving a Journal Entry .....                                                         | 232 |
| Use Case for Creating a Reverse Journal Entry .....                                                   | 232 |
| Use Case for Updating Line Items in a Journal Entry .....                                             | 232 |
| Use Case for Retrieving Journal Entries Created After a Date .....                                    | 233 |
| Event Use Cases .....                                                                                 | 234 |
| Use Case for Creating an Event .....                                                                  | 235 |
| Use Case for Adding an Attendee to an Event .....                                                     | 235 |
| Use Case for Getting a List of Events with Confirmed Status and that are Scheduled After a Date ..... | 236 |
| Use Case for Fetching Details of Events at a Location .....                                           | 237 |
| Use Case for Deleting an Event .....                                                                  | 237 |
| Credit Memo Use Cases .....                                                                           | 238 |

|                                                                                     |     |
|-------------------------------------------------------------------------------------|-----|
| Use Case for Creating a Credit Memo .....                                           | 238 |
| Use Case for Adding Line Items to a Credit Memo .....                               | 239 |
| Use Case for Fetching List of Credit Memos Created After a Date .....               | 239 |
| Use Case for Fetching Credit Memos from a Lead Source .....                         | 240 |
| Use Case for Getting Details of a Credit Memo .....                                 | 240 |
| Vendor Use Cases .....                                                              | 243 |
| Use Case for Creating a Vendor .....                                                | 243 |
| Use Case for Creating a Tax Agency Vendor .....                                     | 244 |
| Use Case for Providing Role Access to a Vendor .....                                | 244 |
| Use Case for Inactivating a Vendor .....                                            | 244 |
| Use Case for Setting Credit Limit for a Vendor .....                                | 244 |
| Use Case for Defining the Preferred Transaction Delivery Methods for a Vendor ..... | 245 |
| Use Case for Finding Vendors that are Assigned an Expense Account .....             | 245 |
| Use Case for Getting Details of Vendors that are Assigned a Category .....          | 246 |
| Use Case for Getting List of Vendors that have Credit Limits Above a Value .....    | 246 |
| Vendor Bill Use Cases .....                                                         | 247 |
| Use Case for Creating a Vendor Bill .....                                           | 247 |
| Use Case for Approving a Vendor Bill .....                                          | 248 |
| Use Case for Adding Line Items to a Vendor Bill .....                               | 248 |
| Use Case for Getting a List of Bills from a Vendor .....                            | 248 |
| Cash Sale Use Cases .....                                                           | 249 |
| Use Case for Creating a Cash Sale .....                                             | 250 |
| Use Case for Updating a Cash Sale .....                                             | 250 |
| Use Case for Issuing a Return Authorization from a Cash Sale .....                  | 250 |
| Use Case for Fetching List of Cash Sales for a Customer .....                       | 250 |

# REST Web Services Supported Records

The following records are supported in REST web services.



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

## Communication Records

- [Event](#)
- [Issue](#)
- [Message](#)
- [Note Type](#)
- [Phone Call](#)
- [Task](#)

## Entity Records

- [Competitor](#)
- [Contact](#)
- [Contact Category](#)
- [Contact Role](#)
- [Customer](#)
- [Employee](#)
- [Partner](#)
- [Vendor Category](#)

## HCM records

- [HCM Job](#)
- [Payroll Item](#)

## List records

- [Bin](#)
- [Merchandise Hierarchy Level](#)
- [Merchandise Hierarchy Node](#)
- [Merchandise Hierarchy Version](#)
- [Unit of Measure](#)

## ERP Records

- [Account](#)
- [Accounting Period](#)
- [Advanced Intercompany Journal Entry](#)
- [Analytical Impact](#)
- [Assembly Build](#)

- Assembly Unbuild
- Billing Revenue Event
- Bin Putaway Worksheet
- Bin Transfer
- Blanket Purchase Order
- BOM
- BOM Revision
- Budget Category
- Class
- Cost Category
- Customer Category
- Customer Deposit
- Customer Message
- Customer Payment
- Customer Refund
- Customer Status
- Customer Subsidiary Relationship
- Department
- Deposit
- Deposit Application
- Description Item
- Discount Item
- Download Item
- Expense Category
- Expense Report
- Fair Value Price
- Fulfillment Request
- Global Account Mapping
- Gift Certificate
- Gift Certificate Item
- Impact Subcategory
- Inbound Shipment
- Intercompany Journal Entry
- Intercompany Transfer Order
- Inventory Adjustment
- Inventory Cost Revaluation
- Inventory Count
- Inventory Item
- Inventory Number
- Inventory Transfer

- Item Account Mapping
- Item Fulfillment
- Item Group
- Item Receipt
- Item Revision
- Job
- Job Status
- Job Type
- Journal Entry
- Kit Item
- Location
- Manufacturing Cost Template
- Manufacturing Operation Task
- Manufacturing Routing
- Nexus
- Non-Inventory Purchase Item
- Non-Inventory Sale Item
- Non-Inventory Resale Item
- Other Charge for Purchase Item
- Other Charge for Resale Item
- Other Charge for Sale Item
- Other Name
- Other Name Category
- Paycheck
- Payment Item
- Payment Method
- Period End Journal Entry
- Project Task
- Purchase Contract
- Purchase Order
- Requisition
- Revenue Recognition Schedule
- Revenue Recognition Template
- Sales Role
- Sales Tax Item
- Service Purchase Item
- Service Resale Item
- Service Sale Item
- Ship Item
- Statistical Journal Entry
- Subsidiary

- Subtotal Item
- Support Case
- Tax Type
- Time Bill (Track Time)
- Topic
- Transfer Order
- Vendor
- Vendor Bill
- Vendor Credit
- Vendor Payment
- Vendor Prepayment
- Vendor Prepayment Application
- Vendor Return Authorization
- Vendor Subsidiary Relationship
- Weekly Timesheet
- Work Order
- Work Order Close
- Work Order Completion
- Work Order Issue

#### Marketing Records

- Campaign
- Campaign Response
- Coupon Code
- Email Template
- Promotion Code

#### SuiteBilling Records

- Assembly Item
- Billing Account
- Billing Schedule
- Change Order
- Charge
- Price Book
- Price Plan
- Pricing Group
- Subscription
- Subscription Line
- Subscription Plan
- Subscription Term
- Usage

## Pricing Records

- [Cash Refund](#)
- [Cash Sale](#)
- [Check](#)
- [Consolidated Exchange Rate](#)
- [Credit Card Charge](#)
- [Credit Card Refund](#)
- [Credit Memo](#)
- [Currency](#)
- [Currency Rate](#)
- [Estimate](#)
- [Invoice](#)
- [Markup Item](#)
- [Opportunity](#)
- [Price Level](#)
- [Return Authorization](#)
- [Sales Order](#)

## Website Records

- [Commerce Category](#)
- [Website](#)

# Account

An account is a record in your general ledger that is used to store transactions. Examples of accounts include accounts payable, cash, or cost of goods sold. Accounts are collected in your Chart of Accounts, which provides a set of destinations for posting transactions, and categorizes these transactions for tracking and reporting purposes.

Accounts have account types, which are used to organize data in account registers and other financial reports.

- Income statement account types include Cost of Goods Sold, Expense, Income, Other Expense, and Other Income.
- Balance sheet account types include Accounts Payable, Accounts Receivable, Bank, Credit Card, Deferred Expense, Deferred Revenue, Equity, Fixed Asset, Long Term Liability, Other Asset, Other Current Asset, Other Current Liability, and Unbilled Receivable.
- Cash flow statement account types include the net of Income and Other Income, Accounts Payable, Accounts Receivable, Equity, Fixed Asset, Long Term Liability, Other Asset, Other Current Asset, Other Current Liability, and Unbilled Receivable.
- Statistical accounts, part of the Advanced Financial module, enable your financial team to track non-monetary data and then use that information about reports and income statements. Financial users can examine the non-monetary data to view its relationship with the financial activity of your organization. For more information, see the help topic [Using Statistical Accounts](#).

For more information, see the help topic [Chart of Accounts Management](#).

The REST API Browser includes information about the field names and field types of the account record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [account](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an account REST record is **account**.

The account record has the following subrecords:

- `accountcontextsearch`
- `localizations`

## Inaccessible Elements

Read-only sublists, including system notes, are not supported in REST.

## Code Sample

```
1 { "acctName": "Test Account333 1692257016", "acctType": { "id": "OthCurrAsset" }, "cashFlowRate": { "id": "AVERAGE" }, "description": "description text", "eliminate": false, "generalRate": { "id": "CURRENT" }, "includeChildren": false, "inventory": true, "isInactive": true, "isSummary": true, "revalue": true, "subsidiary": { "items": [ { "id": 2 }, { "id": 4 } ] } }
```

## Accounting Period

An accounting period record exposes an accounting period to REST web services. This record has the following subrecords:

- `fiscalcalendars`
- `perbookperiodclosing`

Accounting periods support general ledger management. When accounting periods are enabled, users can select a past or future posting period for each transaction, and accounting personnel can more effectively monitor and control the status of accounts.

To access this record in NetSuite, go to Setup > Accounting > Manage G/L > Manage Accounting Periods.

For more information about managing accounting periods in NetSuite, see the help topic [Manage Accounting Periods Page](#).

The REST API Browser includes information about the field names and field types of the accounting period record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [accountingPeriod](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an accounting period REST record is **accountingperiod**.

## Inaccessible Elements

The following sublist is not accessible through REST web services:

- User Notes

## Elements with Different Functionality

The following fields only appear in the base accounting period type:

- allowNonGLChanges
- isAdjust

The following sublist only appears in the base accounting period type:

- fiscalCalendars

**Note:** The accounting period is a base period if isYear = "false" and isQuarter = "false".

The following field only appears in the quarter and base accounting period types:

- parent

**Note:** The accounting period is a quarter period if isQuarter = "True" and isYear = "false".

## Additional Details

You must have a parent period to set up a new quarter or base period.

## Code Sample

This sample shows how to create a new base accounting period:

```
1 POST /services/rest/record/v1/accountingperiod { "allLocked": false, "allowNonGLChanges": false, "apLocked": false, "ar
  Locked": false, "closed": false, "endDate": "2051-01-31", "parent": "546", "isAdjust": false, "isInactive": false, "isPost
  ing": true, "isQuarter": false, "isYear": false, "payrollLocked": false, "periodName": "January 2051 1692277896", "start
  Date": "2051-01-01"
2 }
```

## Advanced Intercompany Journal Entry

An advanced intercompany journal entry subrecord exposes an advanced intercompany journal entry to REST web services.

The advanced intercompany journal entry is a subrecord.

An advanced intercompany journal entry records debits and credits to be posted to ledger accounts for transactions between an originating subsidiary and multiple receiving subsidiaries. In an account that has the Multi-Book Accounting feature enabled, you can also use this record type to create book specific advanced intercompany journal entries.

To access this record in NetSuite, go to Transactions > Financial > Make Advanced Intercompany Journal Entries.

For more information about the advanced intercompany journal entry record, see the help topic [Making Advanced Intercompany Journal Entries](#).

The REST API Browser includes information about the field names and field types of the advanced intercompany journal entry record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [advIntercompanyJournalEntry](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an advanced intercompany journal entry REST record is **AdvIntercompanyJournalEntry**.

## Inaccessible Elements

The following fields are not accessible through REST web services:

- book
- details
- detailsUrl
- line
- message
- messageType
- expensePlanMessage
- expensePlanMessageExists
- expensePlanStatus

## Elements with Different Functionality

The following fields have different functionality through REST web services:

- expensePlanStatus (Read only)
- tosubsidiaries (Read only)

## Usage Notes

The performautobalance field is only available in metadata and schema.

## Code Sample

This sample shows how to retrieve an advanced intercompany journal entry record by ID:

```
1 | GET {{REST_SERVICES}}/record/v1/AdvIntercompanyJournalEntry/1107?expandSubResources=true
```

## Analytical Impact

An analytical impact record exposes analytical impact data to REST web services.

This record:

- is not a subrecord
- has no subrecords

To access this record in NetSuite, go to Setup > Company > Enable Features > Analytics > Advanced SaaS Metrics (Administrator).

To import analytical impact records, verify the following settings:

- On the Analytics subtab, enable the **SaaS Metrics** feature.

For more information about setting up analytical impact records, see the help topic [SuiteBilling Overview](#).

The REST API Browser includes information about the field names and field types of the analytical impact record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [analytical impact](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an analytical impact REST record is **analyticalimpact**.

## Code Sample

This sample shows how to get an analytical impact record.

```
1 | GET {{REST_SERVICES}}/record/v1/analyticalimpact/5
```

This sample shows how to update an analytical impact record.

```
1 | PATCH {{REST_SERVICES}}/record/v1/analyticalimpact/4
2 | Body
3 | {
4 |   "renewedFrom": "3"
5 | }
```

## Assembly Build

An assembly build record exposes an assembly build to REST web services.

This record:

- is not a subrecord
- has one subrecords: component

The REST API Browser includes information about the field names and field types of the assembly build record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [assemblyBuild](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an assembly build REST record is **assemblyBuild**.

## Code Samples

The following samples show common use cases for assembly builds. The example ID is 4.

### Creating an Assembly Build Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/assemblyBuild
2 | { "subsidiary": { "id": "5" }, "location": { "id": "5" }, "item": { "id": "67" }, "quantity": 10.0
3 | }
```

### Retrieving an Assembly Build Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/assemblyBuild/4
```

The following code sample shows how to retrieve an assembly build component. The component ID is 7.

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/assemblyBuild/4/component/lineNumber=7
```

### Updating an Assembly Build Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/assemblyBuild/4
2 | { "quantity": "20"
3 | }
```

The following code sample shows how to update an assembly build component. The component ID is 7.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/assemblyBuild/4/component/lineNumber=7/
2 | { "quantity": "21"
3 | }
```

The following code sample shows how to convert an assembly build to an assembly unbuild.

```
1 | POST {{COMPANY_URL}}/services/rest/record/v1/assemblyBuild/4/!transform/assemblyUnbuild
```

## Assembly Item

An assembly item is an inventory item made up of several components but identified as a single item. Assemblies are manufactured by combining raw materials that you stock.

For more information, see the help topic [Assembly Items](#).

This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the assembly item record, and about the HTTP methods, request parameters, and operations available to this record.

For details, see the REST API Browser's [assembly item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the assembly item REST record is **assemblyItem**.

## Prerequisites

You must enable the Assemblies feature before you can use this record through REST web services.

## Inaccessible Elements

The following sublists are not accessible through REST web services:

- Quantity Pricing Discount/Levels (quantitypricingdiscount, quantitypricinglevel)
- Item Translations subtab (translations)

## Elements with Different Functionality

The following sublists have different functionality through REST web services:

Manufacturing > Bills of Materials > Master Default field is not controllable through REST.

## Code Samples

### Get Record Metadata

```
1 | GET https://<HOST>/services/rest/record/v1/metadata-catalog/assemblyItemHeader: Accept: application/swagger+json
```

### Get Record

```
1 | GET https://<HOST>/services/rest/record/v1/assemblyItem/<ID>?expandSubResources=true
```

### Update Record

```
1 | PATCH
2 | https://<HOST>/services/rest/record/v1/assemblyItem/<ID>
```

```

3 | Header: Content-Type: application/json
4 | Body:
5 | {
6 |     "itemId": "Updated item name",
7 |     "upcCode": "123456"
8 | }

```

## Update Item Price when all Related Features are Enabled

```

1 | PATCH https://<HOST>/services/rest/record/v1/assemblyItem/<ID>/price/currencypage=1,pricelevel=1,quantity=10
2 | Header: Content-Type: application/json
3 | Body:
4 | {
5 |     "price": 10
6 | }

```

## Update Bill of Materials/Default for Location

```

1 | PATCH https://<HOST>/services/rest/record/v1/assemblyItem/<ID>
2 | Header: Content-Type: application/json
3 | Body:
4 | { "billOfMaterials": { "items": [ { "billOfMaterials": { "id": 14 } "defaultForLocation": { "items": [ { "id": 5 }, { "id": 6
  } ] ] ] ] } } } } }

```

## Assembly Unbuild

An assembly unbuild record exposes an assembly unbuild to REST web services.

This record:

- is not a subrecord
- has one subrecord: component

The REST API Browser includes information about the field names and field types of the assembly unbuild record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [assemblyUnbuild](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an assembly unbuild REST record is **assemblyUnbuild**.

## Code Samples

The following samples show common use cases for assembly unbuids. The example ID is 4.

## Creating an Assembly Unbuild Using a POST Request

```

1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/assemblyUnbuild
2 | { "subsidiary": { "id": "5" }, "location": { "id": "5" }, "item": { "id": "67" }, "quantity": 10.0

```

```
3 | }
```

## Retrieving an Assembly Unbuild Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/assemblyUnbuild/4
```

The following code sample shows how to retrieve an assembly unbuild component. The component ID is 7.

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/assemblyUnbuild/4/component/lineNumber=7
```

## Updating an Assembly Unbuild Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/assemblyUnbuild/4
2 | { "quantity": "20"
3 | }
```

The following code sample shows how to update an assembly unbuild component. The component ID is 7.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/assemblyUnbuild/4/component/lineNumber=7/
2 | { "quantity": "21"
3 | }
```

## Billing Account

A billing account record exposes a SuiteBilling account to REST web services. You can enable Billing Accounts without enabling SuiteBilling.

This record:

- is not a subrecord
- has no subrecords

All elements on this record are accessible through REST web services.

The REST API Browser includes information about the field names and field types of the billing account record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [billing account](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the billing account REST record is **billingaccount**.

## Prerequisites

You must [Enabling SuiteBilling Features](#) before you can use this record through REST web services.

## Limitations

A billing account REST record does not show related records.

## Code Sample

```
1 { "startDate": "2019-2-20", "customer": {"id" : 1}, "billingSchedule": {"id": 1} "currency": {"id" : 1}
2 }
```

## Billing Revenue Event

The billing revenue event record is available only when the Advanced Revenue Management (Essentials) feature is enabled. In the UI, you access this record at Lists > Accounting > Revenue Recognition Events > New (Administrator).

Billing revenue events trigger the creation of revenue recognition plans. Each event must have an event type. Event types are not scriptable. To create a revenue recognition event type, go to Lists > Accounting > Revenue Recognition Events > New (Administrator). You must create at least one event type before you can create a billing revenue event.

For help working with this record in the UI, see the help topic [Creating a Custom Revenue Recognition Event](#).

The REST API Browser includes information about the field names and field types of the Billing Revenue Event record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [billingRevenueEvent](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a billing revenue event REST record is **BillingRevenueEvent**.

## Prerequisites

This record is available only when the Product Management and Subscription Billing features are enabled. For more information about how to enable these features, see the help topics [Enabling Project Features](#) and [Enabling SuiteBilling Features](#).

## Limitations

The following fields are read-only for this record in REST:

- eventPurpose (required field)
- eventType (required field)
- projectRevenueRule
- subscriptionLine
- transactionLine

The `eventDate` field is also a required field.

## Additional Information

The **quantity** and **amount** fields appear when the **eventType** field is not set as percent complete. The **cumulativePercentComplete** field appears when **eventType** is set to percent complete.

## Code Samples

The following example shows how to read a billing revenue event:

**Read:**

```
1 | GET {{REST_SERVICES}}/record/v1/BillingRevenueEvent/102?expandSubResources=true
```

## Billing Schedule

The billing schedule record exposes a billing schedule to REST web services. This record:

- is not a subrecord
- has no subrecords

This record enables you to create a billing schedule that can be applied to a sales order, a line item on a sales order, or a project. For details about this type of transaction, see the help topic [Billing Schedules](#).

You must enable the Advanced Billing feature before you can use this record through REST web services.

The REST API Browser includes information about the field names and field types of the billing schedule record and about the HTTP methods, request parameters, and operations available to this record.

For details, see the REST API Browser's [billing schedule](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a billing schedule REST record is **BillingSchedule**.

## Code Samples

These samples show common use cases for billing schedule.

```
1 | POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/BillingSchedule/
2 | { "frequency": { "id": "QUARTERLY", "refName": "Quarterly" }, "inArrears": true, "initialAmount": 100.0, "name": "NetSuite
3 |   Test", "numberRemaining": 4, "repeatEvery": { "id": "1", "refName": "1" }, "scheduleType": { "id": "STD", "refName": "Standard" }
4 | }
5 | GET: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/BillingSchedule/106
6 | PATCH: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/BillingSchedule/106
7 | { "frequency": { "id": "QUARTERLY", "refName": "Quarterly" }, "inArrears": true, "initialAmount": 50.0, "name": "NetSuite
8 |   Test", "numberRemaining": 2, "repeatEvery": { "id": "1", "refName": "1" }, "scheduleType": { "id": "STD", "refName": "Standard" }
9 | }
```

8 | DELETE: <https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/billingSchedule/106>

## Bin

A bin records enable you to organize and track items and their on-hand quantities within a location. This record is available when the Bin Management feature is enabled. For more information about bins, see the help topic [Bin Management](#)

The REST API Browser includes information about the field names and field types of the bin record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [bin](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a bin REST record is **bin**.

## Bin Putaway Worksheet

A bin putaway worksheet record is used to post details of initial bin assignments for items in a location. Aside from received items, you can initially assign bins to items that result from positive adjustments.

This record is available when the Bin Management feature is enabled. For more details about this type of record, see the help topics [Printing Bin Putaway Worksheets](#) and [Updating Bin Putaway Worksheets](#).

The REST API Browser includes information about the field names and field types of the inventory transfer record, and about the HTTP methods, request parameters, and operations available to this record.

For details, see the REST API Browser's [bin worksheet](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a bin putaway worksheet REST record is **binworksheet**.

The record ID for the inventory detail REST subrecord is **inventorydetail**.

## Prerequisites

To put away lot and serialized items in bins, you must enable the Advanced Bin/Numbered Inventory Management feature.

## Code Samples

These sample codes show common use cases for bin putaway worksheet.

Getting a bin putaway worksheet:

```
1 | GET {{REST_SERVICES}}/record/v1/BinWorksheet/102?expandSubResources=true
```

## Bin Transfer

A bin transfer transaction records changes in bin details of inventory items transferred between bins within a location. This record is available when the Bin Management feature is enabled. This record has the following subrecord: inventory detail. For more details about this type of transaction, see the help topic [Bin Transfers](#).

The REST API Browser includes information about the field names and field types of the bin transfer record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [bin transfer](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a bin transfer REST record is **bintransfer**.

The record ID for the inventory detail REST subrecord is **inventorydetail**.

## Prerequisites

To transfer lot and serialized items between bins, you must enable the Advanced Bin/Numbered Inventory Management feature.

## Code Samples

The following example shows how to create a bin transfer:

```
1 | POST {{REST_SERVICES}}/record/v1/bintransfer
2 | { "location": { "refName": "WMS Warehouse 1" }, "memo": "newmemo", "tranId": "BT-1234-new", "inventory": { "items": [ { "description": "BinAssembly", "inventoryDetail": { "customForm": { "id": "-10820", "refName": "Standard Inventory Detail Form" }, "ignoreQtyValidation": false, "inventoryAssignment": { "items": [ { "binNumber": { "id": "14", "refName": "W1-InStage5" }, "internalId": 2061, "inventoryDetail": 114, "inventoryStatus": { "id": "1", "refName": "Good" }, "quantity": 5.0, "toBinNumber": { "id": "13", "refName": "W1-IN-InStage4" }, "toInventoryStatus": { "id": "1", "refName": "Good" } } ] }, "item": { "id": "40", "refName": "BinAssembly" }, "itemDescription": "BinAssembly", "location": { "id": "1", "refName": "WMS Warehouse 1" }, "quantity": 5.0, "toLocation": { "id": "-1", "refName": "-1" } }, "item": { "id": "40", "refName": "BinAssembly" }, "line": 1, "quantity": 5.0 } ] } }
3 | }
```

## Blanket Purchase Order

With this record, you can take advantage of fixed pricing for a preset number of items that you will buy during a specific time period. This approach lets you avoid sporadic pricing negotiations with vendors.

This record is available only when the Blanket Purchase Order feature is enabled at Setup > Company > Setup Tasks > Enable Features (Administrator), on the Transactions subtab.

In the UI, you access this record at Vendor Dashboard > Transactions > Enter Blanket Purchase Order (Administrator). For help working with this record in the UI, see the help topic [Blanket Purchase Order](#).

The **item** element on this record is not accessible through REST web services.

The blanket purchase order record type includes the following actions from the Actions Framework:

- New
- Make Copy
- Make Copy without Schedules
- Email

The REST API Browser includes information about the field names and field types of the blanket purchase order record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [blanketPurchaseOrder](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a blanket purchase order REST record is **blanketPurchaseOrder**.

## Prerequisites

This record is available only when the Blanket Purchase Order feature is enabled at Setup > Company > Setup Tasks > Enable Features (Administrator), on the Transactions subtab.

## Code Samples

The following examples show how to read, create, or update a blanket purchase order:

### Read:

```
1 | GET {{REST_SERVICES}}/record/v1/blanketPurchaseOrder/${transaction-id}
```

### Create:

```
1 | POST {{REST_SERVICES}}/record/v1/blanketPurchaseOrder { "entity": { "id": ${entity-id} }, "item": { "items": [{ "item": { "id": ${item-id} }, "rate": 10, "quantity": 4 } ] }, "memo": "Memo text" }
```

### Update:

```
1 | PATCH {{REST_SERVICES}}/record/v1/blanketPurchaseOrder/${transaction-id} { "memo": "Memo text updated", "item": { "items": [{ "item": { "id": ${item-id} }, "rate": 10 } ] }
```

## BOM

A Bill of Materials, or BOM, record exposes a Bill of Materials, or BOM, to REST web services.

This record:

- is not a subrecord
- has one subrecord: assembly

The REST API Browser includes information about the field names and field types of the BOM record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [bom](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a BOM REST record is **bom**.

## Prerequisites

You must enable the Advanced Bill of Materials feature before you can use this record through REST web services.

## Code Samples

The following samples show common use cases for BOMs. The example ID is 4.

### Creating a BOM Using a POST Request

```

1 POST
2 https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/bom
3 {
4   "name": "bom rest 1",
5   "availableForAllAssemblies": true,
6   "availableForAllLocations": true,
7   "subsidiary": {
8     "items": [
9       {
10        "id": "5"
11      }
12    ]
13  }
14 }
```

### Retrieving a BOM Using a GET Request

```

1 GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/bom/4
```

### Updating a BOM Using a PATCH Request

```

1 PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/bom/4
2 {
3   "assembly": {
4     "items": [
5       {
6         "assembly": {"id": 115},
7         "masterDefault": true
8       },
9       {
10        "assembly": {"id": 67},
11        "defaultforlocation": {
```

```

12         "items": [
13             {
14                 "id": "5"
15             },
16             {
17                 "id": 8
18             }
19         ]
20     }
21 }
22 ]
23 }
24 }

```

## BOM Revision

A BOM revision record exposes a BOM revision to REST web services.

This record:

- is not a subrecord
- has one subrecord: component

The REST API Browser includes information about the field names and field types of the BOM revision record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [bomRevision](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a BOM revision REST record is **bomRevision**.

## Prerequisites

You must enable the Advanced Bill of Materials feature before you can use this record through REST web services.

## Code Samples

The following samples show common use cases for BOM revisions. The example ID is 4.

### Creating a BOM Revision with a Component Using a POST Request

```

1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/bomRevision
2 {
3     "name": "Revision rest 1",
4     "billOfMaterials": {"id": 78},
5     "component": {
6         "items": [
7             {
8                 "item": {"id": 53},

```

```

9      "bomQuantity": 1
10    }
11  ]
12 }
13 }

```

## Retrieving a BOM Revision Using a GET Request

```

1 GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/bomRevision
2 /4

```

## Updating a BOM Revision Using a PATCH Request

The following code sample shows how to edit the quantity on a component line. The component line ID is 7.

```

1 PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/bomRevision/4/component/7
2 {
3   "bomQuantity": 5
4 }

```

## Budget Category

When the Multiple Budgets feature is enabled, you can use budget category values on budget records to categorize types of budgets. In NetSuite OneWorld, budget category values are used to indicate whether budgeting is done at the local subsidiary level in local currency or globally using the root subsidiary's currency. Each budget category is associated with a budget category type of local or global.

After the Multiple Budgets feature is enabled, each budget you set up must be assigned a category. Budget category values allow differentiation between budgets for which other criteria, such as Item, Customer or Project, Class, Department, or Location, are identical. For more information about budget categories in NetSuite, see the help topic [Multiple Budgets and Budget Categories](#). For help creating this record in the user interface, see the help topic [Creating Budget Categories for Local Subsidiary Budgeting](#).

This record isn't a subrecord and doesn't have any subrecords.

The REST API Browser includes information about the field names and field types of the budget category record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [budget category](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a budget category REST record is **budgetcategory**.

## Prerequisites

Budget categories are available only when the Multiple Budgets feature is enabled. Multiple Budgets are on the Enable Features page at Setup > Company > Setup Tasks > Enable Features (Administrator).

The feature is on the **Accounting** subtab under Advanced Features. An administrator must enable the feature.

## Elements with Different Functionality

The **Global** field is accessed in REST using the **budgetType** field. For more information, see [Code Samples](#).

## Usage Notes

Like the **Global** field in the user interface, the **budgetType** field can't be edited after the budget category record is saved.

## Code Samples

The following sample shows how to retrieve a budget category record with an internal ID of 1:

```
1 | GET
2 | {{COMPANY_URL}}/services/rest/record/v1/budgetcategory/1
```

The following sample shows how to retrieve a budget category record with an external ID of 123:

```
1 | GET
2 | {{COMPANY_URL}}/services/rest/record/v1/budgetcategory/eid:123
```

The following sample shows how to create a budget category record:

**Note:** The **Global** field is accessed using the **budgetType** field below. The **budgetType** field is set as true, which means that budgets assigned this budget category have a type of global, and are defined in the base currency of the root parent subsidiary.

```
1 | POST
2 | {{COMPANY_URL}}/services/rest/record/v1/budgetcategory {"externalId": "1", "name": "Test budget rest", "budgetType": true, "isActive": false }
```

The following sample shows how to delete a budget category record with an internal ID of 2:

```
1 | DELETE {{COMPANY_URL}}/services/rest/record/v1/budgetcategory/2
```

The following sample shows how to update the **Name** field on a budget category record with an internal ID of 2:

```
1 | PATCH {{COMPANY_URL}}/services/rest/record/v1/budgetcategory/2 {"name":
2 | "Test name update" }
```

## Campaign

The campaign record exposes a campaign to REST web services.

This record:

- is not a subrecord
- contains the following subrecords:
  - campaignEmail
  - campaignDrip
  - campaignDirectMail
  - campaignEvent
  - defaultEvent (read-only)

For more information about working with campaigns in the UI, see the help topic [Campaign Overview](#).

The REST API Browser includes information about the field names and field types of the campaign record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [campaign](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a campaign REST record is **campaign**.

## Prerequisites

You must enable the Marketing Automation feature before you can use this record through REST web services. For more information, see the help topic [Marketing Automation Overview](#).

## Limitations

Consider the following limitations when using the campaign record in REST.

- The following field is not accessible through REST web services: Template Category.
- The following supporting records are not exposed to REST:
  - Category
  - Audience
  - Vertical
  - Family
  - Search Engine
  - Channel
  - Offer

You can set related fields on the campaign record, but the supporting records cannot be accessed directly or modified.

## Code Samples

The following samples show common actions for a campaign.

## Get Campaign

```
1 | GET <domain>/record/v1/campaign/<ID>
```

## Update the Title Field on an Existing Campaign

```
1 | PATCH <domain>/record/v1/campaign/127
2 | {
3 |   "title": "New Campaign Title"
4 | }
```

## Add an Unscheduled Email Event to an Existing Campaign

```
1 | PATCH <domain>/record/v1/campaign/<ID>
2 | { "campaignEmail": { "items": [{ "campaignGroup": "<Group ID>", "channel": "<Channel ID>", "cost": 11.0, "datescheduled":
   "2023-10-30", "timescheduled": "10:20", "description": "From REST", "promoCode": "<Protocode ID>", "status": "<Status ID>",
   "subscription": "<Subscription ID>", "template": "<Template ID>"
3 |   }] }
4 | }
```

## Campaign Response

The campaign response record exposes a campaign response to REST web services.

This record:

- is not a subrecord
- contains one subrecord: responses

For more information about working with campaign response records in the UI, see the help topic [Tracking Campaign Responses](#).

The REST API Browser includes information about the field names and field types of the campaign response record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [campaign response](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a campaign response REST record is **campaignResponse**.

## Prerequisites

You must enable the Marketing Automation feature before you can use this record through REST web services. For more information, see the help topic [Marketing Automation Overview](#).

## Code Sample

The following sample shows a common action for a campaign response.

## Get Campaign Response

```
1 | GET <domain>/record/v1/campaignResponse/<ID>
```

## Create a Campaign Response

```
1 | POST: /record/v1/campaignresponse
2 | {
3 |     "entity": {"id": 291},
4 |     "leadSource": {"id": 25},
5 |     "campaignEvent": {"id": 17},
6 |     "response": {"id": "RESPONDED"}
7 | }
```

## Cash Refund

The cash refund record exposes a cash refund to REST web services. This record:

- is not a subrecord
- has no subrecords

A cash refund transaction records the return of money to a customer who immediately paid for goods or services using cash, a check or a credit card. For details about this type of transaction, see the help topic [Refunding a Cash Sale](#).

There are no prerequisites for using this record through REST web services.

Using this record requires you to specify an item line number that is 1-indexed.

The REST API Browser includes information about the field names and field types of the cash refund record and about the HTTP methods, request parameters, and operations available to this record.

For details, see the REST API Browser's [cash refund](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

This record has fields related to taxation features. To view the complete list of fields, go to Cash Refund in [SuiteScript Records Browser](#).



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

## Record ID

The record ID for a cash refund REST record is **CashRefund**.

## Usage Notes

To use the **Choose Team** and **Update Customer** fields in the Sales Team subtab, the Team Selling feature must be enabled. To enable Team Selling, go to Setup > Company > Enable Features > CRM. Check the Team Selling box, and then click Save.

The Multi-Partner Management Feature must also be enabled to use the **Update Customer** field in the Relationships subtab. An administrator can enable the Multi-Partner Management feature at Setup > Company > Setup Tasks > Enable Features. Click the CRM subtab, and then under Partners, check the Multi-Partner Management box. Click Save.

## Code Samples

These samples show common use cases for cash refund.

```

1 POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/cashrefund
2 { "account": { "id": "155", "refName": "660 US Bank USD" }, "currency": { "id": "1", "refName": "USA" }, "entity":
  { "id": "110", "refName": "Anonymous Customer" }, "item": { "items": [ { "item": { "id": "98" }, "quantity": 1 } ] }, "location":
  { "id": "6", "refName": "US ONLY LOCATION" }
3 }
4 GET: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/cashrefund/318
5 PATCH: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/cashrefund/318
6 { "account": { "id": "155", "refName": "660 US Bank USD" }, "currency": { "id": "1", "refName": "USA" }, "entity":
  { "id": "110", "refName": "Anonymous Customer" }, "item": { "items": [ { "line": 1, "refName": "Custom", "rate": 49.95, "item":
    { "id": "98" }, "quantity": 2 } ] }, "location": { "id": "6", "refName": "US ONLY LOCATION" }
7 }
8 DELETE: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/cashrefund/318

```

## Setting the Sales Order's Sales Team Members to Match Sales Group 164

```

1 "salesGroup": {
2   "id": 164 }

```

## Update the Customer's Partner Team to Match the Transaction Partners

```

1 "syncPartnerTeams": true

```

## Update the Customer's Sales Team to Match the Transaction Sales Team

```

1 "syncSalesTeams": true

```

## Cash Sale

The cash sale record exposes a cash sale to REST web services.

To access this record in NetSuite, go to Transactions > Sales > Enter Cash Sales.

For information about the cash sale record user interface, see the help topic [Cash Sales](#).

There are no prerequisites for using this record through REST web services.

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the cash sale record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [cash sale](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a cash sale REST record is **cashsale**.

## Usage Notes

To use the **Choose Team** and **Update Customer** fields in the Sales Team subtab, the Team Selling feature must be enabled. To enable Team Selling, go to Setup > Company > Enable Features > CRM. Check the Team Selling box, and then click Save.

The Multi-Partner Management Feature must also be enabled to use the **Update Customer** field in the Relationships subtab. An administrator can enable the Multi-Partner Management feature at Setup > Company > Setup Tasks > Enable Features. Click the CRM subtab, and then under Partners, check the Multi-Partner Management box. Click Save.

## Code Samples

These samples show common use cases for cash sales.

In the following examples,

- <accountID> represents your account ID.
- <recordKey> represents the database key for the invoice.

### Creating a New Cash Sale With an Item

```
1 POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/cashsale { "asofdate": "2021-03-15", "end
  date": "2021-06-15", "entity": { "id": "217" }, "item": { "items": [ { "amount": 1000, "item": { "id": "143" } } ] }, "start
  date": "2021-03-15"
2 }
```

### Creating a New Cash Sale With a Billable Item

```
1 POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/cashsale { "asofdate": "2021-03-15", "entity":
  { "id": "218" }, "itemcost": { "items": [ { "apply": true, "doc": { "id": "305" }, "line": 1 } ] }
2 }
```

### Creating a New Cash Sale With a Billable Expense

```
1 POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/cashsale { "entity": { "id": "217" }, "expcost":
  { "items": [ { "apply": true, "doc": { "id": "303" }, "line": 1 } ] }
2 }
```

```
2 | }
```

## Updating an Existing Cash Sale

**Note:** The start date, end date, and quantity are in line item 1.

```
1 | PATCH: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/cashsale/<recordKey> { "item": { "items":
2 | [ { "line": 1, "quantity": 5 } ] }, "startDate": "2021-03-05", "endDate": "2021-04-05"
  | }
```

## Setting the Sales Order's Sales Team Members to Match Sales Group 164

```
1 | "salesGroup": {
2 |   "id": 164 }
```

## Update the Customer's Partner Team to Match the Transaction Partners

```
1 | "syncPartnerTeams": true
```

## Update the Customer's Sales Team to Match the Transaction Sales Team

```
1 | "syncSalesTeams": true
```

## Change Order

A change order record exposes a SuiteBilling change order to REST web services. This record has the following subrecords:

- subLine
- newSubLine
- renewalSteps

The REST API Browser includes information about the field names and field types of the change order record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [change order](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the change order REST record is subscriptionChangeOrder.

## Prerequisites

You must [Enabling SuiteBilling Features](#) before you can use this record through REST web services.

## Code Sample

```
1 POST 'https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/subscriptionchangeorder' \ "action": "MODI
  FY_PRICING", "subscription": 208, "customer": 5, "billingAccount": 2, "subline": { "items": [ { "apply": true, "sequence": 1, "pri
  ceplannew": 541 } ] }
2 }
```

## Charge

A charge record exposes a SuiteBilling charge to REST web services. This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the charge record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [charge](#) reference page.

## Record ID

The record ID for the charge REST record is **charge**.

## Prerequisites

You must [Enabling SuiteBilling Features](#) before you can use this record through REST web services.

## Additional Details

You must specify the stage field as an enumeration string. Possible values can be obtained through the Metadata Catalog. Charge type must be specified as an internal ID. You can obtain possible values through SuiteScript while creating a charge through the User Interface.

## Code Sample

This sample shows a common billing usage case.

```
1 POST https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/charge
2 { "billTo": 5, "stage": "READY_FOR_BILLING", "chargeDate": "2020-07-14", "chargeType": -19, "billingItem": 5, "rate": 100.0, "quan
  tity": 1
3 }
```

## Check

A check record exposes a check transaction to REST web services.

This record is not a subrecord but has two subrecords: check item and check expense.

All elements on this record are accessible through REST web services.

The REST API Browser includes information about the field names and field types of the check record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [check](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the check REST record is **check**.

The record IDs for the check REST subrecords are:

- **item**
- **expense**

## Limitations

Voiding of checks is not supported in REST web services. You can only void a check in the UI, which creates a voided journal entry and the original check remains and has a **Voided** status.

## Code Samples

These samples show common use cases for check REST methods.

### Getting OpenAPI Metadata from the Metadata Catalog

```
1 | GET https://test.netsuite.com/services/rest/record/v1/metadata-catalog/check
```

### Fetching a Check Record

```
1 | GET https://test.netsuite.com/services/rest/record/v1/check/<id>
```

### Creating a Check Record

```
1 | POST https://test.netsuite.com/services/rest/record/v1/check
2 | { "account": "1", "balance": -171.5, "cleared": false, "createdDate": "2023-06-13T20:32:00Z", "currency": "1", "customForm": { "id": "49", "refName": "Standard Check" }, "entity": "40", "exchangeRate": 1.0, "expense": { "items": [ { "account": { "id": "64" }, "amount": 333.00, "taxCode": { "id": "99" } } ] }, "oldaccount": "151", "postingPeriod": { "id": "553", "refName": "Jun 2023" }, "prevDate": "2023-06-13", "toBePrinted": false, "tranDate": "2023-06-13", "tranId": "1"
3 | }
```

### Updating a Check Record

```
1 | PATCH https://test.netsuite.com/services/rest/record/v1/check/<id>
```

```

2 | { "account": "1", "memo": "new memo"
3 | }

```

## Deleting a Check Record

```

1 | DELETE https://test.netsuite.com/services/rest/record/v1/check/<id>

```

## Class

Classes (or classifications) can be used to identify and categorize records in your NetSuite account. You can also define classifications for your organization's needs by creating custom segments. Classifications enable you to better organize data and preserve accuracy.

For information about working with classes in the UI, see the help topic [Classifications in NetSuite](#).

The REST API Browser includes information about the field names and field types of the class record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [classification](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the classification REST record is **classification**.

The classification record has the following subrecord:

- classTranslation

## Limitations

Read-only sublists, including system notes, are not supported in REST.

## Code Sample

```

1 | { "isInactive": false, "name": "REST test 1693820062", "subsidiary": { "items": [ { "id": 2 }, { "id": 4 } ] }, "parent": { "id": 1
2 | }, "includeChildren": true

```

## Commerce Category

A commerce category record exposes a commerce category to REST web services.

This record:

- is not a subrecord
- has no subrecords

- has only sublists

The REST API Browser includes information about the field names and field types of the commerce category record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [commerce category](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a commerce category REST record is **commercecategory**.

## Prerequisites

You must [Enable Commerce Categories Feature](#) for any interaction with the record. The account must also contain a Commerce Catalog record to reference.

## Inaccessible Elements

The following sublists are not accessible through REST web services:

- URL Fragment Aliases sublist is not exposed
- "isCurrentlyEffective" is internal only

## Elements with Different Functionality

Updating the tags collection using REST does not delete removed tags, it only adds new tags and updates existing tags.

## Additional Details

The "displayinSite" field values must be "T" or "F", not standard boolean.

The "sitemapPriority" field values must be a string with possible values between "0.0" and "1.0" incrementing by tenths. For example, ("0.1", "0.2", "0.3", etc.). This behavior is the same across all channels, not specific to REST web services.

## Code Sample

### Updating a Commerce Category Record

This example shows how to update a commerce category record.

```
1 HTTP Method: PATCH
2 URL: {NETSUITE_DOMAIN}/services/rest/record/v1/commercecategory/{id}
3 Request Parameters: {id} - Commerce Category ID to update
4 HTTP Request Headers: Content-Type: application/json
5 { "name": "REST Category", "catalog": 1, "urlFragment": "rest-category", "isInactive": false, "addToHead": "string", "metaDe
  scription": "string", "metaKeywords": "string", "pageTitle": "string", "pageHeading": "string", "externalId": "string", "de
```

```

scription": "string", "startDate": "2023-07-13T16:37:57.000Z", "endDate": "2024-07-13T16:37:57.000Z", "sequenceNum
ber": 0, "displayInSite": "T", "sitemapPriority": "0.0", "thumbnail": 5034, "pageBanner": 5034, "subcategories": { "items":
  [ { "subcategory": "2", "pageHeadingOverride": "string", "addToHeadOverride": "string", "displayInSiteOverride": "F", "se
quenceNumber": 0, "nameOverride": "Child REST Category", "urlFragmentOverride": "child-rest-category", "pageTitleOver
ride": "string", "metaDescriptionOverride": "string", "metaKeywordsOverride": "string", "sitemapPriorityOverride": "1.0", "de
scriptionOverride": "string", "pageBannerOverride": 3323, "thumbnailOverride": 3323 }, { "subcategory": "3", "sequen
ceNumber": 1, "nameOverride": "Another Child REST Category", "urlFragmentOverride": "another-child-rest-category"
} ] }, "items": { "items": [ { "primaryCategory": true, "sequenceNumber": 0, "item": 206 }, { "primaryCategory": true, "se
quenceNumber": 1, "item": 227 } ] }, "translations": { "items": [ { "metaDescription": "string", "name": "string", "de
scription": "string", "pageTitle": "string", "pageHeading": "string", "locale": "fr_CA", "addToHead": "string", "metaKey
words": "string", "thumbnail": 3323, "pageBanner": 3323 }, { "metaDescription": "string", "name": "string", "pageTi
tle": "string", "pageHeading": "string", "locale": "es_ES", "metaKeywords": "string" } ] }, "tags": { "items":
  [ { "name": "tag1", "id": 1 }, { "name": "tag2", "id": 2 } ] }
6 | }

```

## Competitor

You can create competitor records to track how other businesses in your industry impact your sales. In the UI, this is a user defined list at Lists > Relationships > Competitors > New.

Use the competitor record to create new competitors and expose them to REST web services.

To learn more, see the help topic [Competitors](#).

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the competitor record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [competitor](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a competitor REST record is **competitor**.

## Prerequisites

You must enable Permissions > Lists > Competitors before you can use this record through REST web services.

## Elements Not Accessible Through REST Web Services

The following elements are not accessible through REST web services:

- opportunities

## Usage Notes

Review the following notes on working with the competitor REST record:

- name is required, all other fields are optional

## Code Sample

The following samples show common use cases for competitor.

### Creating a Competitor

```
1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/competitor
2 Body: { "name": "A Competitor", "description": "Description of competitor", "strengths": "Competitor's strengths", "weaknesses": "Competitor's weaknesses", "strategy": "Strategy for winning against competitor", "productService": "Competitor's products and services", "isInactive": false
3 }
```

### Retrieving a Competitor

```
1 GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/competitor/2
```

### Updating a Competitor

```
1 PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/competitor/2
2 Body:
3 { "name": "Updated Competitor", "description": "Updated description of competitor", "strengths": "Updated Competitor's strengths", "weaknesses": "Updated competitor's weaknesses", "strategy": "Updated strategy for winning against competitor", "productService": "Updated competitor's products and services", "isInactive": true
4 }
```

### Deleting a Competitor

```
1 DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/competitor/2
```

## Consolidated Exchange Rate

A consolidated exchange rate record exposes a consolidated exchange rate to REST web services.

Consolidated exchange rate records ensure that currency amounts translate properly from child to parent subsidiaries for consolidated reports. NetSuite creates initial consolidated exchange rates when associated subsidiaries and accounting periods are created.

To access this record, go to Lists > Accounting > Consolidated Exchange Rates.

For more information about consolidated exchange rates in NetSuite, see the help topic [Consolidated Exchange Rates](#).

The REST API Browser includes information about the field names and field types of the consolidated exchange rate record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [consolidatedExchangeRate](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a consolidated exchange rate REST record is **ConsolidatedExchangeRate**.

## Prerequisites

You must enable the Multiple Currencies feature before you can use this record through REST web services.

## Elements with Different Functionality

The following fields have different functionality through REST web services:

- accountingBook (Read only)
- fromCurrency (Read only)
- fromSubsidiary (Read only)
- isDerived (Read only)
- isEliminationSubsidiary (Read only)
- isPeriodClosed (Read only)
- periodStartDate (Read only)
- postingPeriod (Read only)
- toCurrency (Read only)
- toSubsidiary (Read only)

## Usage Notes

The Full Multi-Book Accounting feature must be enabled if you want to view or edit the following field:

- accountingBook

## Code Sample

This sample shows how to update the historical exchange rate using a PATCH request:

```
1 PATCH {{REST_SERVICES}}/record/v1/consolidatedExchangeRate/10
2 {
3   "historicalRate": 0.8655
4 }
```

## Contact

A contact record exposes a contact to REST web services.

This record:

- is not a subrecord
- has no subrecords

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the contact record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [contact](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

Tax-related fields are not supported in REST.

## Record ID

The record ID for a contact REST record is **contact**.

## Code Sample

In the following example, *<accountID>* represents your account ID.

HTTP request: `https://<accountID>/services/rest/record/v1/contact/973`

```
1 { "firstName": "Peter", "contactSource": "99994", "comments": "Works directly with all key decision-makers."
2 }
```

## Contact Category

A contact category record exposes a contact category to REST web services.

This record:

- is not a subrecord
- has no subrecords

All elements on this record are accessible through REST web services.

Record actions supported for this record type include delete, submitnew, and submitter. For more information, see the help topic [Supported Record Actions](#).

The REST API Browser includes information about the field names and field types of the contact category record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [contactCategory](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a contact category REST record is **contactcategory**.

## Code Sample

In the following example, *<accountID>* represents your account ID.

HTTP request: `https://<accountID>/services/rest/record/v1/contactcategory/4`

```
1 { "name": "ContactCategoryFour", "externalid": "3929345", "private": true
2 }
```

## Contact Role

A contact role record exposes a contact role to REST web services.

This record:

- is not a subrecord
- has no subrecords

All elements on this record are accessible through REST web services.

The REST API Browser includes information about the field names and field types of the contact role record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [contactRole](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a contact role REST record is **contactrole**.

## Code Sample

In the following example, `<accountID>` represents your account ID.

HTTP request: `https://<accountID>/services/rest/record/v1/contactrole/1`

```
1 { "name": "ContactRoleOne", "description": "Key Contact", "isinactive": true
2 }
```

## Cost Category

A cost category record exposes a cost category to REST web services. This record is not a subrecord. This record does not have any subrecords.

For help working with this record in the UI, see the help topic [Creating Cost Categories](#).

The REST API Browser includes information about the field names and field types of the cost category record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [costCategory](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a cost category REST record is **CostCategory**.

## Prerequisites

You must enable either the Standard Costing or Landed Cost feature before you can use this record through REST web services.

## Code Samples

The following samples show common use cases for cost categories. The example ID is 4.

### Creating a Cost Category Using a POST Request

This sample shows how to create a cost category when you are using the Standard Costing feature only.

```
1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/costCategory
2 { "itemCostType": { "id": "MATERIAL" }, "name": "Test Name"
3 }
```

This sample shows how to create a cost category when you are using the Landed Cost feature only.

```
1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/costCategory
2 { "itemCostType": { "id": "LANDED" }, "name": "Air Freight Charges", "account": { "id": 69 }
3 }
```

### Retrieving a Cost Category Using a GET Request

```
1 GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/costCategory/4
```

### Updating a Cost Category Using a PATCH Request

This sample shows how to update a cost category when you are using the Standard Costing feature only.

You can update the name only. You cannot change the cost type after you create the cost category.

```
1 PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/costCategory/4
2 { "name": "New Name",
3 }
```

This sample shows how to update a cost category when you are using the Landed Cost feature only.

You can update the name and expense account ID only. You cannot change the cost type after you create the cost category.

```
1 PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/costCategory/4
2 { "name": "FedEx Charges", "account": { "id": 65 }
3 }
```

### Deleting a Cost Category Using a DELETE Request

```
1 DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/costCategory/4
```

## Coupon Code

Promotions enable you to track the source of revenue and to offer discounts in the form of coupons. Each promotion has a promotion code that can be applied to transactions and campaigns.

This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the coupon code record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [couponCode](#) reference page.

For information about coupon codes, see the help topic [Coupon Code](#).

For information about working with coupon codes in the UI, see the help topic [Coupon Codes](#).

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the coupon code REST record is **couponcode**.

## Prerequisites

You must enable the SuitePromotions feature before you can use this record through REST web services.

## Elements with Different Functionality

The following fields have different functionality through REST web services:

- used
- useCount

## Additional Details

For multi-use coupon code type promotions:

- The promotion must have at least one coupon code associated to it so that it is usable.  
If you remove all the coupon codes, you will see an empty coupon code field in the UI, and from the REST point of view, the code field will not be returned with the promotioncode record.  
However, you can add the coupon codes through the user interface or through REST by updating the code field in the promotioncode record.
- You can update the coupon code field using a PATCH request to the promotioncode record.

For single-use coupon code type promotions:

- The promotion must have at least one coupon code associated to it so that it is usable.  
If you remove all the coupon codes, you will have an empty list of coupon codes.  
However, you can add the coupon codes through the user interface or through REST.
- You cannot update the coupon code field using a PATCH request to the promotioncode record.

## Code Sample

In the following example, 123 represents the promotion ID.

The following code shows how to create a coupon code:

```
1 | POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/couponCode
2 | { "promotion": {"id": 123}, "code": "ABC-ABC-100"
3 | }
```

In the following example, 206 represents the promotion ID and 15 represents the recipient ID.

The following code shows how to update a coupon code:

```
1 | PATCH: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/couponCode/206
2 | { "promotion": {"id": 123}, "code": "ABC-CDE-200", "dateSent": "2024-06-18", "recipient": {"id": "15"},
3 | }
```

**Note:** When updating the coupon code records, consider the difference between single-use and multi-use coupon code type promotions.

The following code shows how to obtain a coupon code using a GET Request:

```
1 | GET: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/couponCode/206
```

The following code shows how to delete a coupon code:

```
1 | DELETE: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/couponCode/206
```

**Note:** If a coupon code is already being used in a transaction, you won't be able to remove the promotions record.

## Credit Card Charge

A credit card charge record exposes outgoing credit card transactions to REST web services.

This record is not a subrecord but has two subrecords: credit card charge expense and credit card charge item.

The REST API Browser includes information about the field names and field types of the credit card charge record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [credit card charge](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a credit card charge REST record is **creditCardCharge**.

The record IDs for the credit card charge REST subrecords are:

- **item**

## ■ expense

## Prerequisites

Before you can use this record through REST web services, you must enable and set up the SuiteTax feature as legacy tax fields are not available over REST. See the help topic [Enabling the SuiteTax Feature](#).

## Limitations

Updating legacy tax fields is not supported.

## Code Samples

These samples show common use cases for credit card charge REST methods.

## Getting OpenAPI Metadata from the Metadata Catalog

```
1 | GET https://test.netsuite.com/services/rest/record/v1/metadata-catalog/creditcardcharge
```

## Fetching a Credit Card Charge Record

```
1 | GET https://test.netsuite.com/services/rest/record/v1/creditcardcharge/<id>
```

## Creating a Credit Card Charge Record

```
1 | POST https://test.netsuite.com/services/rest/record/v1/creditcardcharge
2 | {
3 |   "account": "123",
4 |   "entity": "26", "expense": { "items": [ { "account": "58", "amount": 50.00 } ] }
5 | }
```

## Updating a Credit Card Charge Record

```
1 | PATCH https://test.netsuite.com/services/rest/record/v1/creditcardcharge/<id>
2 | { "account": "204", "entity": "10"
3 | }
```

## Deleting a Credit Card Charge Record

```
1 | DELETE https://test.netsuite.com/services/rest/record/v1/creditcardcharge/<id>
```

## Credit Card Refund

A credit card refund record exposes incoming credit card transactions to REST web services.

This record is not a subrecord but has two subrecords: credit card refund expense and credit card refund item.

With the exception of tax-related fields, all elements on this record are accessible through REST web services.

The REST API Browser includes information about the field names and field types of the credit card refund record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [credit card refund](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a credit card refund REST record is **creditCardRefund**.

The record IDs for the credit card refund REST subrecords are:

- **item**
- **expense**

## Prerequisites

Before you can use this record through REST web services, you must enable and set up the SuiteTax feature as legacy tax fields are not available over REST. See the help topic [Enabling the SuiteTax Feature](#).

## Limitations

Updating legacy tax fields is not supported.

## Code Samples

These samples show common use cases for credit card refund REST methods.

### Getting OpenAPI Metadata from the Metadata Catalog

```
1 | GET https://test.netsuite.com/services/rest/record/v1/metadata-catalog/creditcardrefund
```

### Fetching a Credit Card Refund Record

```
1 | GET https://test.netsuite.com/services/rest/record/v1/creditcardrefund/<id>
```

### Creating a Credit Card Refund Record

```
1 | POST https://test.netsuite.com/services/rest/record/v1/creditcardrefund
2 | { "account": "123", "entity": "26", "expense": { "items": [ { "account": "58", "amount": 50.00 } ] }
3 | }
```

## Updating a Credit Card Refund Record

```
1 | PATCH https://test.netsuite.com/services/rest/record/v1/creditcardrefund/<id>
2 | { "account": "204", "entity": "10"
3 | }
```

## Deleting a Credit Card Refund Record

```
1 | DELETE https://test.netsuite.com/services/rest/record/v1/creditcardrefund/<id>
```

## Credit Memo

A credit memo record exposes a credit memo to REST web services. A credit memo is used to refund or credit a customer account. For more information about credit memos and how they function in the user interface, see the help topic [Customer Credit Memos](#).

 **Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the credit memo record and about the HTTP methods, request parameters, and operations available to this record.

For details, see the REST API Browser's [credit memo](#) reference page.

For information about using the REST API browser, see the help topic [The REST API Browser](#).

This record:

- is not a subrecord
- does not have any subrecords

## Record ID

The record ID for the credit memo record is **creditmemo**.

## Prerequisites

Prerequisites include:

- REST Web Services feature
- Two-factor authentication (for REST)
- Accounts receivable (A/R) for the credit memo record

## Usage Notes

To use the **Choose Team** and **Update Customer** fields in the Sales Team subtab, the Team Selling feature must be enabled. To enable Team Selling, go to Setup > Company > Enable Features > CRM. Check the Team Selling box, and then click Save.

The Multi-Partner Management Feature must also be enabled to use the **Update Customer** field in the Relationships subtab. An administrator can enable the Multi-Partner Management feature at Setup > Company > Setup Tasks > Enable Features. Click the CRM subtab, and then under Partners, check the Multi-Partner Management box. Click Save.

## Code Sample

The following example is for creating a credit memo record.

```
1 | Create
2 | POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo { "entity": { "id": "4" }, "item": { "items":
   | [ { "amount": 1000.0, "item": { "id": "5" } } ] }, "subsidiary": { "id": "1" } "location": "5"
3 | } Get
4 | GET: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/101 Update
5 | PATCH: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/101
6 | { "memo": 'test memo' }
```

## Setting the Sales Order's Sales Team Members to Match Sales Group 164

```
1 | "salesGroup": {
2 |   "id": 164 }
```

## Update the Customer's Partner Team to Match the Transaction Partners

```
1 | "syncPartnerTeams": true
```

## Update the Customer's Sales Team to Match the Transaction Sales Team

```
1 | "syncSalesTeams": true
```

## Currency

A currency record exposes a currency to REST web services.

Currency records are created for the different currencies you want to use in your transactions and for your subsidiaries if you use NetSuite OneWorld.

To access this record in NetSuite, go to Lists > Accounting > Currencies > New.

For more information about currencies in NetSuite, see the help topic [Currency Management](#).

The REST API Browser includes information about the field names and field types of the currency record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [currency](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a currency REST record is **Currency**.

## Prerequisites

You must have the Multiple Currencies feature enabled before you can use this record through REST web services. For more information about this feature, see the help topic [Enabling the Multiple Currencies Feature](#).

## Elements with Different Functionality

The following fields have different functionality through REST web services:

- formatSample (Read only)
- isAnchorCurrency (Read only)
- isBaseCurrency (Read only)
- lastModifiedDate (Read only)

## Usage Notes

The Currency Exchange Rate Integration feature must be enabled to view and edit the following fields:

- includeInFxRateUpdates
- fxRateUpdateTimeZone

The Use Triangulation Calculation By NetSuite accounting preference must be enabled to view and edit the following field:

- isAnchorCurrency

## Code Sample

This sample shows how to update the Override Currency Format box from false to true:

```
1 PATCH {{COMPANY_URL}}/services/rest/record/v1/currency/2
2 {
3   "overrideCurrencyFormat": true
4 }
```

## Currency Rate

A currency rate record exposes a currency exchange rate to REST web services.

Currency exchange rates are used to convert foreign currencies to base currencies, providing default rates for transactions in currencies other than the base currency. Exchange rates are expressed in terms of base currency units per foreign currency (source) unit.

To access this record in NetSuite, go to Lists > Accounting > Currency Exchange Rates > New.

For more information about currency exchange rates in NetSuite, see the help topic [Currency Exchange Rates](#).

The REST API Browser includes information about the field names and field types of the currency exchange rate record and about the HTTP methods, request parameters, and operations available to this record. For details, see the Rest API Browser's [currency rate](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a currency rate REST record is **currencyrate**.

## Prerequisites

You must enable the Multiple Currencies feature before you can use this record through REST web services. For more information about this feature, see the help topic [Enabling the Multiple Currencies Feature](#).

## Accessible Elements

The following fields are not accessible through REST web services:

- Previous Effective Date
- Previous Exchange Rate

## Code Sample

The following code sample shows how to create a currency exchange rate record:

```
1 POST: {{COMPANY_URL}}/services/rest/record/v1/currencyrate
2 {
3   "baseCurrency": "1",
4   "transactionCurrency": "3",
5   "exchangeRate": 1.9,
6   "effectiveDate": "2023-11-26",
7   "externalId": "1"
8 }
```

## Customer

A customer record exposes a customer to REST web services.

This record:

- is not a subrecord
- has the following subrecords:
  - addressbook
  - campaigns
  - contactroles

- currencylist
- grouppricing
- itempricing
- partners
- salesteam
- subscriptionmessagehistory
- subscriptions
- taxregistration

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the customer record, and about the HTTP methods, request parameters, and operations available to this record.

For details, see the REST API Browser's [customer](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the customer REST record is **customer**.

## Inaccessible Elements

The following sublists are not accessible through REST web services:

- bulkmerge
- creditcards
- download
- eftacct
- itemorders
- paymentinstruments
- referrerlist
- submachine
- upsell

## Elements With Limited Functionality

The create operation is not supported for the campaigns and contactroles elements.

## Code Sample

The following code sample shows how to create a customer record:

```

1 POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/customer
2 { "companyName": "Company 1615554322", "email": "customer@example.com"
3 }

```

## Customer Category

NetSuite exposes the customer category record to REST web services. Customer category defines a list of values that are used by the customer record to set the type of customer.

### Record ID

The record ID for the customer category REST record is **customerCategory**.

### Prerequisites

There are no prerequisites required for using this record through REST web services.

The customer category record is not a subrecord and is used on the customer record and other CRM records.

There are no elements on this record that are not accessible through REST web services.

The REST API Browser includes information about the field names and field types of the estimate record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [customer category](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Code Samples

The code samples in this section show common use cases for customer category items.

### Creating a Customer Category

```

1 POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/customercategory
2 { "name": "Customer Category 1"
3 }

```

### Updating a Customer Category

```

1 PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/customercategory/{id}
2 { "name": "Customer Category 2"
3 }

```

## Customer Deposit

NetSuite exposes the customer deposit record to REST web services. A customer deposit transaction records the funds received when a customer makes an advance payment for an order. This payment is

recorded in the general ledger as a liability until the goods or services are delivered, and does not affect the customer's accounts receivable balance. After the order is filled, the deposit is applied against the invoice.

For more information about customer deposits, see the help topic [Customer Deposits](#).

There are no elements on this record that are not accessible through REST web services.

The REST API Browser includes information about the field names and field types of the customer deposit record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [customerDeposit](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the customer deposit REST record is **customerDeposit**.

## Sublist

The customer deposit record has a sublist of the **accountingBookDetail** subrecord.

## Prerequisite

Before you work with customer deposits, enable the A/R feature.

## Code Samples

The code samples in this section show common use cases for customer deposits.

### Creating a Customer Deposit

```
1 POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/customerdeposit
2 { "customer":{"id": 6}, "currency":{"id": 1}, "exchangerate": 1.00, "payment": 10.00, "paymentOption": {"id": 4}
3 }
```

### Updating a Customer Deposit

```
1 PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/customerdeposit/{id}
2 { "payment": 12.0, }
```

## Customer Message

A customer message record exposes a customer message to REST web services.

For more information about customer messages and how they function in the user interface, see the help topic [Setting Up Accounting Lists](#).

To access this record in NetSuite, go to Setup > Accounting > Accounting Lists.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the customer message record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [customer message](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#)

## Record ID

The record ID for the customer message REST record is **customermessage**.

## Code Sample

The following samples show common use cases for a customer message record. The example ID is 6.

### Retrieving Customer Message Metadata

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/metadata-catalog/customermessage
```

### Creating a Customer Message Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/customermessage
2 | { "description": "example message description", "isInactive": false, "name": "ExampleCustomerMessage", "preferred": false
3 | }
```

### Deleting a Customer Message Using a DELETE Request

```
1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/customermessage/6
```

## Customer Payment

NetSuite exposes the customer payment record to REST web services. A customer payment transaction records a customer payment and applies it to the appropriate invoice or cash sale, decreasing the amount due, and tracking revenue.

The customer payment record has no subrecords.

There are no prerequisites for using this record through REST web services.

For more information about the customer payment record, see the help topic [Customer Payments](#).

The REST API Browser includes information about the field names and field types of the customer payment record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [customerPayment](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the customer payment REST record is **customerpayment**.

## Code Samples

The following code samples show how to work with the customer payment record.

### Creating a Customer Payment Record

```

1  {
2    "account": {
3      "id": "1"
4    },
5    "apply": {
6      "items": [
7        {
8          "amount": 200.0,
9          "apply": true,
10         "doc": {
11           "id": "104"
12         },
13         "line": 0
14       }
15     ]
16   },
17   "currency": {
18     "id": "1"
19   },
20   "customer": {
21     "id": "4"
22   },
23   "memo": "10",
24   "payment": 250,
25   "toBeEmailed": false
26 }
```

### Getting an Existing Customer Payment by ID

```

1  HTTP method: GET
2  URL: {{COMPANY_URL}}/services/rest/record/v1/customerpayment/{{customerPaymentId}}
```

### Changing the Applied Invoice Amount by Updating an Existing Customer Payment by ID

```

1  HTTP method: PATCH
2  URL: {{COMPANY_URL}}/services/rest/record/v1/customerpayment/{{customerPaymentId}}
3  Request body: { "apply": { "items": [ { "doc": { "id": "4" }, "amount": 200 } ] }, "customer": { "id": "4" } }
4  }
```

### Adding an Invoice to Apply by Updating an Existing Customer Payment by ID

```

1  HTTP method: PATCH
```

```

2 | URL: {{COMPANY_URL}}/services/rest/record/v1/customerpayment/{{customerPaymentId}}
3 | Request body: { "apply": { "items": [ { "doc": { "id": "104" }, "amount": 10 } ] }, "customer": { "id": "4" }
4 | }

```

## Unapply an Invoice by Updating an Existing Customer Payment by ID

```

1 | { "apply": { "items": [ { "doc": { "id": "104" }, "apply": false } ] }, "customer": { "id": "4" }
2 | }

```

```

1 | { "apply": { "items": [ { "doc": { "id": "104" }, "amount": 0 } ] }, "customer": { "id": "4" }
2 | }

```

## Customer Refund

NetSuite exposes the customer refund record to REST web services. A customer refund transaction records the return of funds to a customer who paid for goods or services using cash, a check, or a payment card. The refund is generally made in cash or by check.

For more information about customer refunds, see the help topic [Customer Refunds](#).

The REST API Browser includes information about the field names and field types of the customer refund record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [customerRefund](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the customer refund REST record is **customerRefund**.

## Subrecords and Sublists

The customer refund record has the following sublists and subrecord:

- The apply sublist of the CustomerRefundApplyLine subrecord.
- The deposit sublist of the CustomerRefundDepositLine subrecord.
- The TransactionAccountingBookDetail subrecord.

## Prerequisite

Before you work with customer refunds, enable the A/R feature.

## Limitations

Certain elements on the customer refund record are not accessible through REST web services: the deposit sublist in the GET request is empty. The deposit sublist is hidden in edit mode in UI, and the applied deposit is represented by Deposit Application in apply sublist. This is how sublists are displayed in GET response in REST.

Certain elements on the customer refund record have a different level of functionality than the rest of the record:

- The deposit sublist is always empty in response for GET request and cannot be updated.
- Lines on the deposit sublist are not identified by sublist line number, but by the doc field as the key. See the [Creating a Customer Refund](#) code sample.
- Lines on the apply sublist are not identified by the line number, but by the doc field as the key with the line as a secondary key. The line represents the transaction line ID, not the line number on the sublist. See the [Creating a Customer Refund](#) code sample.

## Code Samples

The code samples in this section show common use cases for customer refunds.

### Creating a Customer Refund

```
1 POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/customerrefund
2 { "customer": { "id": "6"}, "deposit": { "items": [ { "doc": { "id": "60"}, "apply": true, "amount": 25.00 } ] }, "apply": { "items":
  [ { "doc": { "id": "59"}, "line": 1, "apply": true, "amount": 4.20 }, { "doc": { "id": "56"}, "line": 1, "apply": true, "amount": 5.00 } ] }
3 }
```

### Updating a Customer Refund

```
1 PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/customerrefund/{id}
2 { "memo": "Updated refund"
3 }
```

## Customer Status

A customer status record exposes a customer status to REST web services. It describes a lead, prospect, or a customer's stage in the sales cycle.

For more information about customer statuses and how they function in the user interface, see the help topic [Customer Statuses](#).

To access this record in NetSuite, go to Setup > Sales > Setup Tasks > Customer Statuses > New.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the customer status record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [customer status](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#)

## Record ID

The record ID for the customer status REST record is **customerstatus**.

## Code Sample

The following samples show common use cases for a customer status record. The example ID is 6.

### Retrieving Customer Status Metadata

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/metadata-catalog/customerstatus
```

### Updating a Customer Status Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/customerstatus/6
2 | { "probability": 99.0
3 | }
```

### Deleting a Customer Status Using a DELETE Request

```
1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/customerstatus/6
```

## Customer Subsidiary Relationship

NetSuite exposes the customer subsidiary relationship record to REST web services. This record enables you to manage a specific customerSubsidiaryRelationship record. You can also get a list of customer subsidiary relationship records. This record is not accessible through the user interface.

The REST API Browser includes information about the field names and field types of the customer subsidiary relationship record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [customerSubsidiaryRelationship](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

### Record ID

The record ID for the customer subsidiary relationship REST record is **customerSubsidiaryRelationship**.

### Prerequisites

You must either use NetSuite OneWorld or have the Subsidiaries hidden feature enabled before you can use this record through REST web services.

### Updatable Fields on the customerSubsidiaryRelationship Record

You can update the following fields on the customerSubsidiaryRelationship record:

- custom fields (available on the customer record, Subsidiary subtab)

- (externalid) - external id

## Supported Operations

The following operations are supported for REST web services:

- GET (Read, Search for a list of customer subsidiary relationship records)
- POST (insert a record)
- DELETE (a specific record)
- GET (Read, Search a specific record)
- PATCH (update a specific record)
- PUT (insert or update a specific record)

Tax-related fields are not supported in REST.

## Code Samples

The following code sample shows how to get a specific customerSubsidiaryRelationship record:

```
1 | GET https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/customersubsidiaryrelationship?q=entity
2 | EQUAL 13 AND subsidiary EQUAL 5
```

The following code sample shows how to get the ID of a specific customerSubsidiaryRelationship record:

```
1 | POST https://123456.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql?limit=5
```

```
1 | Header:
2 | 'prefer: transient'
3 | 'Content-Type: application/json'
4 | Body:
5 | '{ "q": "SELECT id FROM customerSubsidiaryRelationship WHERE entity = 13 AND subsidiary = 5"
6 | }'
```

The following code sample shows how to set the externalId for a specific customerSubsidiaryRelationship record:

```
1 | PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/customersubsidiaryrelationship/1
```

```
1 | Header:
2 | 'Content-Type: application/json'
3 | Body:
4 | '{ "externalId": "VSR1" }'
```

## Department

Departments can be used to identify and categorize records in your NetSuite account. You can also define departments for your organization's needs by creating custom segments. Departments enable you to better organize data and preserve accuracy.

For information about working with departments in the UI, see the help topic [Classifications in NetSuite](#).

The REST API Browser includes information about the field names and field types of the department record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [department](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the department REST record is **department**.

The department record has the following subrecord:

- classTranslation

## Limitations

Read-only sublists, including system notes, are not supported in REST.

## Code Sample

```
1 { "isInactive": false, "name": "REST test 1693820309", "subsidiary": { "items": [ { "id": 2 }, { "id": 4 } ] }, "parent": { "id": 1
2 }, "includeChildren": true
}
```

## Deposit

A deposit record exposes a deposit to REST web services.

In NetSuite, record deposits to your bank accounts to capture customer payments and other monies received in the course of doing business. By recording funds deposited, you can accurately track income. For information about working with this record in the UI, see the help topic [Deposits](#).

This record:

- is not a subrecord
- has the following three sublists:
  - Deposit Cashback (ID: cashback)
  - Deposit Other (ID: other)
  - Deposit Payment (ID: payment)

The REST API Browser includes information about the field names and field types of the deposit record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [deposit](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Usage Notes

The **To Be Printed** field is not exposed to REST web services.

In both the UI and in REST web services, the **Exchange Rate** field is a create-time-only field that is used to specify the initial exchange rate with respect to the base currency. After the initial exchange rate value is set, this value does not get updated to reflect real-time exchange rates.

## Record ID

The record ID for the deposit REST record is **deposit**.

## Code Samples

The following code samples show how to work with the deposit record.

### Getting OpenAPI Metadata from the Metadata Catalog

```
1 | GET https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog?select=deposit
```

### Creating a Deposit with Other Deposit and Cash Back Lines

```
1 | POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/deposit
2 | { "account": { "id": "1" }, "cashback": { "items": [ { "account": { "id": "10" }, "amount": 18.46 } ] }, "other": { "items":
  | [ { "account": { "id": "97" }, "amount": 178.24, "entity": { "id": "107" }, "paymentMethod": { "id": "1" } } ] }
3 | }
```

### Adding a Deposit Other Line

```
1 | PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/deposit/{{depositId}} { "other": { "items": [ { "account":
  | { "id": "10" }, "amount": 78.5, "entity": { "id": "69" }, "memo": "Deposit Other MEMO", "paymentMethod": { "id": "5" } } ] }
2 | }
```

### Replacing a Deposit Cashback Sublist

```
1 | PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/deposit/{{depositId}}?replace=cashback { "cashback":
  | { "items": [ { "account": { "id": "97" }, "amount": 12.0, "memo": "Cashback item of new sublist" } ] }
2 | }
```

### Creating a Deposit with Payment Lines

```
1 | POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/deposit { "account": { "id": "1" }, "payment": { "items":
  | [ { "deposit": true, "id": 379 }, { "deposit": true, "id": 380 } ] }
2 | }
```

### Adding a Deposit Payment Line

```
1 | PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/deposit/{{depositId}} { "payment": { "items":
  | [ { "deposit": true, "id": 381 } ] }
2 | }
```

## Getting a Deposit by ID

```
1 | GET https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/deposit/{{depositId}}
```

## Removing a Deposit by ID

```
1 | DELETE https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/deposit/{{depositId}}
```

## Deposit Application

NetSuite exposes the deposit application record to REST web services. A deposit application transaction applies a customer deposit against an invoice after the order is complete. For details, see the help topic [Applying a Customer Deposit](#).

The REST API Browser includes information about the field names and field types of the deposit application record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [depositApplication](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the deposit application REST record is **depositApplication**.

## Subrecords

The deposit application record has a list of `depositApplicationApplyLine` and `accountingBookDetail` subrecords.

## Prerequisite

Before you work with deposit applications, enable the A/R feature.

**Note:** Lines on the apply sublist are not identified by line numbers, but by the doc field as a key with the line as a secondary key. The line represents the transaction line ID, not the number of line on the sublist.

Also, note that you can create a deposit application only by transformation. See [Creating a Deposit Application](#).

## Code Samples

### Creating a Deposit Application

```
1 | POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/customerDeposit/<depositId>/!transform/depositApplication
```

```

2 | { "apply": { "items": [ { "doc": { "id": 261, "apply": true } } ] } }
3 | }

```

## Updating a Deposit Application

```

1 | PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/depositapplication/{id}
2 | { "memo": "UPDATED TEST"
3 | }

```

## Description Item

Description line items let you place sentence or paragraph long descriptions on items you are not selling. For example, you can enter special shipping instructions or a disclaimer.

To learn more, see the help topic [Description Items](#).

This record is not a subrecord, nor does it have any subrecords.

The REST API Browser includes information about the field names and field types of the description item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [descriptionItem](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the description item REST record is **descriptionitem**.

## Code Sample

### Creating a Description Item

```

1 | POST {{COMPANY_URL}}/services/rest/record/v1/descriptionitem { "itemId": "MY-CUSTOM-UNIQUE-ITEM-ID", "description": "Special
2 |   shipping instructions..."
3 | }

```

### Getting a Description Item by ID

```

1 | GET {{COMPANY_URL}}/services/rest/record/v1/descriptionitem/{{descriptionItemId}}
2 |
3 |
4 | {{descriptionItemId}} - existing description items record's ID

```

### Finding a Description Item by itemId

```

1 | GET {{COMPANY_URL}}/services/rest/record/v1/descriptionitem?q=itemId IS MY-CUSTOM-UNIQUE-ITEM-ID
2 |
3 | Query parameters: q={{expression}}

```

```

4
5 possible expression: field {{operator}} field's value
6
7 example: itemId IS MY-CUSTOM-UNIQUE-ITEM-ID

```

## Updating an Existing Description Item

```

1 PATCH {{COMPANY_URL}}/services/rest/record/v1/descriptionitem/{{descriptionItemId}} { "description": "Updated descrip
2 }

```

## Removing a Description Item

```

1 DELETE {{COMPANY_URL}}/services/rest/record/v1/descriptionitem/{{descriptionItemId}}

```

## Discount Item

A discount item record exposes a discount item to REST web services. This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the discount item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [discountItem](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a discount item REST record is **discountItem**.

## Taxation

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The discount item includes the Apply Before Tax field related to taxation features.

## Code Samples

The following samples show common use cases for discount items. The example ID is 4.

## Creating a Discount Item Using a POST Request

```

1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/discountItem
2 { "isInactive": false, "itemId": "031036c6-1815-4b95-87d6-d5f9367a1cf5", "rate": 8.0, "displayName": "Test nonposting", "account":
  { "id": "105", "refName": "Discount 500$ After Tax Non Post", "externalId": "2" }, "nonPosting": false

```

```
3 | }
```

## Retrieving a Discount Item Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/discountItem/4
```

## Updating a Discount Item Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/discountItem/4
2 | { "isInactive": false, "description": "Discount 10$ After Tax Non Post", "itemId": "03", "rate": "", "displayName": "Test", "account": { "id": "105", "refName": "Discount 5$ After Tax Non Post", "externalId": "2" }
3 | }
```

## Deleting a Discount Item Using a DELETE Request

```
1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/discountItem/4
```

## Download Item

A download item record exposes a download item to REST web services. This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the download item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [DownloadItem](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a download item REST record is **downloadItem**.

## Prerequisites

You must first enable the feature.

## Code Samples

The following samples show common use cases for download items. The example ID is 4.

## Creating a Download Item Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/downloadItem
2 | { "itemId": "download item", "subsidiary": { "items": [ { "id": "5" } ] }
3 | }
```

## Retrieving a Download Item Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/downloadItem/4
```

## Updating a Download Item Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/downloadItem/4
2 | { "parent": {"id": 315}
3 | }
```

## Email Template

An email template record exposes an email template to REST web services.

This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the email template record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [emailTemplate](#) reference page.

For information about using email templates, including adding fields to an email template, see the help topic [Email Template](#).

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the email template REST record is **emailtemplate**.

The record ID for the email template REST subrecord is **CrmTemplateCategorySubsidiary**.

## Prerequisites

You must enable the CRM feature before you can use this record through REST web services.

## Exposed Elements

The following email template record elements are exposed to REST web services:

- {addcompanyaddress} – add company address
- {addunsubscribeLink} – add unsubscribe link
- {content} – content
- {description} – description
- {id} – ID
- {isautoconverted} – is automatically converted
- {isinactive} – is inactive
- {isprivate} – is private

- {mediaitem} – media item
- {name} – name
- {recordtype} – record type
- {subject} – subject
- {subscription} – subscription

## Code Sample

In the following example, <accountID> represents your account ID.

HTTP request: `https://<accountID>/services/rest/record/v1/emailtemplate/973`

```
1 { "name": "EmailTemplateNine", "addunsubscribelink": true, "description": "Marketing Campaign Template"
2 }
```

## Employee

An employee record exposes an employee to REST web services.

This record:

- is not a subrecord
- has the following subrecords:
  - AccruedTime
  - AddressBook
  - Certificate
  - CompanyContribution
  - CorporateCards
  - Deduction
  - DirectDepositList
  - Earning
  - EmpPerms
  - EmployeeEmergencyContact
  - EmployeeCurrency
  - EmployeeDirectDeposit
  - EmployeeHCMPosition
  - JurisdictionHist
  - Rates
  - TaxOptions

The REST API Browser includes information about the field names and field types of the employee record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [employee](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the employee REST record is **employee**.

## Prerequisites

You must enable the Advanced Employee Permissions feature, the Employee Record, the Perform Search, the SuiteAnalytics Workbook, and SuiteScript before you can use this record through REST web services.

## Inaccessible Elements

The following sublists are not accessible through REST web services:

- hcmposition
- usernotes

The following fields are not accessible through REST web services:

- adpwarning
- billpay
- directdeposit
- hasshippingaddress
- jurisdictionschool
- origbinactive
- origgiveaccess
- origsubstatus
- roleforsearch
- shipaddr1
- shipaddr2
- shipaddr3
- shipaddressee
- shipattention
- shipcity
- shipcountry
- shipstate
- shipzip
- unsubscribe

## Elements with Different Functionality

The following sublists have different functionality through REST web services:

- directdepositlist
- jurisdictionhist

- roles

The following fields have different functionality through REST web services:

- adipid
- altname
- autoname
- changedetails
- conflictresults
- customform
- defaytkaddress
- defaultbillingaddress
- defaultshippingaddress
- effectivedatemode
- empcenterqty
- emplcenterqtymax
- employeechangereason
- entitynumber
- fulluserqty
- fulluserqtymax
- image
- isempcenterqtyenforced
- isfulluserqtyenforced
- isretailuserqtyenforced
- jurisdictioncounty
- jurisdictionfederal
- jurisdictionhist
- jurisdictionstate
- password
- password2
- phoneticname
- requiredpwdchange
- retailuserqty
- retailuserqtymax
- sendemail
- socialsecuritynumber
- terminationbydeath
- wasempcenterhasaccess
- wasfulluserhasaccess
- wasinactive
- wasretailuserhasaccess

## Actions

The employee record has the following actions through REST web services:

- Delete
- Save As
- Save & New
- Save & Next
- Save
- Reset
- Customize Form
- New Field

## Elements Related to Taxation Features

 **Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The edition field is related to the ADVTAXENGINE feature.

The following fields are related to the ANYPAYROLL feature:

- jurisdictioncounty
- jurisdictionfederal
- jurisdictionhist
- jurisdictionlocal
- jurisdictionstate
- lastpaiddatetime
- payfrequency
- useperquest
- usetimedate
- workplace
- adpwarning
- accruedtime
- companycontribution
- deduction
- earning
- emptaxoptions

## Additional Details

The employee record is complex because it is integrated with many features.

Some elements of the employee form behave differently when accessed through REST web services.

## Code Sample

The following code shows how to create a new employee record:

```
1 POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/employee
2 { "firstName": "TEST FIRSTNAME", "lastName": "TEST LASTNAME", "subsidiary": {"id": "1"}
3 }
```

The following code shows how to update an employee's last name:

```
1 PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/employee/{id}
2 { "lastName": "NEW LASTNAME"
3 }
```

## Estimate

An estimate record exposes an estimate to REST web services. This record is not a subrecord. This record has one subrecord: record ID item.

The REST API Browser includes information about the field names and field types of the estimate record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [estimate](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the estimate REST record is **estimate**.

## Prerequisites

You must enable estimates before you can use this record through REST web services. To do so, go to Setup > Company > Enable Features. On the **Transactions** subtab under Basic Features, check the **Estimates** box, and then click **Save**.

## Actions

The estimates record has the following actions through REST web services:

- New
- Make Copy
- Email
- Show Activity
- Go To Register
- GL Impact

## Field Exposure

Fields exposed through SuiteScript should all be exposed through REST web services. Some fields may be hidden on the UI.

## Taxation

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The following fields on the estimate REST record are related to taxation:

- tax code
- tax rate
- shipping tax code
- shipping tax rate

## Usage Notes

To use the **Choose Team** and **Update Customer** fields in the Sales Team subtab, the Team Selling feature must be enabled. To enable Team Selling, go to Setup > Company > Enable Features > CRM. Check the Team Selling box, and then click Save.

The Multi-Partner Management Feature must also be enabled to use the **Update Customer** field in the Relationships subtab. An administrator can enable the Multi-Partner Management feature at Setup > Company > Setup Tasks > Enable Features. Click the CRM subtab, and then under Partners, check the Multi-Partner Management box. Click Save.

## Code Samples

These samples show common use cases for estimates. The example ID is 6.

### Creating an Estimate Using a POST Request

```
1 POST {{REST_SERVICES}}/record/v1/estimate
2 { "altSalesTotal": 0.0, "canHaveStackable": false, "currency": { "id": "1", "refName": "USA" }, "discountItem": { "id": "-6", "refName": "Partner Discount" }, "discountRate": -5.0, "discountTotal": -5.0, "dueDate": "2023-08-09", "entity": { "id": "110", "refName": "Anonymous Customer" }, "entityStatus": { "id": "10", "refName": "Proposal" }, "exchangeRate": 1.0, "expectedCloseDate": "2023-07-10", "forecastType": { "id": "2", "refName": "Most Likely" }, "item": { "items": [ { "line": 1, "item": { "id": "98", "rate": 39.95, "quantity": 1 } } ], "subsidiary": { "id": "1", "refName": "Parent Company" }, "subtotal": 39.95, "toBeEmailed": false, "toBeFaxed": false, "toBePrinted": false, "total": 34.95, "tranDate": "2023-07-10", "tranId": "1", "visibleToCustomer": true
3 }
```

### Retrieving an Estimate Using a GET Request

```
1 GET {{REST_SERVICES}}/record/v1/estimate/6
```

## Updating an Estimate Using a PATCH Request

```

1 | PATCH {{REST_SERVICES}}/record/v1/estimate/6
2 | { "altSalesTotal": 0.0, "canHaveStackable": false, "currency": { "id": "1", "refName": "USA" }, "discountItem": { "id": "-6", "ref
  Name": "Partner Discount" }, "discountRate": -5.0, "discountTotal": -5.0, "dueDate": "2023-08-09", "entity": { "id": "110", "ref
  Name": "Anonymous Customer" }, "entityStatus": { "id": "10", "refName": "Proposal" }, "exchangeRate": 1.0, "expectedClose
  Date": "2023-07-10", "forecastType": { "id": "2", "refName": "Most Likely" }, "item": { "items": [{ "line": 1, "item":
    { "id": 98, "rate": 39.95, "quantity": 1 }, { "line": 2, "item": { "id": 5, "rate": 10, "quantity": 2 } } ], "subsidiary":
    { "id": "1", "refName": "Parent Company" }, "subtotal": 39.95, "toBeEmailed": false, "toBeFaxed": false, "toBePrinted": false, "to
    tal": 34.95, "tranDate": "2023-07-10", "tranId": "1", "visibleToCustomer": true
  }
3 | }

```

## Deleting an Estimate Using a DELETE Request

```

1 | DELETE {{REST_SERVICES}}/record/v1/estimate/6

```

## Setting the Sales Order's Sales Team Members to Match Sales Group 164

```

1 | "salesGroup": {
2 |   "id": 164 }

```

## Update the Customer's Partner Team to Match the Transaction Partners

```

1 | "syncPartnerTeams": true

```

## Update the Customer's Sales Team to Match the Transaction Sales Team

```

1 | "syncSalesTeams": true

```

## Event

An event record exposes an event to REST web services.

This record:

- is not a subrecord
- has the following subrecords:
  - Attendee
  - CalendarEventResourceMap
  - EventFile

The REST API Browser includes information about the field names and field types of the event record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [event](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the event REST record is **calendarevent**.

## Exposed Elements

The following event record elements are exposed to REST web services:

- accesslevel
- alldayevent
- company
- contact
- createddate (Read only – set to current date)
- customform
- endtime (For alldayevent, set to 6pm)
- id
- location
- message
- organizer
- owner (read only)
- relateditem
- resource
- response
- startdate
- starttime (For alldayevent, set to 8am)
- status
- supportcase
- timedevent
- title
- transaction
- usernotes

## Code Sample

In the following example, <accountID> represents your account ID.

HTTP request: `https://<accountID>/services/rest/record/v1/calendarevent`

## Request Body

```
1 { "title": "Meeting with Tom", "status": "CONFIRMED", "message": "Bring Monthly Results", "location": "Caffe West", "start  
  date": "2022-11-02", "starttime": "14:30", "timedevent": true
```

2 | }

## To Obtain Previously-Created Event Record (where ID is 100425)

HTTP Method: GET

URL: <https://<accountID>/services/rest/record/v1/calendarevent/100425>

## Response Body

```
1 { "links": [ { "rel": "self", "href": "http://<accountID>/services/rest/record/v1/calendarevent/100425" }, { "accessLevel":
  { "links": [], "id": "PUBLIC", "refName": "Public" }, "allDayEvent": false, "attendee": { "links": [ { "rel": "self", "href":
    http://<accountID>/services/rest/record/v1/calendarevent/100425/attendee } ] }, "createdDate": "2020-11-25T16:40:00Z", "cus
    tomForm": { "links": [], "id": "-110", "refName": "Standard Event Form" }, "endTime": "15:30", "id": "100425", "lastModi
    fiedDate": "2020-11-25T16:40:00Z", "location": "Caffe West", "message": "Bring Monthly Results", "organizer": { "links":
      [ { "rel": "self", "href": "http://<accountID>/services/rest/record/v1/customer/-5" }, { "id": "-5", "refName": "A Wolfe-admin"
        }, "owner": { "links": [ { "rel": "self", "href": "http://<accountID>/services/rest/record/v1/customer/-5" }, { "id": "-5", "ref
          Name": "-5" } "resource": { "links": [ { "rel": "self", "href": "http://<accountID>.eng.netsuite.com/services/rest/record/v1/
            calendarevent/100425/resource" } ] }, "startDate": "2022-11-02", "startTime": "14:30", "status": { "links": [], "id": "CONFIRMED",
              "refName": "Confirmed" }, "timedEvent": true, "timezone": "America/Los_Angeles", "title": "Meeting with Tom"
            }
          }
        }
      }
    }
  }
2 }
```

## Expense Category

An expense category request record exposes an expense category to REST web services. Expense categories are used to group expenses. Each expense category is linked to an account. When an employee enters an expense report, they select a category for each expense, and the expense automatically posts to the associated expense account.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the expense category record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [expense category](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an expense category REST record is **expensecategory**.

## Prerequisites

You must enable the Expense Reports feature before you can use this record through REST web services.

## Limitations

Tax-related fields are not supported in REST.

## Code Sample

This sample shows how to create the record:

```
1 POST /services/rest/record/v1/expenseCategory HTTP/1.1
2 Host: runbox.
3 Content-Type: application/json
4 Authorization: .
5 Content-Length: 47
6 { "name": "5", "subsidiaries": "3"
7 }
```

This sample shows how to update the record:

```
1 PATCH /services/rest/record/v1/expenseCategory/2 HTTP/1.1
2 "name": "updated name"
3 }
```

## Expense Report

An expense report record exposes an expense report to REST web services. An expense report transaction records an employee's expenses for approval and conversion into a bill. The expense total remains in an unapproved expense account and has no accounting impact until the expense is approved by someone with accounting authority. After an expense report is approved, a bill is created and the expense amount is reflected on the books.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the expense report record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [expense report](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an expense report REST record is **expensereport**.

## Prerequisites

You must enable the Expense Reports feature before you can use this record through REST web services.

## Code Sample

This sample shows how to update the record:

```
1 PATCH /services/rest/record/v1/expenseReport/4 HTTP/1.1
2 "complete": true, "tranDate": "2023-08-18", "supervisorapproval": true
3 }
```

## Fair Value Price

Fair Value Price is a record that defines the fair value for items. Fair value price is used to allocate revenue in revenue arrangements.

For help working with this record in the UI, see the help topic [Fair Value Setup](#).

The REST API Browser includes information about the field names and field types of the fair value price record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [fairValuePrice](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a fair value price REST record is **fairvalueprice**.

## Prerequisites

This record is part of the Advanced Revenue Management (Revenue Allocation) feature. You must enable the Accounting Periods feature, Advanced Revenue Management (Essentials) feature, and Advanced Revenue Management (Revenue Allocation) feature to use advanced revenue management (revenue allocation). Before you begin working with advanced revenue management programmatically, see the help topic [Advanced Revenue Management \(Essentials\) and \(Revenue Allocation\)](#).

## Limitations

The following fields on this record are required:

- **currency**
- **fairValue**
- **fairValueFormula**

The following fields are disabled when the **FairValueRangePolicy** field is null:

- **highValue**
- **highValuePercent**
- **lowValue**
- **lowValuePercent**

The following fields on this record are not available in REST:

- **units**
- **unitsType**

## Additional Information

The **quantity** and **amount** fields appear when the **eventType** field is not set as percent complete. The **cumulativePercentComplete** field appears when **eventType** is set to percent complete.

## Code Samples

The following example shows how to read a fair value price record:

**Read:**

```
1 | GET {{REST_SERVICES}}/record/v1/fairvalueprice/2
```

## Fulfillment Request

A fulfillment request record exposes a fulfillment request to REST web services.

A fulfillment request is created by doing a transformation on a sales order.

The status of fulfillment request can be changed. The fulfillment request exception can be added.

This record:

- is not a subrecord
- has the following subrecords:
  - FulfillmentRequestException
  - FulfillmentRequestItem
  - FulfillmentRequestItemDetail
  - FulfillmentRequestItemDetailInventoryAssignment
  - FulfillmentRequestItemException

The REST API Browser includes information about the field names and field types of the fulfillment request record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [fulfillment request](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a fulfillment request REST record is fulfillmentrequest.

## Prerequisites

You must enable Fulfillment Request before you can use this record through REST web services.

Optionally, you must enable Store Pickup to use store pickup on the fulfillment request record through REST web services.

## Elements Not Accessible Through REST Web Services

The following subtabs are not accessible using REST web services:

- deletionreason

- transactionnumber
- lastmodifiedby
- createdby

## Code Sample

This sample shows how to transform a sales order to a fulfillment request:

```

1 POST -> {{COMPANY_URL}}/services/rest /record/v1/salesOrder/989?transform/fulfillmentRequest
2 With body: { "items": { "items": [ { "orderLine": 1, "quantity": 1 } ] } }
3 }
4 Update transaction status to "In progress":
5 PATCH -> {{COMPANY_URL}}/services/rest/record/v1/fulfillmentrequest/992
6 Body: {
7   "tranStatus": { "id": "B" }
8 }

```

## Gift Certificate

NetSuite exposes the gift certificate record to REST web services.

A gift certificate record is created when a customer purchases a gift certificate item. For example, you might create a gift certificate item and make it available for purchase. A customer can then buy this gift certificate item. When the customer buys the gift certificate item and designates a recipient, the system creates a gift certificate, which can be redeemed for goods and services offered by your business. The gift certificate record includes properties such as an expiration date, remaining value, and the recipient's email address.

The gift certificate record isn't a subrecord and doesn't have any subrecords of its own.

For more information, see the help topic [Gift Certificates](#).

## Record ID

The record ID for the gift certificate REST record is **giftcertificate**.

## Prerequisite

Before you work with gift certificates, enable the Gift Certificates feature.

## Limitations

You can't access the **internalid** field through REST. Use the **id** field to get the unique identifier of a gift certificate record instance.

Also, you can't access the **redeemedat** and **soldon** fields through REST.

The REST API Browser includes information about the field names and field types of the gift certificate record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [gift certificate](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Code Sample

You can only read gift certificates through REST. To create new ones, use the [Gift Certificate Item](#) record.

To read a gift certificate record, use a GET request:

```
1 | GET https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/giftcertificate/<id>
```

## Gift Certificate Item

NetSuite exposes the gift certificate item record to REST web services.

You can create gift certificate items that allow customers to purchase store credit they can send to someone as a gift. The recipient uses the gift certificate code when placing an order through your Web store or entering a transaction with a sales representative. You can set a preference in NetSuite for how you want to generate the gift certificate codes: you can create them yourself, or use a random hash code automatically generated by the system.

Gift certificate codes are not active until the order used to purchase the gift certificate is billed.

For more information, see the help topic [Gift Certificates](#).

## Record ID

The record ID for the gift certificate item REST record is **giftCertificateItem**.

## Prerequisite

Before you work with gift certificate items, enable the Gift Certificates feature.

**Note:** The gift certificate item record contains certain tax fields. If the SuiteTax feature is not enabled, you cannot create a gift certificate because of the required taxSchedule field that is annotated as a legacy tax field.

The REST API Browser includes information about the field names and field types of the estimate record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [gift certificate item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Code Samples

### Creating a Gift Certificate Item

```
1 | POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/giftcertificateitem
2 | { "itemId": "TEST", "liabilityaccount": {"id": 25}
3 | }
```

## Updating a Gift Certificate Item

```
1 PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/giftcertificateitem/{id}
2 { "itemId": "TEST_UPDATED", "upcCode": "123456"
3 }
```

## Global Account Mapping

For accounts using Multi-Book Accounting, the global account mapping record enables you to configure secondary accounting books to post to accounts different from the primary book. These mappings are used by transactions where the user can manually select the account to which the transaction posts.

Using this record requires that, as part of your Multi-Book configuration, you select the Chart of Accounts Mapping option at Setup > Company > Enable Features, on the Accounting subtab.

In the user interface, you access this record at Setup > Accounting > Global Account Mappings > New.

For information about working with this record in the UI, see the help topic [Global Account Mapping](#).

Global Account Mapping is not a subrecord and does not have a subrecord.

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the global account mapping record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [global account mapping](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the global account mapping REST record is **globalaccountmapping**.

## Prerequisites

This record requires the COACCLASSIFICATIONMANAGEMENT feature (Chart of Accounts Mapping).

## Code Sample

The following example shows how to update a global account mapping.

```
1 PATCH https://<accountID>/services/rest/record/v1/globalaccountmapping/{GLOBAL_ACCOUNT_MAPPING_ID}
2 { "destinationAccount": { "id": 10 }
3 }
```

## HCM Job

The HCM Job record exposes a job to REST web services.

This record:

- Is not a subrecord

The REST API Browser includes information about the field names and field types of the HCM job record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [HCM job](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an HCM Job REST record is **hcmjob**.

## Prerequisites

Before you can use this record with REST web services, you must enable the Job Management feature.

## Exposed Elements

The following record fields are exposed to REST web services:

- Description
- Employmentcategory
- Externalid
- Id
- Internalid
- Isinactive
- Jobid
- Jobidlabel
- Title

## Supported Operations

The following actions are available on this record:

- Submit
- Submit New
- Delete

## Code Samples

The following code shows how to get a job:

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/hcmjob/326
```

The following code shows how to update a job description:

```
1 PATCH
2 https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/hcmjob/326
3 {
4   "description": "Updated description"
5 }
```

The following code shows how to create a job:

```
1 POST
2 https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/hcmjob
3 {
4   "description": "New Description",
5   "employmentCategory": {
6     "id": "-10"
7   },
8   "title": "New job rest"
9 }
```

## Impact Subcategory

An impact subcategory record exposes an impact subcategory to REST web services. This record:

- has no subrecords
- is not a subrecord

To access this record in NetSuite, go to Setup > Company > Enable Features > Analytics > Advanced SaaS Metrics (Administrator).

To import impact subcategory records, verify the following settings:

- On the Analytics subtab, enable the **SaaS Metrics** feature.

For more information about setting up impact subcategory records, see the help topic [SuiteBilling Overview](#).

The REST API Browser includes information about the field names and field types of the impact subcategory record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [impact subcategory](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an impact subcategory REST record is **impactsubcategory**.

## Code Sample

This sample shows how to get an impact subcategory record:

```
1 GET {{REST_SERVICES}}/record/v1/impactsubcategory/-21
```

This sample shows how to create an impact subcategory record:

```

1 | POST {{REST_SERVICES}}/record/v1/impactsubcategory Body
2 | {
3 |   "name": "impact_subcategory_001"
4 | }

```

## Inbound Shipment

An inbound shipment record exposes an inbound shipment to REST web services. This record is not a subrecord. If the Inventory Detail feature is enabled, this record has one subrecord: inventorydetail.

For help working with this record in the UI, see the help topic [Inbound Shipment Management](#).

The REST API Browser includes information about the field names and field types of the inbound shipment record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [inboundShipment](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an inbound shipment REST record is **inboundshipment**.

The record ID for an inventory detail REST subrecord is **inventorydetail**.

## Code Samples

The following samples show common use cases for inbound shipments. The example ID is 4.

### Creating an Inbound Shipment From a Purchase Order

This sample shows how to create an inbound shipment from a purchase order. The shipmentitem field value should be the lineuniquekey attribute of the purchase order and not the internal id of the item.

```

1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/inboundshipment
2 | { "shipmentstatus": "inTransit", "shipmentmemo": "Test Memo1", "items": { "items": [ { "purchaseorder": 2172, "shipmentitem": "4827", "quantityexpected": 1 }, { "purchaseorder": 2172, "shipmentitem": "4828", "quantityexpected": 1 } ] } }
3 | }

```

### Retrieving an Inbound Shipment Using a GET Request

```

1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/inboundshipment/4

```

### Updating an Inbound Shipment Using a PATCH Request

This sample shows how to update an inbound shipment to add a line.

```

1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/inboundshipment/4
2 | { "items": { "items": [ { "purchaseorder": 2172, "shipmentitem": "4829", "quantityexpected": 1 } ] } }
3 | }

```

## Deleting an Inbound Shipment Using a DELETE Request

1 | **DELETE** <https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/inboundshipment/4>

## Intercompany Journal Entry

An intercompany journal entry record is a specialized type of record available only in OneWorld accounts. An intercompany journal entry records debits and credits to be posted to ledger accounts for transactions between two subsidiaries. In an account that has the Multi-Book Accounting feature enabled, you can also use this record type to create book specific intercompany journal entries.

In the user interface, you access this record as follows:

- **Intercompany journal entries** – Go to Transactions > Financial > Make Journal Entries (Administrator). For help working with this record in the user interface, see the help topic [Making Intercompany Journal Entries](#).
- **Book specific intercompany journal entries** – Go to Transactions > Financial > Make Journal Entries (Administrator). Note that this form is available only to accounts that use Multi-Book Accounting.

For information about working with intercompany journal entries in the UI, see the help topic [Making Intercompany Journal Entries](#).

The REST API Browser includes information about the field names and field types of the intercompany journal entry record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [intercompany journal entry](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the intercompany journal entry REST record is **intercompanyjournalentry**.

The intercompany journal entry record has the following subrecords:

- accountingbookdetail
- appliedrules
- expenseplanmessage

## Limitations

Read-only sublists, including system notes, are not supported in REST.

## Code Sample

```
1 | { "approved": true, "createdDate": "2023-08-03T06:27:00Z", "currency": { "id": "4" }, "customForm": { "id": "30" }, "exchangeRate": 0.01241975, "lastModifiedDate": "2023-08-03T06:27:00Z", "line": { "items": [ { "lineSubsidiary": { "id": "4" }, "account": { "id": "6" }, "credit": 10 }, { "lineSubsidiary": { "id": "4" }, "account": { "id": "9" }, "debit": 10 }, { "lineSubsidiary": { "id": "5" }, "account": { "id": "6" }, "credit": 10 }, { "lineSubsidiary": { "id": "5" }, "account": { "id": "9" }, "debit": 10 } ] }, "memo": "memo test", "postingPeriod": { "id": "267" }, "prevDate": "2023-08-02", "reversalDefer": false, "subsidiary": { "id": "4" }, "toSubsidiary": { "id": "5" }, "tranDate": "2023-08-02", "tranId": "1"
2 | }
```

## Intercompany Transfer Order

In NetSuite OneWorld accounts, an inventory transfer order transaction records the change in location of inventory items from a subsidiary to a location in another subsidiary. It also records the quantity and shipping details.

This transaction is available when the Locations feature and the Multi-Location Inventory (MLI) feature are enabled. It has the following subrecord: inventory detail. For information about how intercompany transfer orders are used, see the help topic [Intercompany Inventory Transfers - Non-Arm's Length](#).

The REST API Browser includes information about the field names and field types of the intercompany transfer order record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [intercompany transfer order](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for the intercompany transfer order REST record is **intercompanytransferorder**.

The record ID for an inventory detail REST subrecord is **inventorydetail**.

## Prerequisites

There are no prerequisites for using the record in REST.

## Usage Notes

You should read the usage notes for [Intercompany Transfer Order](#) because these notes also apply to the Intercompany Transfer Order in REST web services.

## Code Samples

The following sample shows you how to add an intercompany transfer order record:

```
1 {{REST_SERVICES}}/record/v1/intercompanytransferorder
2 { "currency": { "id": "1", "refName": "British pound" }, "incoTerm": { "links": [], "id": "1", "refName": "DAP" }, "item":
  { "items": [ { "amount": 0.0, "inventoryDetail": { "inventoryAssignment": { "items": [], "totalResults": 0 } }, "isClosed":
    false, "item": { "id": "22" }, "itemType": { "id": "InvPart" }, "line": 1, "quantity": 25.0, "rate": 2.0 } ] }, "location":
  { "id": "1", "refName": "WMS Warehouse 1" }, "subsidiary": { "id": "1", "refName": "WMS" }, "tosubsidiary": { "id": "3", "ref
    Name": "Canada" }, "transferlocation": { "id": "18", "refName": "Toronto" }
3 }
```

## Inventory Adjustment

An inventory adjustment transaction records the change in quantity and resulting value of inventory items within a location. For example, this transaction can be used to account for clerical errors, changes in cost, thefts, or miscounts. If you use the LIFO or FIFO costing methods, this transaction preserves the quantity and value of an inventory item and preserves the costing history of the item.

This transaction is available when the Inventory feature is enabled. This record has one subrecord: inventory detail. For more details about this type of transaction, see the help topic [Inventory Adjustments](#).

The REST API Browser includes information about the field names and field types of the inventory adjustment record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [inventory adjustment](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an inventory adjustment REST record is **inventoryadjustment**.

The record ID for an inventory detail REST subrecord is **inventorydetail**.

## Prerequisites

There are no prerequisites for using the record in REST.

## Sample Codes

The following example shows how to create an inventory adjustment:

```
1 POST {{REST_SERVICES}}/record/v1/inventoryadjustment{ "account": { "id": "1", "refName": "Checking" }, "adjLocation":
  { "id": "1", "refName": "WMS Warehouse 1" }, "customer": { "id": "19" }, "inventory": { "items": [ { "adjustQtyBy": 5.0, "item":
  { "id": "20", "refName": "Item" }, "line": 1, "location": { "id": "1", "refName": "WMS Warehouse 1" } }, { "adjustQty
  By": 10.0, "item": { "id": "21", "refName": "Item2" }, "line": 2, "location": { "id": "2", "refName": "WMS Warehouse 2"
  } } ] }, "postingPeriod": { "id": "194", "refName": "Jun 2023" }, "subsidiary": { "id": "1", "refName": "WMS" }
2 }
```

## Inventory Cost Revaluation

An inventory cost revaluation record exposes an inventory cost revaluation to REST web services. This record is not a subrecord. This record has one subrecord: inventory cost revaluation cost component.

For help working with this record in the UI, see the help topics [Manually Entering an Inventory Cost Revaluation](#) and [Revalue Standard Cost Inventory](#).

The REST API Browser includes information about the field names and field types of the inventory cost revaluation record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [inventory cost revaluation](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an inventory cost revaluation REST record is **inventorycostrevaluation**.

The record ID for an inventory cost revaluation cost component REST subrecord is **InventoryCostRevaluationCostComponent**.

## Prerequisites

You must enable the Standard Costing feature before you can use this record through REST web services.

When the feature is enabled, you can access the inventory cost revaluation record in the UI at Transactions > Inventory > Revalue Inventory Cost (Administrator).

## Code Samples

The following samples show common use cases for inventory cost revaluations. The example ID is 4.

### Creating an Inventory Cost Revaluation for a Non-Assembly Item

This example shows how to create an inventory cost revaluation for a non-assembly item (itemType = "InvPart") using a POST request.

```
1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/inventorycostrevaluation
2 { "account": { "id": "94" }, "costComponent": { "items": [{ "cost": 12.0, "costCategory": { "id": 1 }, "items":
  [{ "cost": 10.0, "costCategory": { "id": 1 } }], "item": { "id": "69" }, "location": { "id": "6" }, "subsidiary": { "id": "1"
  }, "tranDate": "2023-05-01"
3 }
```

### Creating an Inventory Cost Revaluation for an Assembly Item

This examples shows how to create an inventory cost revaluation for an assembly item (itemType = "Assembly") using a POST request. It defines additional fields for component items.

```
1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/inventorycostrevaluation
2 { "account": { "id": "94" }, "costComponent": { "items": [{ "cost": 12.0, "costCategory": { "id": 1 }, "componentItem": { "id": 60
  }, "quantity": 1 }, { "cost": 10.0, "costCategory": { "id": 1 }, "componentItem": { "id": 61 }, "quantity": 1 } ] }, "item":
  { "id": "169" }, "location": { "id": "6" }, "subsidiary": { "id": "1" }, "tranDate": "2023-05-01"
3 }
```

### Retrieving an Inventory Cost Revaluation Using a GET Request

```
1 GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/inventorycostrevaluation/4
```

### Updating an Inventory Cost Revaluation Using a PATCH Request

This sample shows how to update cost component values for an inventory cost revaluation record with a transaction id of 4. It sets the cost category ID to 3 and the cost value to 14.0.

```
1 PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/inventorycostrevaluation/4
2 { "costComponent": { "items": [{ "cost": 14.0, "costCategory": { "id": 3 } } ] }
3 }
```

### Deleting an Inventory Cost Revaluation Using a DELETE Request

```
1 DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/inventorycostrevaluation/4
```

## Inventory Count

An inventory count transaction records the results of a physical count of items in a location. Inventory counts can initiate an inventory adjustment to update on-hand quantities based on the results.

This transaction is available when Inventory Count feature is enabled. Inventory counts for lot and serialized items require the Advanced Bin/Numbered Inventory Management feature. Inventory count records have one subrecord: count detail. For more information, see the help topic [Inventory Count](#).

The REST API Browser includes information about the field names and field types of the inventory count record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [inventory count](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an inventory count REST record is **inventorycount**.

The record ID for a count detail REST subrecord is **countdetail**.

## Prerequisites

There are no prerequisites for using the record in REST.

## Usage Notes

Review the following notes on working with the inventory count REST record:

- Snapshot inventory detail and adjustment inventory detail are not accessible through REST but available in NetSuite UI.
- Start, Approve, and Complete actions are not accessible through REST. Perform any of these actions on the NetSuite UI before you can perform the corresponding updates through REST.
- When an inventory count is in Started status, items cannot be added or deleted from the list.

## Sample Codes

The sample codes show common use cases for inventory counts.

### Creating an Inventory Count

The following example shows how to create an inventory count:

```
1 POST {{REST_SERVICES}}/record/v1/inventorycount { "subsidiary": {"id": 1}, "location": {"id": 1}, "item": { "items": [{ "item":
  { "id": "206" }, "binNumber": { "id": "20", "refName": "W1-Z3-Storage" } } ] }
2 }
```

### Updating an Inventory Count

The following example shows how to enter the count quantity and details of an inventory count in Started status:

```

1 | PATCH {{REST_SERVICES}}/record/v1/inventorycount/2177 { "item": { "items": [ { "countLine": 1, "countQuantity": 4.0, "countDetail":
2 | { "inventoryDetail": { "items": [ { "inventoryStatus": { "id": "1" }, "quantity": 4.0 } ] } } } } }

```

## Inventory Item

Inventory item records are used to track information about items for which you maintain a stock.

For more information, see the help topic [Items](#).

This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the inventory item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [inventory item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the inventory item REST record is **inventoryItem**.

## Prerequisites

There are no prerequisites for using this record in REST.

## Limitations

An inventory item REST record does not show Quantity Pricing Discount/Levels (quantitypricingdiscount, quantitypricinglevel) or the Item Translations subtab (translations).

 **Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

## Code Sample

The following example shows how to get record metadata for an inventory item.

### Get Record Metadata:

```

1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/metadata-catalog/inventoryItemHeader: Accept: applica
   | tion/swagger+json

```

### Get Record:

```

1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/inventoryItem/<ID>?expandSubResources=true

```

### Update Record:

```

1 PATCH https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/inventoryItem/<ID>
2 Header: Content-Type: application/json
3 Body:
4 {
5   "itemId": "Updated item name",
6   "upcCode": "123456"
7 }

```

### Update Item Price When Related Features are Enabled:

```

1 PATCH
2 https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/inventoryItem/<ID>/price/currencypage=1,pricelevel=1,quantity=10
3 Header: Content-Type: application/json
4 Body:
5 {
6   "price": 10
7 }

```

## Inventory Number

An inventory number record uniquely identifies either an item in physical inventory with a serial number or a group of items with a lot number. This record is available when the Serialized Inventory or Lot Tracking feature is enabled. For details about these types of items, see the help topic [Serial Numbered Items](#) or [Lot Numbered Items](#).

The REST API Browser includes information about the field names and field types of the inventory number record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [inventory number](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an inventory number REST record is **inventorynumber**.

## Prerequisites

There are no prerequisites for using the record in REST.

## Limitations

Review the following limitations when working with the inventory number REST record:

- You cannot create standalone inventory number records. The system generates these records when a serialized item or lot numbered item is created, based on the serial number or lot number values entered for the item record. See the help topic [Add New Serial Numbers to Inventory](#) or [Creating Lot Numbered Items](#).
- You cannot delete inventory number records.
- The Inventory Number record includes a sublist that has the following fields for serial or lot numbers:
  - Issue Inventory Number (ID: issueinventorynumber) – In this field, you can select lot or serial numbers that exist in your inventory. Be sure to reference this field to select numbers for items that you include in a sales order, item fulfillment, and negative adjustments, among others.

- Receipt Inventory Number (ID: receiptinventorynumber) – In this field, you can enter lot or serial numbers for items that you want to add to your inventory. Be sure to use this field to assign numbers to items within item receipts and positive adjustments, among others.

## Sample Codes

The following sample shows you how to update an inventory number record:

```
1 | {{REST_SERVICES}}/record/v1/inventorynumber/
2 | { "memo": "test lot num inventory number", "expirationDate": "2023-06-19"
3 | }
```

## Inventory Transfer

An inventory transfer transaction records the change in quantity when items are transferred between locations. It also decreases the on-hand quantity in the source location and increases it in the new location.

This transaction is available when the Locations feature and the Multi-Location Inventory (MLI) feature are enabled. For more details about this type of transaction, see the help topic [Transferring Inventory](#).

The REST API Browser includes information about the field names and field types of the inventory transfer record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [inventory transfer](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an inventory transfer REST record is **inventorytransfer**.

The record ID for an inventory detail REST subrecord is **inventorydetail**.

## Prerequisites

There are no prerequisites for using the record in REST.

## Sample Codes

The sample codes show common use cases for inventory transfers.

## Creating an Inventory Transfer

The following example shows how to create an inventory transfer:

```
1 | POST {{REST_SERVICES}}/record/v1/inventorytransfer
2 | { "subsidiary": {"id": 1}, "location": {"id": 1}, "transferLocation": {"id": 2}, "inventory": { "items": [{ "item": { "id": 20
  | }, "adjustQtyBy": 10 }} }
```

```
3 | }
```

## Updating an inventory transfer

The following example shows how to update an inventory transfer:

```
1 PATCH {{REST_SERVICES}}/record/v1/inventorytransfer/2072 { "total": 10.0, "inventory": { "items": [{ "line": 1, "adjustQty
  By": 2, "inventoryDetail": { "inventoryAssignment": { "items": [ { "internalId": 7207, "inventoryDetail": 1476, "inventoryStatus":
    {"id": 1}, "toInventoryStatus": {"id": 1}, "quantity": 2.0 } ] } } ] } } }
2 }
```

## Invoice

The invoice record exposes an invoice to REST web services.

To access this record in NetSuite, go to Transactions > Sales > Create Invoices.

For information about the invoice record user interface, see the help topic [Invoices](#).

There are no prerequisites for using this record through REST web services.

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the invoice record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [invoice](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an invoice REST record is **invoice**.

## Usage Notes

To use the **Choose Team** and **Update Customer** fields in the Sales Team subtab, the Team Selling feature must be enabled. To enable Team Selling, go to Setup > Company > Enable Features > CRM. Check the Team Selling box, and then click Save.

The Multi-Partner Management Feature must also be enabled to use the **Update Customer** field in the Relationships subtab. An administrator can enable the Multi-Partner Management feature at Setup > Company > Setup Tasks > Enable Features. Click the CRM subtab, and then under Partners, check the Multi-Partner Management box. Click Save.

## Code Samples

These samples show common use cases for invoices.

In the following examples,

- <accountID> represents your account ID.
- <recordKey> represents the database key for the invoice.

## Creating a New Invoice With an Item

```
1 POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/invoice { "asofdate": "2021-03-15", "end
  date": "2021-06-15", "entity": { "id": "220" }, "item": { "items": [ { "amount": 1000.0, "item": { "id": "144" } } ] }, "start
  date": "2021-03-15", "subsidiary": { "id": "1" }, "terms": { "id": "1" } }
```

## Creating a New Invoice With a Billable Item

```
1 POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/invoice { "asofdate": "2021-03-15", "entity":
  { "id": "221" }, "itemcost": { "items": [ { "apply": true, "doc": { "id": "310" }, "line": 1 } ] }
2 }
```

## Creating a New Invoice With Billable Time

```
1 POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/invoice { "entity": { "id": "220" }, "time": { "items":
  [ { "apply": true, "doc": { "id": "1" }, "rate": 10 } ] }
2 }
```

## Updating an Existing Invoice

 **Note:** The transaction date and rate are in line item 1.

```
1 PATCH: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/invoice/<recordKey> { "item": { "items":
  [ { "line": 1, "rate": 50.00 } ] }, "tranDate": "2021-03-05"
2 }
```

## Setting the Sales Order's Sales Team Members to Match Sales Group 164

```
1 "salesGroup": {
2   "id": 164 }
```

## Update the Customer's Partner Team to Match the Transaction Partners

```
1 "syncPartnerTeams": true
```

## Update the Customer's Sales Team to Match the Transaction Sales Team

```
1 "syncSalesTeams": true
```

## Issue

Issue is used to document and manage issues that occur with the products you manufacture and sell. In the UI, you can log issues at Issues > Issues > Issues > New.

Use the issue record to create new issues and expose them to REST web services.

To learn more, see the help topic [Logging Issues](#).

This record:

- is not a subrecord
- has no subrecords

There are no prerequisites for using this record through REST web services.

The REST API Browser includes information about the field names and field types of the issue record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [issue](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an issue REST record is **issue**.

## Code Sample

The following samples show common use cases for issue.

### Creating an Issue

```
1 | POST https:// {{REST_SERVICES}} /services/rest/record/v1/issue
2 | Body: { "duplicateStatus": "F", "emailAssignee": false, "isOwner": true, "isShowstopper": false, "issueAbstract": "Issue
   | abstract 1701112365", "issueNumber": "issue_1701112365", "issueStatus": { "id": "1", "refName": "Submitted" }, "issueTags":
   | { "count": 0, "hasMore": false, "items": [], "offset": 0, "totalResults": 0 }, "issueType": { "id": "1", "refName": "Problem"
   | }, "mediaItem": { "count": 0, "hasMore": false, "items": [], "offset": 0, "totalResults": 0 }, "priority": { "id": "1", "ref
   | Name": "P1" }, "product": { "id": "103" }, "relatedIssues": { "items": [ ], "totalResults": 0 }, "reportedBy": { "id": "-5", "ref
   | Name": "Mark Wheel" }, "reproduce": { "id": "1", "refName": "Test Environment" }, "severity": { "id": "1", "refName": "S1 - Block
   | ing" }, "source": { "id": "1", "refName": "Internal" }, "statusType": "OPEN", "targetVersion": { "items": [], "totalResults": 0
   | }, "trackCode": { "id": "1", "refName": "Never" }
3 | }
```

### Retrieving an Issue

```
1 | GET
2 | https:// {{REST_SERVICES}} /services/rest/record/v1/issue
```

### Updating an issue

```
1 | PATCH https:// {{REST_SERVICES}} /services/rest/record/v1/issue/{{ISSUE_NUMBER}}
```

```

2 Body:
3 { "duplicateStatus": "F", "emailAssignee": false, "isOwner": true, "isShowstopper": false, "issueAbstract": "Issue abstract
  1701112438", "issueNumber": "issue_1701112438", "issueStatus": { "id": "1", "refName": "Submitted" }, "issueTags":
  { "count": 0, "hasMore": false, "items": [], "offset": 0, "totalResults": 0 }, "issueType": { "id": "1", "refName": "Problem"
  }, "mediaItem": { "count": 0, "hasMore": false, "items": [], "offset": 0, "totalResults": 0 }, "priority": { "id": "1", "ref
  Name": "P1" }, "product": { "id": "103" }, "relatedIssues": { "items": [ { "issuenum": "303", "issuestatus": "Submitted", "re
  lationship": { "id": "M3", "refName": "Blocks" } } ], "totalResults": 1 }, "reportedBy": { "id": "-5", "refName": "Mark Wheel"
  }, "reproduce": { "id": "1", "refName": "Test Environment" }, "severity": { "id": "1", "refName": "S1 - Blocking" }, "source":
  { "id": "1", "refName": "Internal" }, "statusType": "OPEN", "targetVersion": { "items": [], "totalResults": 0 }, "trackCode":
  { "id": "1", "refName": "Never" }
4 }'

```

## Item Account Mapping

For accounts using Multi-Book Accounting, the item account mapping record enables you to configure secondary accounting books to post to accounts different from the primary book, based on the item that is the subject of the transaction. These mappings are used by transactions where the item determines the account to which the transaction posts.

Using this record requires that, as part of your Multi-Book configuration, you select the Chart of Accounts Mapping option at Setup > Company > Enable Features, on the Accounting subtab.

In the user interface, you access this record at Setup > Accounting > Item Account Mappings > New.

For information about working with this record in the UI, see the help topic [Item Account Mapping](#).

Item Account Mapping is not a subrecord and does not have a subrecord.

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the item account mapping record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [item account mapping](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the item account mapping REST record is **itemaccountmapping**.

## Prerequisites

This record requires the COACCLASSIFICATIONMANAGEMENT feature (Chart of Accounts Mapping).

## Code Sample

The following example shows how to update an item account mapping.

```

1 PATCH https://<accountID>/services/rest/record/v1/itemaccountmapping/{ITEM_ACCOUNT_MAPPING_ID}
2 { "destinationAccount": { "id": 10 }
3 }

```

## Item Fulfillment

An item fulfillment transaction records the shipment of some or all items on an order to the customer. The processes for item fulfillment transactions depend on whether the Advanced Shipping feature is enabled.

- If Advanced Shipping is not enabled, the fulfillment and invoicing processes are combined. When an item fulfillment is created, a related invoice is created at the same time.
- If Advanced Shipping is enabled, fulfillment and invoicing are two independent processes, and shipments can be recorded separately from billing.

For more information about item fulfillment entries, see the help topic [Order Fulfillment](#).

This record:

- ItemFulfillment is not a subrecord.
- InventoryDetail is the subrecord.



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the item fulfillment record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [item fulfillment](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the item fulfillment REST record is **ItemFulfillment**.

## Prerequisites

There are no prerequisites for using this record in REST.

## Code Sample

The following example shows how to create an item fulfillment from a sales order record.

```
1 POST https://<accountID>/services/rest/record/v1/salesorder/{{SALES_ORDER_ID}}!transform/itemFulfillment
2 { "item":{ "items":[ { "orderLine":1, "location":6, "itemreceive": true } { "orderLine": 3 "location": 6, "itemreceive":
   false } ] }
3 }
```

## Item Group

An item group is sold as a single unit but made up of several individual items. An item group is a group of items on a sales or purchase order that need to be sold or purchased together. The item group is not

fulfilled, received, or stocked. However, the item group is available in the item list on a sales or purchase order and can be added to those transactions.

For more information, see the help topic [Item Groups](#).

The item group record has one subrecord: **member**.

The REST API Browser includes information about the field names and field types of the item group record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [item group](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the item group REST record is **itemgroup**.

## Inaccessible Elements

The **translations** sublist is not exposed to REST web services.

## Code Sample

### Creating an Item Group

```
1 | POST {{COMPANY_URL}}/services/rest/record/v1/itemGroup { "itemId": "ItemGroup REST TEST", "member": { "items": [ { "item":
  { "id":"233" }, "quantity":1 } ] } }
2 | }
```

### Getting an Existing Item Group by ID

```
1 | GET {{COMPANY_URL}}/services/rest/record/v1/itemGroup/{{itemGroupId}}
```

### Updating an Existing Item Group by ID

```
1 | PATCH {{COMPANY_URL}}/services/rest/record/v1/itemGroup/{{itemGroupId}} { "itemId": "ItemGroup REST TEST updated", "member":
  { "items": [ { "lineNumber": 1, "quantity":2 } ] } }
2 | }
```

## Item Receipt

An item receipt transaction records the receipt of items from orders or returns that arrive at your location. How you receive items depends on whether you use the Advanced Receiving feature, as described in the following:

- If you do not use Advanced Receiving, the receiving and billing processes are combined. When you receive an item, you create a vendor bill for it simultaneously.

- If you prefer to have separate processes to receive items and create vendor bills, you can use the Advanced Receiving feature.

With Advanced Receiving, you can use separate processes to receive items separately from billing items. Then, you can receive parts of an order before creating a bill for the whole order.

- If you use return authorizations, you can also receive authorized returns in parts using Advanced Receiving and the Item Receipt page.

This transaction is available when the Locations feature and the Multi-Location Inventory (MLI) feature are enabled. The item receipt record has one subrecord: inventory detail. For more information about item receipts, see the help topic [Receiving Orders](#).

The REST API Browser includes information about the field names and field types of the item receipt record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [item receipt](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an item receipt REST record is **itemreceipt**.

The record ID for an inventory detail REST subrecord is **inventorydetail**.

## Prerequisites

There are no prerequisites for using the record in REST.

## Usage Notes

You should read the usage notes on initializing item receipts from the supported transactions: [Item Receipt](#).

## Sample Codes

The following sample codes show you how to create an item receipt from a purchase order record. It uses the following details for a sample purchase order:

- Purchase order internal ID – 2173
- Purchase order item quantity – 4

```
1 POST {{REST_SERVICES}}/record/v1/purchaseorder/2173/!transform/itemreceipt { "item": { "items": [ { "line": 1, "inventoryde
tail":{"inventoryassignment": { "items": [{ "receiptInventoryNumber": "L1", "inventorystatus": 1, "binnumber": 20, "quantity":4
} ] } } } } }
2 }
```

## Item Revision

An item revision record exposes an item revision item to REST web services.

This record:

- is not a subrecord
- has the following subrecords:
  - component
  - operation

The REST API Browser includes information about the field names and field types of the item revision record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [item revision](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an item revision REST record is **ItemRevision**.

## Code Samples

The following samples show common use cases for item revisions. The example ID is 4.

### Creating an Item Revision Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/itemRevision
2 | { "item": { "id": 410, "name": "REV_2022", "effectiveDate": "2022-01-01", "memo": "memo"
3 | }
```

### Retrieving an Item Revision Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/itemRevision/4
```

### Updating an Item Revision Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/itemRevision/4
2 | { "memo": "new memo"
3 | }
```

## Job

A job record exposes a job (project) status to REST web services.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the job record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [job](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a job REST record is **job**.

## Prerequisites

You must enable the Projects feature before you can use this record through REST web services.

## Limitations

Tax-related fields are not supported in REST.

The following fields are not supported in REST:

- currencyPrecision
- bBudgetShowCalculatedLines
- cBudgetShowCalculatedLines
- customer
- originatingDoc
- originatingLine
- originatingActualLine
- originatingItem
- projectTasks\_count
- isBudgetRebuilding
- revenueArrangement
- primaryContact
- alternateContact
- baselineBudget
- estimateAtCompletionBudget

The following sublists are not supported in REST:

- ProjectResource
- ProjectBillingBudget
- ProjectCostBudget
- Work Assignments

## Code Sample

This sample shows how to update the record:

```
1 POST /services/rest/record/v1/job
2 { "subsidiary": "1", "entityId": "title", "projectExpenseType": "-2"
```

3 | }

## Job Status

A job status record exposes a job (project) status to REST web services. The job status record indicates the progress of the project.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the job status record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [job status](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a job status REST record is **jobstatus**.

## Prerequisites

You must enable the Projects feature before you can use this record through REST web services.

## Limitations

Tax-related fields are not supported in REST.

## Code Sample

This sample shows how to update the record:

```
1 PATCH /services/rest/record/v1/jobstatus/17 {  
2   "name": "Updated name", "description": "desc"  
3 }
```

## Job Type

A job type record exposes a job (project) type to REST web services. Project types are used to define basic project records if the Projects feature is enabled, and the Project Management feature is not enabled. You can use the Projects feature to track information about projects you are working on for customers, including time, contacts, and transactions.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the job type record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [job type](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an job type REST record is **jobtype**.

## Prerequisites

You must enable the Projects feature before you can use this record through REST web services.

## Limitations

Tax-related fields are not supported in REST.

## Code Sample

This sample shows how to update the record:

```
1 PATCH /services/rest/record/v1/jobtype/1 {
2   "name": "updated name", "isInactive": true
3 }
```

## Journal Entry

You use the journal entry record to adjust balances in accounts. Journal entries let you change the value of any set of accounts without having to enter a posting transaction. In an account that has the Multi-Book Accounting feature enabled, you can also use this record to create book specific journal entries.

For more information about journal entries in NetSuite, see the help topic [Journal Entries](#).

This record:

- is not a subrecord
- has the following subrecords:
  - journalentryline
  - accountingbookdetail

The REST API Browser includes information about the field names and field types of the journal entry record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [journal entry](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the journal entry REST record is **journalentry**.

## Prerequisites

There are no prerequisites for using the record in REST.

## Limitations

- Tax-related fields are not supported in REST.
- The accounting book detail sublist is part of the Multi-Book Accounting feature (and its related features). For more information, see the help topic [Multi-Book Accounting](#).
- Read-only sublists, including system notes and GL impact, are not supported in REST.

## Code Sample

```
1 {
2   "subsidiary": { "id": "1" }, "line": { "items": [ { "account": { "id": "58" }, "debit": 187, "memo": "Journal Example" },
3     { "account": { "id": "4" }, "credit": 187 } ] }
```

## Kit Item

A kit item record exposes a kit item to REST web services. This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the kit item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [kit item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

## Record IDs

The record ID for a kit item REST record is **kitItem**.

## Elements with Different Functionality

The record has a matrix pricing machine. Otherwise, it behaves in the same way as the inventory item.

## Code Samples

The following samples show common use cases for kit items. The example ID is 4.

### Creating a Kit Item Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/kitItem
```

```

2 | { "itemId": "Kit A", "member": { "items": [ { "item": { "id": "71", "refName": "InvItem_1" }, "quantity": 1.0 } ] } }
3 | }

```

## Retrieving a Kit Item Using a GET Request

```

1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/kitItem/4

```

## Updating a Kit Item Using a PATCH Request

```

1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/kitItem/4
2 | { "itemId": "Kit B", "description": "Kit Item description",
3 | }

```

```

1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/kitItem/4
2 | { "item": { "id": "71" }, "quantity": 5.0
3 | }

```

## Deleting a Kit Item Using a DELETE Request

```

1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/kitItem/4

```

## Location

Locations can be used to identify and categorize records in your NetSuite account. You can also define locations for your organization's needs by creating custom segments. Locations enable you to better organize data and preserve accuracy.

For information about working with locations in the UI, see the help topic [Classifications in NetSuite](#).

The REST API Browser includes information about the field names and field types of the location record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [location](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the location REST record is **location**.

The location record has the following subrecords:

- classTranslation
- businessHours (as part of the Store Pickup feature)
- mainAddress
- inventoryBalance
- returnAddress
- tranNumbering
- docNumbering

- includeLocationRegions (as part of the Automatic Location Assignment feature)
- excludeLocationRegions (as part of the Automatic Location Assignment feature)

## Limitations

Read-only sublists, including system notes, are not supported in REST.

## Code Sample

```
1 { "isInactive": false, "name": "REST test 1692272155", "subsidiary": { "items": [ { "id": 2 } ] }, "makeInventoryAvailableStore": true, "latitude": 90, "tranPrefix": "tp", "defaultAllocationPriority": 112, "locationType": 1, "longitude": 180, "makeInventoryAvailable": true, "timeZone": { "id": "Asia/Dacca"}, "logo": { "id": 14 } }
```

## Manufacturing Cost Template

A manufacturing cost template record exposes a manufacturing cost template to REST web services.

This record:

- is not a subrecord
- has one subrecord: costDetail

The REST API Browser includes information about the field names and field types of the manufacturing cost template record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [manufacturing cost template](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a manufacturing cost template REST record is **manufacturingCostTemplate**.

## Prerequisites

You must enable the Manufacturing Routing and Work Center features before you can use this record through REST web services.

## Code Samples

The following samples show common use cases for manufacturing cost templates. The example ID is 4.

### Creating a Manufacturing Cost Template Using a POST Request

```
1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingCostTemplate
2 {
3   "subsidiary": { "id": "5"},
4   "name": "rest mfg cost template",
5   "costDetail": {
```

```

6      "items": [
7        {
8          "costCategory": {"id": 5},
9          "item": {"id": 71}
10       },
11       {
12         "costCategory": {"id": 3},
13         "item": {"id": 69}
14       }
15     ]
16   }
17 }

```

## Retrieving a Manufacturing Cost Template Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingCostTemplate/4
```

## Updating a Manufacturing Cost Template Using a PATCH Request

The following code sample shows how to edit an item on the cost detail line. The component line ID is 7.

```

1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingRoutingCostTemplate/4/costDetail/7
2 | }

```

# Manufacturing Operation Task

A manufacturing operation task record exposes a manufacturing operation task to REST web services.

This record:

- is not a subrecord
- has the following subrecords:
  - costDetail
  - predecessor

The REST API Browser includes information about the field names and field types of the manufacturing operation task record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [manufacturing operation task](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a manufacturing operation task REST record is **manufacturingOperationTask**.

## Prerequisites

You must enable the Manufacturing Routing and Work Center features before you can use this record through REST web services.

## Code Samples

The following samples show common use cases for manufacturing operation tasks. The example ID is 4.

### Creating a Manufacturing Operation Task Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingOperationTask
2 | {
3 |   "manufacturingWorkCenter" : {"id": "39"},
4 |   "manufacturingCostTemplate" : {"id": 4},
5 |   "title": "My operation",
6 |   "operationSequence": 60,
7 |   "setupTime": 6,
8 |   "runRate": 4,
9 |   "machineResources": 1,
10 |  "laborResources": 1,
11 |  "workOrder": {
12 |    "id": 159
13 |  }
14 | }
```

### Retrieving a Manufacturing Operation Task Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingOperationTask/4
```

### Retrieving a Predecessor Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingOperationTask/238/predecessor
```

### Updating a Manufacturing Operation Task Using a PATCH Request

The following code sample shows how to add an entry to the cost detail sublist.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingOperationTask/4
2 | {
3 |   "costDetail": {
4 |     "items": [
5 |       {
6 |         "costCategory" : {"id": 1},
7 |         "item" : {"id": 72}
8 |       }
9 |     ]
10 |   }
11 | }
```

## Manufacturing Routing

A manufacturing routing record exposes a manufacturing routing to REST web services.

This record:

- is not a subrecord
- has the following subrecords:
  - routingComponent
  - routingStep

The REST API Browser includes information about the field names and field types of the manufacturing routing record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [manufacturing routing](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a manufacturing routing REST record is **manufacturingRouting**.

## Prerequisites

You must enable the Manufacturing Routing and Work Center features before you can use this record through REST web services.

## Additional Details

When you use a POST request to create a Manufacturing Routing, the routingComponent machine is indexed by the "lineNumber" fieldID. When you update the fieldID, the routingComponent machine is indexed by the "component" fieldID.

## Code Samples

The following samples show common use cases for manufacturing routings. The example ID is 4.

### Creating a Manufacturing Routing Using a POST Request

The following code sample shows how to create a manufacturing routing with routing steps and set a value for the operation on components using the operation sublist.

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingRouting
2 | { "name": "routing via rest", "billOfMaterials": { "id": 12 }, "subsidiary": { "id": 5 }, "location": { "items": [ { "id": 5 } ] }, "routingStep": { "items": [ { "operationSequence": 10, "operationName": "10", "manufacturingWorkCenter": { "id": 40 }, "manufacturingCostTemplate": { "id": 4 }, "setupTime": 5, "runRate": 10 }, { "operationSequence": 20, "operationName": "20", "manufacturingWorkCenter": { "id": 40 }, "manufacturingCostTemplate": { "id": 4 }, "setupTime": 5, "runRate": 10, "lagType": { "id": "time", "lagAmount": 9 } } ], "routingComponent": { "items": [ { "lineNumber": 1, "operationSequenceNumber": 10 }, { "lineNumber": 2, "operationSequenceNumber": 20 } ] } }
3 | { "itemId": " Other Charge for Purchase", "displayName": "displayName", "upcCode": "code" }
```

### Retrieving a Manufacturing Routing Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingRouting/4
```

## Updating a Manufacturing Routing Using a PATCH Request

The following code sample shows how to edit a component line using the operation sublist. The component line ID is 7.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingRouting/4/routingComponent/7
2 | { "operationSequenceNumber": 10
3 | }
```

The following code sample shows how to edit the setup time on the routing step sublist.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingRouting/4/routingStep/10
2 | { "setupTime": 25
3 | }
```

## Deleting a Manufacturing Routing Using a DELETE Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/manufacturingRouting/4
```

## Markup Item

A markup item record exposes a markup item to REST web services. To access this record in NetSuite, go to Lists > Accounting > Items > New > Markup.

The REST API Browser includes information about the field names and field types of the markup item record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [markup item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the markup item REST record is **markupitem**.

## Code Samples

These samples show common use cases for markup items. The example ID is 6.

### Creating a Markup Item Using a POST Request

```
1 | {
2 |   "itemId": "New Markup",
3 |   "itemType": {
4 |     "id": "Markup",
5 |     "refName": "Markup"
6 |   },
7 |   "nonPosting": true,
8 |   "rate": 60
9 | }
```

## Retrieving a Markup Item Using a GET Request

```
1 | GET: {{REST_SERVICES}}/record/v1/markupitem/6
```

## Updating a Markup Item Using a PATCH Request

```
1 | PATCH: {{REST_SERVICES}}/record/v1/markupitem/6
2 |
3 | {
4 |   "itemId": "Newer Markup Item",
5 |   "rate": 80
6 | }
```

## Deleting a Markup Item Using a DELETE Request

```
1 | DELETE: {{REST_SERVICES}}/record/v1/markupitem/6
```

## Message

A message record exposes a message to REST web services.

This record has the following subrecords:

- MessageFile
- CcBccRecipient

The REST API Browser includes information about the field names and field types of the message record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [message](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the message REST record is **Message**.

## Exposed Elements

The following message record elements are exposed to REST web services:

- activity
- author
- authoremail
- bcc
- cc
- emailed
- entity

- externalid
- hasattachment
- htmlmessage
- id
- incoming
- lastmodifieddate
- message
- messagedate
- recipient
- recipientemail
- record
- recordtype
- subject
- template
- time
- transaction

## Usage Notes

All fields are read-only.

## Code Sample

In the following example, <accountID> represents your account ID.

HTTP request: <https://<accountID>/services/rest/record/v1/message>

## Request Body

```
1 { "subject": "Meeting agenda", "message": "Please send the agenda for tomorrow's meeting.", "author": "-5", "authorEmail": "sender@your.example.com", "recipientEmail": "recipient@your.example.com", "cc": "cc_recipient@your.example.com", "bcc": "bcc_recipient@your.example.com"
2 }
```

## To Obtain Previously-Created Message Record (where ID is 230)

HTTP Method: GET

URL: <https://<accountID>/services/rest/record/v1/message/230>

## Response Body

```
1 { "links": [ { "rel": "self", "href": "http://<accountID>/services/rest/record/v1/message/230" }, { "author": { "links": [ { "rel": "self", "href": "http://<accountID>/services/rest/record/v1/customer/-5" }, { "id": "-5", "refName": "A Wolfe-admin"
```

```

1 }, "authorEmail": "sender@your.example.com", "bcc": "bcc_recipient@your.example.com", "cc": "cc_recipient@your.example.com", "e
2 mailed": true, "hasAttachment": false, "htmlMessage": true, "htmlMessage": true, "id": "230", "incoming": true, "lastModified
   Date": "2020-11-25T16:32:00Z", "message": "Please send the agenda for tomorrow's meeting.", "messageDate": "2020-11-25", "recipien
   t": { "links": [ { "rel": "self", "href": "<accountID>/services/rest/record/v1/customer/-5" } ], "id": "-5", "refName": "A Wolfe-
   admin" }, "recipientEmail": "recipient@your.example.com", "subject": "Meeting agenda", "time": "08:32"
3 }

```

## Merchandise Hierarchy Level

A merchandise hierarchy level record exposes a merchandise hierarchy level to REST web services. This record is not a subrecord and does not contain a subrecord.

The REST API Browser includes information about the field names and field types of the merchandise hierarchy level record and about the HTTP methods, request parameters, and operations available to this record.

For details, see the REST API Browser's [merchandise hierarchy level](#) reference page.

For information on using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the merchandise hierarchy level REST record is **merchandisehierarchylevel**.

## Prerequisites

You must enable the Merchandise Hierarchy feature before you can use this record through REST web services. This feature depends on the Custom Segments feature.

## Code Sample

### Creating a Merchandise Hierarchy Level

```

1 POST {{REST_SERVICES}}/record/v1/merchandisehierarchylevel
2 {
3   "name": "Level 2",
4   "description": "L2",
5   "insertbefore": {
6     "id": 1
7   }
8 }

```

### Updating a Merchandise Hierarchy Level

```

1 PATCH {{REST_SERVICES}}/record/v1/merchandisehierarchylevel/{{id}}
2 {
3   "name": "Level 2 updated",
4   "description": "L2 updated",
5   "insertbefore": {
6     "id": 1
7   }
8 }

```

## Merchandise Hierarchy Node

A merchandise hierarchy node record exposes a merchandise hierarchy node to REST web services. This record is not a subrecord. This record contains one subrecord with the ID **hierarchyversions**.

The REST API Browser includes information about the field names and field types of the merchandise hierarchy node record and about the HTTP methods, request parameters, and operations available to this record.

For details, see the REST API Browser's [merchandise hierarchy node](#) reference page.

For information on using the REST API Browser, see the help topic [The REST API Browser](#).

### Record ID

The record ID for the merchandise hierarchy node REST record is **merchandisehierarchynode**.

### Prerequisites

You must enable the Merchandise Hierarchy feature before you can use this record through REST web services. This feature depends on the Custom Segments feature.

### Code Sample

#### Creating a Merchandise Hierarchy Node

```
1 POST {{REST_SERVICES}}/record/v1/merchandisehierarchynode
2 {
3   "name": "Node 2",
4   "description": "N2"
5 }
```

#### Updating a Merchandise Hierarchy Node

```
1 PATCH {{REST_SERVICES}}/record/v1/merchandisehierarchynode/{{id}}
2 {
3   "name": "Node 2 updated",
4   "description": "N2 updated",
5   "hierarchyversions":{
6     "items":
7     [
8       {
9         "hierarchylevel":{
10          "id": 3
11        },
12        "hierarchyversion":{
13          "id": 2
14        },
15        "parentnode": 1,
16        "isincluded": true
17      }
18     ]
19   }
20 }
```

## Merchandise Hierarchy Version

A merchandise hierarchy version record exposes a merchandise hierarchy version to REST web services. This record is not a subrecord. This record contains one subrecord with the ID **hierarchylevels**.

The REST API Browser includes information about the field names and field types of the merchandise hierarchy version record and about the HTTP methods, request parameters, and operations available to this record.

For details, see the REST API Browser's [merchandise hierarchy version](#) reference page.

For information on using the REST API Browser, see the help topic [The REST API Browser](#).

### Record ID

The record ID for the merchandise hierarchy node REST record is **merchandisehierarchyversion**.

### Prerequisites

You must enable the Merchandise Hierarchy feature before you can use this record through REST web services. This feature depends on the Custom Segments feature.

### Code Sample

#### Creating a Merchandise Hierarchy Version

```
1 POST {{REST_SERVICES}}/record/v1/merchandisehierarchyversion
2 {
3   "name": "Version 1",
4   "description": "V1",
5   "startdate": "2024-09-01"
6 }
```

#### Updating a Merchandise Hierarchy Version

```
1 PATCH {{REST_SERVICES}}/record/v1/merchandisehierarchyversion/{{id}}
2 {
3   "name": "Version 1 updated",
4   "description": "V1 updated",
5   "startdate": "2024-09-02"
6 }
```

## Nexus

NetSuite uses the term nexus to describe a tax jurisdiction or geographic area where you do business and which has its own tax regulations. For example, a nexus can be a country, municipality, city, state, province, county, or economic zone.

For more information, see the help topic [Understanding Nexuses in SuiteTax](#).

This record is not a subrecord.

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the nexus record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [nexus](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the nexus REST record is **nexus**.

## Prerequisites

The SuiteTax feature must be enabled.

## Actions

The nexus record has the following actions through REST web services:

- Submit new
- Submit
- Reset
- Delete

## Code Samples

This code sample shows how to post the nexus record:

```
1 POST /services/rest/record/v1/nexus { "country": "US", "state": "CA", "description": "California", "parentNexus": "1", "taxA
  agency": "2"
2 }
```

This code sample shows how to get the nexus record:

```
1 GET /services/rest/record/v1/nexus/13 { "links": [ { "rel": "self", "href": "https://barochpatrik-runbox-parity-deuf1-001.eng.net
  suite.com/services/rest/record/v1/nexus/13" } ], "country": { "id": "US", "refName": "United States" }, "description": "Califor
  nia", "id": "13", "isInactive": false, "parentNexus": { "links": [ { "rel": "self", "href": "https://barochpatrik-runbox-parity-
  deuf1-001.eng.netsuite.com/services/rest/record/v1/nexus/1" } ], "id": "1", "refName": "US California" }, "state": { "links":
  [], "id": "CA", "refName": "California" }, "taxAgency": { "links": [ { "rel": "self", "href": "https://barochpatrik-runbox-parity-
  deuf1-001.eng.netsuite.com/services/rest/record/v1/vendor/2" } ], "id": "2", "refName": "Tax Agency CA" }, "taxDateFromFulfill
  ment": false
2 }
```

## Non-Inventory Purchase Item

Non-inventory items for purchase can only be bought and entered on vendor-facing transactions such as purchase orders and vendor bills. These items are purchased by your organization but not resold, such as office supplies.

To learn more, see the help topic [Non-Inventory Items](#).

This record is not a subrecord and it does not have a subrecord.

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the non-inventory purchase items record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [non-inventory purchase item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#)

## Record ID

The record ID for the non-inventory purchase item record is **noninventorypurchaseitem**.

## Actions

You can convert a non-inventory purchase item to an inventory item.

## Code Sample

### Creating a Non-Inventory Purchase Item

```
1 | POST {{COMPANY_URL}}/services/rest/record/v1/noninventorypurchaseitem { "itemId": "NONINVENTORYPURCHASEITEM_ID"
2 | }
```

### Getting a Non-Inventory Purchase Item by ID

```
1 | GET {{COMPANY_URL}}/services/rest/record/v1/noninventorypurchaseitem/{{nonInvPurchaseItemId}} { "itemId": "NEW_NONINVENTORYPURCHASEITEM_ID"
2 | }
```

### Deleting a Non-Inventory Purchase Item

```
1 | DELETE {{COMPANY_URL}}/services/rest/record/v1/noninventorypurchaseitem/{{nonInvPurchaseItemId}}
```

## Non-Inventory Resale Item

Non-inventory resale item records are used to track something you buy and then sell for a profit, but do not stock. Non-inventory items for resale can be bought and sold and appear on all applicable transaction types. This includes drop-ship items that you do not store but sell directly from the vendor.

To learn more, see the help topic [Non-Inventory Items](#).

This record does not have a subrecord.

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the non-inventory resale item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [non-inventory resale item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the non-inventory resale item REST record is **noninventoryresaleitem**.

## Inaccessible Elements

**ItemImages** is not accessible in REST web services.

## Elements with Different Functionality

The record has a matrix pricing machine. Otherwise, it behaves in the same way as the non-inventory purchase item and non-inventory resale item records.

## Actions

You can convert a non-inventory resale item to an inventory item.

## Code Sample

### Creating a Non-Inventory Resale Item

```
1 PATCH {{COMPANY_URL}}/services/rest/record/v1/itemGroup/ noninventoryresaleitem { "itemId": "52", "price": { "items":
  [ { "priceLevel": { "id": "1" }, "quantity": { "value": "5" }, "currencypage": { "id": "1" }, "price": 100 } ] }, "costEsti
  mate": 64200.0
2 }
```

### Getting a Non-Inventory Resale Item

```
1 GET {{COMPANY_URL}}/services/rest/record/v1/noninventoryresaleitem/{nonInventoryResaleItemId}
```

### Finding a Non-Inventory Resale Item by itemId

```
1 GET {{COMPANY_URL}}/services/rest/record/v1/noninventoryresaleitem?q=itemId IS MY-CUSTOM-UNIQUE-ITEM-ID
2
3 Query parameters
4
5 q={{expression}}
```

```

6 |
7 | possible expression: field {{operator}} field's value
8 |
9 | example: itemId IS MY-CUSTOM-UNIQUE-ITEM-ID

```

## Updating an Existing Non-Inventory Resale Item

```

1 | PATCH {{COMPANY_URL}}/services/rest/record/v1/ noninventoryresaleitem/{{nonInventoryResaleItemId}} { "price": { "items":
  | [{ "priceLevel": { "id": "1" }, "quantity": { "value": "5" }, "currencypage": { "id": "1" }, "price": 200 } ] }, "costEsti
  | mate": 379,
2 | }

```

## Deleting a Non-Inventory Resale Item

```

1 | DELETE {{COMPANY_URL}}/services/rest/record/v1/ noninventoryresaleitem/{{nonInventoryResaleItemId}}

```

## Non-Inventory Sale Item

You can record and track items that you always drop ship as non-inventory items. You can also record and track other items that you sell or purchase but do not stock as non-inventory items.

For more information, see the help topic [Non-Inventory Items](#).

This record is not a subrecord.

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the non-inventory sale item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [non-inventory sale item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the non-inventory sale item REST record is **noninventorySaleItem**.

## Prerequisites

There are no prerequisites for using this record in REST.

## Code Samples

The following example shows how to get record metadata for a non-inventory sale item.

### Get Record Metadata:

```

1 | GET https://<accountID>/services/rest/record/v1/metadata-catalog/nonInventorySaleItem

```

```
2 | Header: Accept: application/swagger+json
```

### Get Record:

```
1 | GET https://<accountID>/services/rest/record/v1/nonInventorySaleItem/<ID>
```

### Update Record:

```
1 | PATCH https://<accountID>/services/rest/record/v1/nonInventorySaleItem/<ID>
2 | Header: Content-Type: application/json
3 | Body:
4 | {
5 |   "itemId": "Updated item name",
6 |   "upcCode": "123456"
7 | }
```

### Update Item Price When Related Features are Enabled:

```
1 | PATCH
2 | https://<accountID>/services/rest/record/v1/nonInventorySaleItem/<ID>/price/currencypage=1,pricelevel=1,quantity=10
3 | Header: Content-Type: application/json
4 | Body:
5 | {
6 |   "price": 10
7 | }
```

## Note Type

Note type defines a list of values that are used by the note record to set the type of note. In the UI, this is a user defined list at Setup > Sales > Setup Tasks > CRM Lists.

Use the note type record to create new note types and expose them to REST web services.

To learn more, see the help topic [Note Type](#).

This record:

- is not a subrecord
- has no subrecords

There are no prerequisites for using this record through REST web services.

The REST API Browser includes information about the field names and field types of the note type record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [note type](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a note type REST record is **notetype**.

## Code Sample

The following samples show common use cases for notetype.

## Creating a Note Type

```
1 POST https://{REST_SERVICE}/services/rest/record/v1/notetype
2 Body: {
3   "description": "NoteType description 1696527953", "isInactive": false, "name": "NoteType 1696527953", "externalId": "ext_1696527953",
4 }
```

## Retrieving a Note Type

```
1 GET
2 https://{REST_SERVICE}/services/rest/record/v1/notetype/209?expandSubResources=true
```

## Updating a Note Type

```
1 PATCH https://REST_SERVICE/services/rest/record/v1/notetype/209
2 Body:
3 { "description": "Updated NoteType description 1696528478", "isInactive": true, "name": "Update NoteType 1696528478"
4 }
```

## Deleting a Note Type

```
1 DELETE https://REST_SERVICE/services/rest/record/v1/notetype/209
```

## Opportunity

An opportunity record exposes an opportunity to REST web services. This record is not a subrecord. This record has one subrecord: record ID item.

The REST API Browser includes information about the field names and field types of the opportunity record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [opportunity](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for an opportunity REST record is **opportunity**.

## Limitations

When updating an existing Opportunity record through REST, do not include the "line" element when adding an item line.

## Incorrect

```
1 { "item": { "items": [{ "item": { "id": 5 }, "line": 1, "quantity": 2.0, "rate": 12.78 }] }
2 }
```

## Correct

```
1 "item": { "items": [ { "item": { "id": 5, "quantity": 2.0, "rate": 12.78 } }
2 ] }
```

## Usage Notes

To use the **Choose Team** and **Update Customer** fields in the Sales Team subtab, the Team Selling feature must be enabled. To enable Team Selling, go to Setup > Company > Enable Features > CRM. Check the Team Selling box, and then click Save.

The Multi-Partner Management Feature must also be enabled to use the **Update Customer** field in the Relationships subtab. An administrator can enable the Multi-Partner Management feature at Setup > Company > Setup Tasks > Enable Features. Click the CRM subtab, and then under Partners, check the Multi-Partner Management box. Click Save.

## Code Samples

These samples show typical use cases for opportunity. The sample ID is 132.

## Creating an Opportunity Using a POST Request

```
1 https://0000071.suitetalk.api.snap.netsuite.com/services/rest/record/v1/opportunity
2 { "altSalesRangeHigh": 0.0, "altSalesRangeLow": 0.0, "altSalesTotal": 0, "canHaveStackable": false, "competitors": { "items":
  [ { "competitor": { "id": "1" } } ] }, "currency": { "id": "1" }, "entity": { "id": "110" }, "entityStatus": { "id": "17"
  }, "exchangeRate": 1.0, "expectedCloseDate": "2023-07-13", "forecastType": { "id": "2" }, "isBudgetApproved": false, "item":
  { "items": [ { "fromJob": false, "isEstimate": false, "item": { "id": "98" }, "marginal": false, "printItems": false, "quan
  tity": 4.0, "rate": 39.95 }, { "fromJob": false, "isEstimate": false, "item": { "id": "5" }, "marginal": false, "printItem
  s": false, "rate": 954.21 } ] }, "leadSource": { "id": "-2" }, "prevDate": "2023-07-13", "probability": 10.0, "projAltSale
  sAmt": 0.0, "projectedTotal": 100.0, "rangeHigh": 100.0, "rangeLow": 100.0, "salesTeam": { "items": [ { "contribution": 100.00, "em
  ployee": { "id": "40" }, "isPrimary": false, "salesRole": { "id": "-2" } }, { "contribution": 100.00, "employee": { "id": "134"
  }, "isPrimary": true, "salesRole": { "id": "3" } } ] }, "shipIsResidential": false, "shipOverride": false, "subsidiary":
  { "id": "1" }, "tranDate": "2023-07-13", "weightedTotal": 10.0
3 }
```

## Retrieving an Opportunity Using a GET Request

```
1 GET https://0000071.suitetalk.api.snap.netsuite.com/services/rest/record/v1/opportunity/132
```

## Updating an Opportunity Using a PATCH Request

```
1 https://0000071.suitetalk.api.snap.netsuite.com/services/rest/record/v1/opportunity/132
2 { "altSalesRangeHigh": 0.0, "altSalesRangeLow": 0.0, "altSalesTotal": 0, "canHaveStackable": false, "competitors":
  { "items": [ { "competitor": { "id": "1" } } ] }, "currency": { "id": "1" }, "entity": { "id": "110" }, "entityStatus":
  { "id": "17" }, "exchangeRate": 1.0, "expectedCloseDate": "2023-07-13", "forecastType": { "id": "2" }, "isBudgetAp
  proved": false, "item": { "items": [ { "description": "Ergonomic Keyboard", "fromJob": false, "isEstimate": false, "item":
  { "id": "98" }, "line": 1, "marginal": false, "printItems": false, "quantity": 4.0, "rate": 39.95 }, { "description": "NonIn
  vItem", "fromJob": false, "isEstimate": false, "item": { "id": "5" }, "marginal": false, "printItems": false, "rate": 954.21
  } ] }, "leadSource": { "id": "-2" }, "prevDate": "2023-07-13", "probability": 10.0, "projAltSalesAmt": 0.0, "projected
  Total": 100.0, "rangeHigh": 100.0, "rangeLow": 100.0, "salesTeam": { "items": [ { "contribution": 100.00, "employee":
  { "id": "40" }, "isPrimary": false, "salesRole": { "id": "-2" } }, { "contribution": 100.00, "employee": { "id": "134" }, "isPri
  mary": true, "salesRole": { "id": "3" } } ] }, "shipIsResidential": false, "shipOverride": false, "subsidiary": { "id": "1"
  }, "tranDate": "2023-07-13", "weightedTotal": 10.0
```

```
3 | }
```

## Deleting an Opportunity Using a DELETE Request

```
1 | DELETE https://0000071.suitetalk.api.snap.netsuite.com/services/rest/record/v1/opportunity/132
```

## Setting the Sales Order's Sales Team Members to Match Sales Group 164

```
1 | "salesGroup": {
2 |   "id": 164 }
```

## Update the Customer's Partner Team to Match the Transaction Partners

```
1 | "syncPartnerTeams": true
```

## Update the Customer's Sales Team to Match the Transaction Sales Team

```
1 | "syncSalesTeams": true
```

## Other Charge for Purchase Item

An other charge for purchase item record exposes an other charge for purchase item to REST web services. This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the other charge for purchase item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [other charge purchase item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

## Record IDs

The record ID for an other charge for purchase item REST record is **otherChargePurchaseItem**.

## Code Samples

The following samples show common use cases for other charge for purchase items. The example ID is 4.

## Creating an Other Charge for Purchase Item Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargePurchaseItem
2 | { "itemId": " Other Charge for Purchase", "displayName": "displayName", "upcCode": "code" }
```

## Retrieving an Other Charge for Purchase Item Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargePurchaseItem/4
```

## Updating an Other Charge for Purchase Item Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargePurchaseItem/4
2 | { "itemId": "Payment Item B", "displayName": "Payment Item B DisplayName", "description": "description"
3 | }
```

## Deleting an Other Charge for Purchase Item Using a DELETE Request

```
1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargePurchaseItem/4
```

## Other Charge for Resale Item

An other charge for resale item record exposes an other charge resale item to REST web services. This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the other charge for resale item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [other charge resale item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

## Record IDs

The record ID for an other charge for resale item REST record is **otherChargeResaleItem**.

## Inaccessible Elements

**ItemImages** is not accessible in REST web services.

## Elements with Different Functionality

The record has a matrix pricing machine. Otherwise, it behaves in the same way as the non-inventory purchase item and non-inventory resale item records.

## Code Samples

The following samples show common use cases for other charge resale items. The example ID is 4.

### Creating an Other Charge for Resale Item Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargeResaleItem
2 | { "itemId": " Other Charge Resale Item", "cost": "1", "incomeAccount": { "id": "54", "refName": "Sales" }
3 | }
```

### Retrieving an Other Charge for Resale Item Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargeResaleItem/4
```

### Updating an Other Charge for Resale Item Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargeResaleItem/4
2 | { "cost": "10",
3 | }
```

### Deleting an Other Charge for Resale Item Using a DELETE Request

```
1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargeResaleItem/4
```

## Other Charge for Sale Item

An other charge for sale item record exposes an other charge for sale item to REST web services. This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the other charge for sale record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [other charge sale item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for an other charge for sale item REST record is **otherChargeSaleItem**.

## Code Samples

The following samples show common use cases for other charge for sale items. The example ID is 4.

### Creating an Other Charge for Sale Item Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargeSaleItem
2 | { "itemId": " Other Charge for Sale", "displayName": "displayName", "upcCode": "code" }
```

### Retrieving an Other Charge for Sale Item Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargeSaleItem/4
```

### Updating an Other Charge for Sale Item Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/otherChargeSaleItem/4
2 | { "itemId": " Other Charge for Sale A", "displayName": "displayName A", "upcCode": "code A"
3 | }
```

## Other Name

An other name record exposes a collection of records for people or companies who are not vendors, customers or employees to REST web services.

This record is not a subrecord and has no subrecords.

With the exception of tax-related fields, all elements on this record are accessible through REST web services.

The REST API Browser includes information about the field names and field types of the other name record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [other name](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the other name REST record is **otherName**.

## Limitations

Updating legacy tax fields is not supported.

## Code Samples

These samples show common use cases for other name REST methods.

### Getting OpenAPI Metadata from the Metadata Catalog

```
1 | GET https://test.netsuite.com/services/rest/record/v1/metadata-catalog/othername
```

### Fetching an Other Name Record

```
1 | GET https://test.netsuite.com/services/rest/record/v1/othername/<id>
```

### Creating an Other Name Record

```
1 | POST https://test.netsuite.com/services/rest/record/v1/othername
2 | { "entityId": "Sophia Company", "subsidiary": { "refName": "Parent Company" }, "companyName": "test company name", "phone": "123
  123 1234", "fax": "123 123 1234", "email": "test-email@netsuite.com", "printOnCheckAs": "123", "emailPreference":
  { "refName": "PDF" }, "phoneticName": "so-FEE-uh", "comments": "Test company comment", "url": "https://test-url.com/", "image":
  { "refName": "testimage.gif" }, "creditLimit": 200, "terms": { "refName": "Net 60" }, "category": { "refName": "A" }, "accountNum
  ber": 123, "isInactive": false
3 | }
```

### Updating an Other Name Record

```
1 | PATCH https://test.netsuite.com/services/rest/record/v1/othername/<id>
2 | { "email": "updated-test-email@netsuite.com", "emailPreference": { "refName": "HTML" }, "comments": "Updated test company comment"
3 | }
```

### Deleting an Other Name Record

```
1 | DELETE https://test.netsuite.com/services/rest/record/v1/othername/<id>
```

## Other Name Category

An other name category record exposes a user-defined mapping to help categorize other name records to REST web services. Other name category records are used to help group other name records by user-created categories.

This record is not a subrecord and has no subrecords.

All elements on this record are accessible through REST web services.

The REST API Browser includes information about the field names and field types of the other name category record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [other name category](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the other name category REST record is **otherNameCategory**.

## Code Samples

These samples show common use cases for other name category REST methods.

### Getting OpenAPI Metadata from the Metadata Catalog

```
1 | GET https://test.netsuite.com/services/rest/record/v1/metadata-catalog/othernamecategory
```

### Fetching an Other Name Category Record

```
1 | GET https://test.netsuite.com/services/rest/record/v1/othernamecategory/<id>
```

### Creating an Other Name Category Record

```
1 | POST https://test.netsuite.com/services/rest/record/v1/othernamecategory
2 | { "name": "Charities", "isInactive": true
3 | }
```

### Updating an Other Name Category Record

```
1 | PATCH https://test.netsuite.com/services/rest/record/v1/othernamecategory/<id>
2 | { "name": "Nonprofits", "isInactive": false
3 | }
```

### Deleting an Other Name Category Record

```
1 | DELETE https://test.netsuite.com/services/rest/record/v1/othernamecategory/<id>
```

## Partner

A partner record exposes a partner to REST web services.

For more information about partners and how they function in the user interface, see .

To access this record in NetSuite, go to Lists > Relationships > Partners.

This record:

- is not a subrecord
- has no subrecords

The following sublists are not accessible through REST web services:

- bulkmerge
- subscriptionmessagehistory
- subscriptions
- taxregistration

The REST API Browser includes information about the field names and field types of the partner record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [partner](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#)



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

## Record ID

The record ID for the partner REST record is **partner**.

## Prerequisites

You must enable the CRM feature before you can use this record through REST web services.

## Code Sample

The following samples show common use cases for a partner record. The example ID is 5.

### Retrieving Partner Metadata

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/metadata-catalog/partner
```

### Retrieving a Partner Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/partner/5
```

### Creating a Partner Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/partner
2 | { "partnerCode": "EXAMPLE COMPANY", "companyName": "Example Company", "subsidiary": "3"
3 | }
```

### Updating a Partner Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/partner/5
2 | { "companyName": "Updated Company Name"
3 | }
```

## Deleting a Partner Using a DELETE Request

1 | **DELETE** <https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/partner/5>

## Paycheck

A paycheck record exposes a paycheck to REST web services.

This record:

- is not a subrecord
- has the following subrecords:
  - Account
  - Contact
  - payBonuses
  - payComm
  - payContrib
  - PayDeduct
  - payDisburse
  - payrollitem
  - payEarn
  - payExp
  - payPto
  - paySummary
  - payTax
  - payTime
  - accountingperiod

The REST API Browser includes information about the field names and field types of the paycheck record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [paycheck](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the paycheck REST record is **paycheck**.

## Prerequisites

Before you can use this record with REST web services, you must enable the following features:

- REST Web Services
- Token-Based Authentication
- Client SuiteScript

- Server SuiteScript
- Payroll

## Inaccessible Elements

Reversed paychecks and paycheck adjustments do not appear in the REST web services paycheck list. However they can be accessed by an administrator by appending the paycheck ID directly at the end of the URL request.

 **Note:** A known issue exists in which an employee with REST permissions can see a paycheck with a future check date in the paycheck list when the **Paychecks Visible to Employees** setting is set to **On Check Date**.

The following subrecords are not exposed in REST:

- payComm
- payExp

## Elements with Different Functionality

The following sublists are editable:

- payContrib
- payDeduct
- payEarn

Other sublists are not editable.

The editing of some of this record's subrecords and sublists works differently.

- payExp: When you edit a line item in the UI, the "Apply" checkbox can be unchecked. If the line is unchecked, the line is removed when you save. Through REST web services, this functionality is not available.
- payPto: This sublist is not returned when querying `/services/rest/record/v1/paycheck/{id}`. To retrieve this record, the user needs to query `/services/rest/record/v1/paycheck{id}/payPto`.  
payTime: When you edit a line item in the UI, the "Apply" checkbox can be unchecked. If the line is unchecked, the line is removed when you save. Through REST web services, this functionality is not available.

## Actions

The paycheck record accessed through REST web services can be edited and deleted. Accessing paycheck records through REST web services will give you information in a different format and include more detailed information about the paycheck entries. Examples include:

- The inclusion internal IDs of the payitems shown on the paycheck record
- The display of residential, editability and exemption status of taxes
- The display of REST reference URLs of subresources returned in the request
- A check date in the following format: "2023-10-23T19:12:00Z"
- Number of total results in the requested record

## Elements Related to Taxation Features

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

Fields on the paycheck record are related to payrolltaxes and to the ANYPAYROLL feature.

the payTax sublist represents a list of taxes on the paycheck.

## Additional Details

Appending "?expandSubResources=True" to the end of an individual paycheck request URL expands the obtained subresources' details.

## Code Sample

The following code shows how to access the list of paycheck records:

```
1 | https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/paycheck
```

The following code shows how to access a specific paycheck by ID (123):

```
1 | https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/paycheck/123
```

The following code shows how to access a specific paycheck by ID (123), expanding the subresources returned in the response:

```
1 | https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/paycheck/123?expandSubResources=True
```

The following code shows how to get a paycheck by ID (304):

```
1 | {
2 |   "name": "Example 3.6: Get Paycheck",
3 |   "request": {
4 |     "method": "GET",
5 |     "header": [
6 |       {
7 |         "key": "Content-Type",
8 |         "name": "Content-Type",
9 |         "value": "application/json",
10 |        "type": "text"
11 |      },
12 |      {
13 |        "key": "Accept",
14 |        "value": "application/swagger+json",
15 |        "type": "default",
16 |        "disabled": true
17 |      }
18 |    ],
19 |    "body": {
20 |      "mode": "raw",
21 |      "raw": ""
22 |    },
23 |    "url": {
24 |      "raw": "{{COMPANY_URL}}/services/rest/record/v1/paycheck/304?expandSubResources=true",
25 |      "host": [
26 |        "{{COMPANY_URL}}"

```

```

27     },
28     "path": [
29         "services",
30         "rest",
31         "record",
32         "v1",
33         "paycheck",
34         "304",
35         ""
36     ],
37     "query": [
38         {
39             "key": "expandSubResources",
40             "value": "true"
41         }
42     ]
43 }
44 }
45 }

```

The following code shows how to update a paycheck by ID (606):

```

1  {
2      "name": "Example 3.5: Update paycheck memo",
3      "request": {
4          "method": "PATCH",
5          "header": [
6              {
7                  "key": "Content-Type",
8                  "name": "Content-Type",
9                  "value": "application/json",
10                 "type": "text"
11             }
12         ],
13         "body": {
14             "mode": "raw",
15             "raw": "{\"memo\": \"new memo\"}"
16         },
17         "url": {
18             "raw": "{{COMPANY_URL}}/services/rest/record/v1/paycheck/606",
19             "host": [
20                 "{{COMPANY_URL}}"
21             ],
22             "path": [
23                 "services",
24                 "rest",
25                 "record",
26                 "v1",
27                 "paycheck",
28                 "606"
29             ]
30         }
31     }
32 }

```

The following code shows how to delete a paycheck by ID (705):

```

1  {
2      "name": "Example 2.4: Remove Paycheck",
3      "request": {
4          "method": "DELETE",
5          "header": [],
6          "body": {
7              "mode": "raw",
8              "raw": ""
9          },
10         "url": {
11             "raw": "{{COMPANY_URL}}/services/rest/record/v1/paycheck/705",
12             "host": [
13                 "{{COMPANY_URL}}"

```

```

14 |         ],
15 |         "path": [
16 |             "services",
17 |             "rest",
18 |             "record",
19 |             "v1",
20 |             "paycheck",
21 |             "705"
22 |         ]
23 |     }
24 | }
25 | }

```

## Payment Item

A payment item record exposes a payment item to REST web services. This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the payment item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [payment item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a payment item REST record is **paymentItem**.

## Code Samples

The following samples show common use cases for payment items. The example ID is 4.

### Creating a Payment Item Using a POST Request

```

1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/paymentItem
2 | { "itemId": "Payment Item A", "displayName": "Payment Item A DisplayName"
3 | }

```

### Retrieving a Payment Item Using a GET Request

```

1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/paymentItem/4

```

### Updating a Payment Item Using a PATCH Request

```

1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/paymentItem/4
2 | { "itemId": "Payment Item B", "displayName": "Payment Item B DisplayName", "description": "description"
3 | }

```

### Deleting a Payment Item Using a DELETE Request

```

1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/paymentItem/4

```

## Payment Method

NetSuite exposes the payment method record to REST web services. A payment method represents an account that transactions write into. You can create payment methods to add selections to the Payment Method field on transactions and in your Web store.

For information about working with the payment method record in the UI, see the help topic [Creating a Payment Method](#).

There are not any prerequisites required for using the payment method record through REST web services.

Note that when you create a new payment method, you have to provide the **methodtype** ID, for example "1" for the Payment Card payment method type. For a list of available payment method type IDs, see the help topic [Creating a Payment Method](#).

The REST API Browser includes information about the field names and field types of the payment method record and about the HTTP methods, request parameters, and operations available to this record.

For details, see the REST API Browser's [payment method](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the payment method REST record is **paymentMethod**.

## Sublist

The payment method record has a sublist of the **paymentMethodVisual** subrecord, which is not exposed to REST web services.

## Code Samples

The code samples in this section show common use cases for payment methods.

### Creating a Payment Method

```
1 POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/paymentMethod
2 { "name": "Credit Card PM", "methodtype": { "id": "1"}, /* Id of Payment Card type */ "cardBrands":{ "items":[ { "id": "6"},
3   { "id": "3"} ] }, "merchantAccounts":{ "items":[ { "id": "1"} ] } }
```

### Updating a Payment Method

```
1 PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/paymentMethod/{id} { "name": "Updated Payment Method"
2 }
```

## Payroll Item

A payroll item record exposes a payroll item to REST web services.

The REST API Browser includes information about the field names and field types of the payroll item record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [payroll item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the payroll item REST record is **payrollitem**.

This record:

- is not a subrecord
- has the following subrecords:
  - `assetAccount`
  - `expenseAccount`
  - `itemType`
  - `liabilityAccount`
  - `reportingSection`
  - `subsidiary`
  - `vendor`

## Prerequisites

Before you can use this record with REST web services, you must enable the Token-Based Authentication feature.

## Inaccessible Elements

A list of all employees and applying payroll items to employees is not available in REST web services. However, they can be accessed in the UI.

## Elements with Different Functionality

The following elements are display-only:

- `itemType`
- `limitType`
- `subsidiary`

## Actions

The following actions are available for the payroll item record:

- Delete
- Reset

- Save
- Save and New

## Elements Related to Taxation Features

The payroll item has the following tax-related fields that are not exposed to REST:

- companytaxoptions
- emptaxoptions

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

In REST web services, the name key for subrecords is shown as *refName*. In SOAP web services, the name key is shown as *name*.

## Code Samples

The following code sample shows how to create a payroll item:

```
1 POST
2 https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/payrollItem
3 '{
4   "companyName": "Company 1753208683",
5   "email": "customer@example.com",
6   "subsidiary": {
7     "id": 1
8   },
9   "itemType": {
10    "id": 2
11  },
12  "name": "Salary REST TEST",
13  "description": "Description for Salary REST Test Payroll Item",
14  "accountcategory": {
15    "id": "Expense"
16  },
17  "expenseAccount": {
18    "id": 84
19  }
20 }'
```

The following code sample shows how to delete a payroll item:

```
1 DELETE https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/payrollItem/20
```

The following code sample shows how to update a payroll item:

```
1 PATCH
2 https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/payrollItem/<PAYROLL_ITEM_ID>
3 {
4   "name": "Salary Test"
5 }'
```

The following code sample shows how to get a payroll item:

```
1 GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/payrollitem/21
```

```

2 {
3   "accountcategory": {
4     "id": "Expense",
5     "refName": "Expense"
6   },
7   "description": "Description for Salary REST Test Payroll Item",
8   "employeePaid": false,
9   "expenseAccount": {
10    "id": "84",
11    "refName": "Payroll Expenses"
12  },
13  "id": "21",
14  "inactive": false,
15
16  "itemType": {
17    "id": "2",
18    "refName": "Earning:Salary"
19  },
20  "limitiswithholding": {
21    "id": "wage",
22    "refName": "Wage"
23  },
24  "name": "Salary REST TEST",
25  "subsidiary": {
26    "id": "1",
27    "refName": "Parent Company"
28  }
29 }

```

## Period End Journal Entry

The Period End Journal Entries feature processes and posts subsidiary consolidation and income statement closing using a transaction type called Period End Journal. Period end journals are generated automatically when you complete the Create Period End Journals task on the Period Close Checklist.

For more information about period end journal entries in NetSuite, see the help topic [Period End Journal Entries](#). For help creating this record in the user interface, see the help topic [Period End Journal Creation Process](#).

This record isn't a subrecord and doesn't have any subrecords.

The REST API Browser includes information about the field names and field types of the period end journal entry record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [period end journal](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a period end journal entry REST record is **periodendjournal**.

## Prerequisites

The Period End Journal Entry feature is available only on NetSuite OneWorld accounts. You must enable the Accounting Periods feature to enable this feature.

To use the Period End Journal Entries feature, you must enable the feature for your account and then for specific subsidiaries. For more information, see the help topic [Enabling the Period End Journal Entries Feature](#).

## Supported Operations

The following operations are supported when working with this record through REST web services:

- GET (Read, Search for a list of period end journal entries)
- PATCH (update a specific record)

Only the **Memo** and **External ID** fields can be updated.

## Inaccessible Elements

The following sublists aren't accessible through REST web services:

- Communication
- System Information
- GL Impact

## Code Samples

The following sample shows how to retrieve a period end journal entry record with an internal ID of 1:

```
1 GET:
2 {{COMPANY_URL}}/services/rest/record/v1/periodendjournal/1
```

The following sample shows how to update the **Memo** field on a period end journal entry record with an internal ID of 1:

```
1 PATCH:
2 {{COMPANY_URL}}/services/rest/record/v1/periodendjournal/1 { "externalId": "PERJNL1", "memo": "Updated memo using REST"}
```

## Phone Call

A phone call record exposes a phone call to REST web services.

This record:

- is not a subrecord
- has one subrecord: EventFile

The REST API Browser includes information about the field names and field types of the phone call record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [phone call](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the phone call REST record is **phonecall**.

## Exposed Elements

The following phone call record elements are exposed to REST web services:

- accesslevel
- assigned
- bom (Requires Advanced Bill of Materials feature)
- bomversion (Requires Advanced Bill of Materials feature)
- company
- completed date (a change here can be overwritten by change in status when set to completed in a separate modification, and cannot be set to custom date in the same message that completes the call)
- contact
- create date (Read only)
- customform (Enumerated)
- enddate
- endtime
- externalid
- id
- lastmodifieddate (Cannot be manually changed)
- message
- owner
- phone
- priority
- relateditem
- reminderminutes (Enumerated)
- startdate
- starttime
- status
- supportcase
- timedvent
- title
- transaction
- mfgrouting (Manufacturing Routing feature required)

## Code Sample

In the following example, <accountID> represents your Account ID.

HTTP request: <https://<accountID>/services/rest/record/v1/phonecall>

## Request Body

```
1 | { "title": "Call with Tom", "phone": "650-555-1234", "priority": "MEDIUM", "starttime": "16:30", "timedevent": true
```

2 | }

## To Obtain Previously-Created Phone Call Record (where ID is 100525)

HTTP Method: GET

URL: <https://<accountID>/services/rest/record/v1/phonecall/100525>

### Response Body

```
1 { "links": [ { "rel": "self", "href": "http://<accountID>/services/rest/record/v1/phonecall/100525" }, "accessLevel": false, "assigned":
  { "links": [ { "rel": "self", "href": "http://<accountID>/services/rest/record/v1/customer/-5" }, "id": "-5", "refName": "A
    Wolfe-admin" }, "createdDate": "2020-11-25T16:54:00Z", "customForm": { "links": [], "id": "-150", "refName": "Standard Phone Call
    Form" }, "endTime": "17:30", "id": "100525", "lastModifiedDate": "2020-11-25T16:54:00Z", "owner": { "links": [ { "rel": "self",
    href": "http://<accountID>/services/rest/record/v1/customer/-5" }, "id": "-5", "refName": "-5" }, "phone": "650-555-1234",
    priority": { "links": [], "id": "MEDIUM", "refName": "Medium" }, "sendEmail": false, "startDate": "2022-11-01", "startTime":
    "16:30", "status": { "links": [], "id": "SCHEDULED", "refName": "Scheduled" }, "timedEvent": true, "timezone": "America/Los_Ange
    les", "title": "Call with Tom"
  }
2 }
```

## Price Book

A price book record exposes a SuiteBilling price book to REST web services.

This record:

- is not a subrecord
- has no subrecords

The REST API browser includes information about the field names and field types of the price book record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [price book](#) reference page.

For information about using the REST API browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a price book REST record is **pricebook**.

## Prerequisites

You must [Enabling SuiteBilling Features](#) before you can use this record through REST web services.

## Code Samples

These samples show common use cases for creating price books with all header fields.

```
1 { "subscriptionplan": { "id": "{{existing_subscription_plan_id}}", "currency": { "id": "1", "name": "My Price Book {{timestamp}}", "priceinterval": { "items": [ { "priceplan": { "id": "{{last_priceplan}}", "frequency": "MONTHLY", "repeatevery": "1", "startoffsetvalue": 1 } ] }
```

```
2 | }
```

```
1 | { "priceinterval": { "items": [ { "groupid": "438", "linenumber" : "1", "chargetype": "2", "priceplan": { "id": "1017", "frequen
cy": "MONTHLY", "repeatevery": "1", "startoffsetvalue": "8", "startoffsetunit": "MONTH", "isrequired": true, "item": { "id": "15"
}, "subscriptionplanlinenumber" : "1", "subscriptionplanline": "20" } ] }
2 | }
```

## Price Level

A price level record exposes a price level to REST web services.

To access this record in NetSuite, go to Setup > Accounting > Accounting Lists > New > Price Level.

The REST API Browser includes information about the field names and field types of the price level record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [price level](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a price level REST record is **pricelevel**.

## Code Samples

The following samples show common use cases for price levels. The example ID is 6.

### Creating a Price Level Using a POST Request

```
1 | POST: {{REST_SERVICES}}/record/v1/pricelevel
2 |
3 | {
4 |   "name": "Test Price Level",
5 |   "discountPct": -15.0,
6 |   "isInactive": false,
7 |   "isOnline": false,
8 |   "updateExistingPrices": false
9 | }
```

### Retrieving a Price Level Using a GET Request

```
1 | GET: {{REST_SERVICES}}/record/v1/pricelevel/6
```

### Updating a Price Level Using a PATCH Request

```
1 | PATCH: {{REST_SERVICES}}/record/v1/pricelevel/6
2 |
3 | {
4 |   "discountPct": -20.0,
```

```

5  "isInactive": false,
6  "isOnline": true,
7  "name": "Toolbox Price Level",
8  }

```

## Deleting a Price Level Using a DELETE Request

```

1 | DELETE: {{REST_SERVICES}}/record/v1/pricelevel/6

```

## Price Plan

A price plan record exposes a SuiteBilling price plan to REST web services.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the price plan record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [price plan](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a price plan REST record is **priceplan**.

## Prerequisites

You must [Enabling SuiteBilling Features](#) before you can use this record through REST web services.

## Code Sample

```

1 | { "currency": {"id": "1"}, "priceplantype": "2", "pricetiers": { "items": [ { "fromval": 0, "pricingoption": {"id": "-102"}, "value": 50.00 } ] } }
2 | }

```

## Pricing Group

A pricing group record exposes a pricing group to REST web services.

To access this record in NetSuite, go to Setup > Accounting > Accounting Lists > New > Pricing Group.

The REST API Browser includes information about the field names and field types of the pricing group record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [pricing group](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a pricing group REST record is **pricinggroup**.

## Prerequisites

You must enable Multiple Prices before you can use this record through REST web services. To do so, go to Setup > Company > Enable Features. On the **Transactions** subtab under Sales, check the **Multiple Prices** box, and then click **Save**.

## Actions

The pricing group record has the following actions through REST web services:

- Create
- Read
- Update
- Delete

## Code Samples

The following samples show common use cases for pricing groups. The example ID is 6.

### Creating a Pricing Group Using a POST Request

```
1 | POST {{REST_SERVICES}}/services/rest/record/v1/pricinggroup
2 | { "name": "Test Pricing Group", "externalid": "pricinggroup1", "isInactive": false
3 | }
```

### Retrieving a Pricing Group Using a GET Request

```
1 | GET {{REST_SERVICES}}/services/rest/record/v1/pricinggroup/6
```

### Updating a Pricing Group Using a PATCH Request

```
1 | PATCH {{REST_SERVICES}}/services/rest/record/v1/pricinggroup/6
2 | { "name": "Updated Pricing Group", "externalid": "pricinggroup2", "isInactive": true
3 | }
```

### Deleting a Pricing Group Using a DELETE Request

```
1 | DELETE {{REST_SERVICES}}/services/rest/record/v1/pricinggroup/6
```

## Project Task

A project task record exposes a project task to REST web services. You can use the project task record to keep track of specific activities and milestones associated with a project.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the project task record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [project task](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a project task REST record is **projecttask**.

## Prerequisites

You must enable the Project management feature before you can use this record through REST web services.

## Limitations

Tax-related fields are not supported in REST.

The following fields are not supported in REST:

- isMilestone
- isOnCriticalPath
- isSummaryTask
- lateEnd
- lateStart
- slackMinutes
- bBudgetShowCalculatedLines
- cBudgetShowCalculatedLines

The following sublists are not supported in REST:

- ProjectTaskAssignee
- ProjectTaskBillingBudget
- ProjectTaskCostBudget

## Code Sample

This sample shows how to update the record:

```

1 POST /services/rest/record/v1/projecttask
2 { "project": "328", "title": "startDateTest", "constraintType": "ASAP", "startDate": "2013-11-15"
3 }

```

## Promotion Code

Promotions enable you to track the source of revenue and to offer discounts in the form of coupons. Each promotion has a promotion code that can be applied to transactions and campaigns.

This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the promotion code record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [promotion code](#) reference page.

For information about promotion, see the help topic [Promotion](#).

For information about working with promotions in the UI, see the help topic [The Promotion Record](#).

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the promotion code REST record is **promotioncode**.

## Prerequisites

- You must enable the SuitePromotions feature before you can use this record through REST web services.
- If you need to create shipping promotions, you need to enable Charge for Shipping.
- If you need to use the auto-apply configuration, you need to enable the Auto-apply feature.
- Other subtabs or sublists may require additional features to be enabled.

## Elements with Different Functionality

The following sublist has different functionality through REST web services:

- campaigns

The following fields related to CSV import files under the Coupon Codes subtab are not accessible through REST web services:

- lastUploadDate
- lastUploadStatus
- lastUploadFile
- couponCodeFile

The following field is not accessible through REST web services:

- useMaterializedSavedSearch

**Note:** The useMaterializedSavedSearch field can be found under the Saved search preferences subtab and it applies to promotions with items saved search.

## Elements Related to Taxation Features

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The discount item for accounting field on the promotioncode REST record is related to taxation.

## Additional Details

Considerations related to features:

- For sales channels, you need both the locations and website features enabled even if you do not have the sites configured.

Considerations related to forms:

- Only forms provided by NetSuite for SuitePromotions are supported.

Considerations related to fields:

- Do not use fields and forms that do not apply to SuitePromotions or are not supported. For example, forms and fields specific to advanced promotions.
- The lastUploadDate, lastUploadStatus, lastUploadFile, and couponCodeFile fields are related to the coupon code file uploads and are not available.
- The campaigns field is not available through REST.
- When you set a dynamic customer group as audience, you must explicitly set a value to the customerGroupPreference field.
- Before updating sublists, refer the documentation for working with sublists. For example, working with items, discountItems, and so on.
- You should use the supported SuitePromotion form IDs for customForm.
- Do not use customForm to try to change the promotion type. Instead, delete the existing one and create a new one.

**Note:** The promotion type we create is based on the value of customForm field. You can create an Item promotion by setting the customForm field value to -10501. Similarly, we can use -10502 to create an Order promotion, -10503 to create a Shipping promotion, -10504 to create a Fixed price promotion, and -10507 to create a Free gift promotion.

Considerations when testing your requests:

- Open the promotion in the user interface and verify that all the values are the same as if you were to create the promotion manually.
- Verify the promotion and its sublist and tab contents in the user interface when updating it.
- Use orders that meet the conditions of the promotion and verify that the promotion is applied as expected.

Other considerations:

- In case of both single-use and multi-use coupon code types, delete and create a new promotion if you want to change the coupon code type. Updating the coupon code type from single-use to multi-use or from multi-use to single-use is not supported.
- You cannot remove a promotion that is being used in a transaction.

## Code Sample

In the following example, we create an order promotion scheduled for year 2024 applying a 10% discount for orders with a minimum amount of 150.00 and 10 automatically generated coupon codes. Also, in the request we use 10 as the discount item ID for accounting.

The following code shows how to create a promotion:

```
1 POST: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/promotionCode
2 { "customForm": { "id": "-10502" }, "name": "My order promotion", "startDate": "2024-01-01", "endDate": "2024-12-31", "combination
  Type": "ORDERTYPEEXCLUSIVE", "discount": { "id": "10" }, "whatTheCustomerNeedsToBuy": { "id": "MINIMUMORDERAMOUNTORSPECIFICITEMS"
  }, "minimumOrderAmountCheck": true, "minimumOrderAmount": 150.0, "discountType": { "id": "percent" }, "rate": 10.0, "useType":
  { "id": "SINGLEUSE" }, "codePattern": "ABC-[AN.3]-[N.3]", "numberToGenerate": 10
3 }
```

In the following example, 206 represents the promotion ID.

The following code shows how to retrieve a promotions record using a GET Request:

```
1 GET: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/promotioncode/206
```

The following code shows how you can add an optional parameter to the GET request so that the subrecords and sublists are expanded and not returned only as links:

```
1 GET: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/promotioncode/206?expandSubResources=true
```

The following code shows how to update a promotions record by removing the minimum order amount condition and setting the condition for specific items with IDs 150 and 200 and a quantity of 10:

```
1 PATCH: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/promotionCode/206
2 { "minimumOrderAmountCheck": false, "minimumOrderAmount": null, "specificItemsCheck": true, "items": { "items": [ {"item":
  {"id": 150}}, {"item": {"id": 200}} ] }, "itemQuantifier": 10
3 }
```

The following code shows how to delete a promotions record:

```
1 DELETE: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/promotioncode/206
```



**Note:** If the promotion is already being used in a transaction, you won't be able to remove the promotions record.

## Purchase Contract

This record enables you to take advantage of contracted quantity-based terms and discounts when creating purchase orders.

This record is available only when the Purchase Contracts feature is enabled at Setup > Company > Setup Tasks > Enable Features (Administrator), on the Transactions subtab.

In the UI, you access this record at Vendor Dashboard > Transactions > Enter Purchase Contracts (Administrator).

For help working with this record in the UI, see the help topic [Creating Purchase Contracts](#).

The item element on this record is not accessible through REST web services.

The purchase contract record type includes the following actions from the Actions Framework:

- New
- Make Copy
- Make Copy without Pricing Tiers
- Email

The REST API Browser includes information about the field names and field types of the purchase contract record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [purchase contract](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a purchase contract REST record is **purchaseContract**.

## Prerequisites

This record is available only when the Purchase Contract feature is enabled at Setup > Company > Setup Tasks > Enable Features (Administrator), on the Transactions subtab.

## Code Samples

The following examples show how to read, create, update, or delete a purchase contract:

### Read:

```
1 | GET {{REST_SERVICES}}/record/v1/purchaseContract/${transaction-id}
```

### Create:

```
1 | POST {{REST_SERVICES}}/record/v1/purchaseContract { "entity": { "id": {entity-id} }, "effectivitybasedon": { "id" : "ORDERDATE"
2 | }, "item": { "items": [ { "item": { "id": "${item-id}" }, "itemPricing": { "baseRate": 10.0, } }
3 | }
```

### Update:

```
1 | PATCH {{REST_SERVICES}}/record/v1/purchaseContract/${transaction-id}
2 |
3 | {
4 |   "memo" : "Memo text"
5 | }
```

### Delete:

```
1 | DELETE {{REST_SERVICES}}/record/v1/purchaseContract/${transaction-id}
```

## Purchase Order

NetSuite exposes the purchase order record to REST web services. This record has the following subrecords:

- billingAddress
- shippingAddress

The following elements are not accessible through REST web services:

- accounting books
- items
- tax detail

If the Allow Expenses on Purchases accounting preference is set, the expenses sublist is also not accessible.

All other elements on this record are accessible through REST web services.



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the purchase order record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [purchase order](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a purchase order REST record is **purchaseOrder**.

## Prerequisites

You must enable the Purchase Orders feature before you can use this record through REST web services.

## Code Samples

The following samples show common use cases for a purchase order record.

### Creating a Purchase Order

```
1 | POST https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/purchaseOrder
2 | Body:
3 | { "entity": { "id": {{vendorId}} }, "item": { "items": [{ "item": { "id": {{ItemId}} }, "rate": 10 }] }
4 | }
```

## Reading a Purchase Order

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/purchaseOrder/{{id}}?expandSubResources=true
```

## Updating a Purchase Order

```
1 | PATCH https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/purchaseOrder/{{id}}
2 | Body:
3 | { "memo": "This is a test"
4 | }
```

## Deleting a Purchase Order

```
1 | DELETE https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/purchaseOrder/{{id}}
```

## Return Authorization

A return authorization record exposes a return authorization to REST web services. This record is not a subrecord. This record has one subrecord: record ID item.

The REST API Browser includes information about the field names and field types of the return authorization record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [return authorization](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a return authorization REST record is **returnAuthorization**.

## Exceptions

These fields are not exposed:

- Taxable
- Tax
- Tax %

## Actions

The return authorization record has the following actions through REST web services:

- New
- Email
- Show Activity

- Go To Register
- GL Impact

## Label Mapping

The REST label for customer is **entity**. The corresponding UI form label is **CUSTOMER**.

## Taxation

 **Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The return authorization record includes the following fields related to taxation features:

- Taxable
- Tax
- Tax %
- Customer Tax Reg. Number
- Subsidiary Tax Reg. Number
- Tax Details Override
- Tax Point Date
- Tax Point Date Override
- Tax Registration Override
- Tax

## Usage Notes

To use the **Choose Team** and **Update Customer** fields in the Sales Team subtab, the Team Selling feature must be enabled. To enable Team Selling, go to Setup > Company > Enable Features > CRM. Check the Team Selling box, and then click Save.

The Multi-Partner Management Feature must also be enabled to use the **Update Customer** field in the Relationships subtab. An administrator can enable the Multi-Partner Management feature at Setup > Company > Setup Tasks > Enable Features. Click the CRM subtab, and then under Partners, check the Multi-Partner Management box. Click Save.

## Code Samples

These samples show common use cases for return authorizations. The example ID is 132.

### Creating a Return Authorization Using a POST Request

```
1 POST https://0000071.suitetalk.api.snap.netsuite.com/services/rest/record/v1/returnauthorization
2 { "altSalesTotal": 0.0, "canHaveStackable": false, "currency": { "id": 3 }, "deferredRevenue": 0.0, "discountTotal": 1.0, "entity":
  { "id": 164 }, "exchangeRate": 1.0, "excludeCommission": false, "item": { "items": [ { "item": { "id": 5 }, "line": 1, "quantity": 2.0, "rate": 12.78 } ] }, "netAltSalesTotal": 0.0, "salesEffectiveDate": "2023-05-30", "salesTeam": { "items": [ { "contribu
```

```

tion": 100.00, "employee": { "id": "40" }, "isPrimary": false, "salesRole": { "id": "-2" } }, { "contribution": 100.00, "employee":
{ "id": "134" }, "isPrimary": true, "salesRole": { "id": "3" } } ] }, "shipIsResidential": false, "shipOverride": false, "subto
tal": 39.95, "toBeEmailed": false, "toBeFaxed": false, "toBePrinted": false, "total": 39.95, "tranDate": "2023-07-13", "tranIsVsoe
Bundle": false, "vsoeAutoCalc": false, "weekendpreference": "ASIS"
3 }

```

## Retrieving a Return Authorization Using a GET Request

```

1 | GET https://0000071.suitetalk.api.snap.netsuite.com/services/rest/record/v1/returnauthorization/132

```

## Updating a Return Authorization Using a PATCH Request

```

1 | PATCH https://0000071.suitetalk.api.snap.netsuite.com/services/rest/record/v1/returnauthorization/132{ "altSalesTotal": 0.0, "can
HaveStackable": false, "currency": { "id": 3 }, "deferredRevenue": 0.0, "discountTotal": 1.0, "entity": { "id": 164 }, "exchang
eRate": 1.0, "excludeCommission": false, "item": { "items": [ { "item": { "id": 5 }, "line": 1, "quantity": 2.0, "rate": 12.78
} ] }, "netAltSalesTotal": 0.0, "salesEffectiveDate": "2023-05-30", "salesTeam": { "items": [ { "contribution": 100.00, "em
ployee": { "id": "40" }, "isPrimary": false, "salesRole": { "id": "-2" } }, { "contribution": 100.00, "employee": { "id": "134"
}, "isPrimary": true, "salesRole": { "id": "3" } } ] }, "shipIsResidential": false, "shipOverride": false, "subtotal": 39.95, "to
BeEmailed": false, "toBeFaxed": false, "toBePrinted": false, "total": 39.95, "tranDate": "2023-07-13", "tranIsVsoeBundle":
false, "vsoeAutoCalc": false, "weekendpreference": "ASIS"
2 }

```

## Deleting a Return Authorization Using a DELETE Request

```

1 | DELETE https://0000071.suitetalk.api.snap.netsuite.com/services/rest/record/v1/returnauthorization/132

```

## Setting the Sales Order's Sales Team Members to Match Sales Group 164

```

1 | "salesGroup": {
2 |   "id": 164 }

```

## Update the Customer's Partner Team to Match the Transaction Partners

```

1 | "syncPartnerTeams": true

```

## Update the Customer's Sales Team to Match the Transaction Sales Team

```

1 | "syncSalesTeams": true

```

## Requisition

Requisition records are used to initiate the purchase process for goods and services needed within your company. For help working with this record in the UI, see the help topic [Entering a Requisition](#).

NetSuite exposes the purchase requisition record to REST web services. This record is not a subrecord and does not contain a subrecord.

The following elements are not accessible through REST web services:

- Item
- Expense sublist
- Accounting book sublist

The REST API Browser includes information about the field names and field types of the requisition record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [requisition](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a purchase requisition REST record is **purchaserequisition**.

## Prerequisites

To use the purchase requisition REST record, you must have the Requisition feature enabled.

## Code Samples

The following code sample shows how to create a purchase requisition:

```
1 | POST {{REST_SERVICES}}/record/v1/purchaseRequisition Body:
2 | { "entity": { "id": {{entityId}} }, "item": { "items": [{ "item": { "id": {{item}} }, "rate": 10 }] }
3 | }
```

## Revenue Recognition Schedule

A revenue recognition schedule indicates the posting periods in which revenue should be recognized, and the amount to be recognized in each period, for an item sale. A revenue recognition schedule is generated for any sales transaction item that has an associated revenue recognition template. The point at which a revenue recognition schedule is generated for an item sale depends upon the type of sales transaction and enabled features and preferences set in your account. The schedule could be generated when a transaction is first saved, when it is approved, or when it is billed. Revenue recognition schedules provide a basis for the generation of journal entries that record the impact of item sales. This record is available when the Revenue Recognition feature is enabled.

For help working with this record in the UI, see the help topic [Working with Revenue Recognition Schedules](#).

Revenue recognition schedules are system-generated.

The REST API Browser includes information about the field names and field types of the revenue recognition schedule record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [revRecSchedule](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a revenue recognition schedule REST record is **RevRecSchedule**.

## Prerequisites

You must enable the following features or preferences to access this record:

- Revenue Recognition
- Set Up Auto-Generated Numbers
- Allow Users to Modify Revenue Recognition
- Sales Order Revenue Forecasting

For more information about using record, see the help topic [Working with Revenue Recognition Schedules](#).

## Limitations

All fields of the revenue recognition schedule record are read-only except for the name and recurrence fields. To modify these two fields, you must enable the Allow Users to Modify Revenue Recognition Schedule preference.

## Additional Information

If you set **isAmortization** to true, the schedule changes to amortization. If you set **isSchedule** to true, the schedule changes to revenue recognition. The **isrecognized** field appears when the **amortizationtype** is Variable. **forecastTemplate** appears when Sales Order Revenue Forecasting is enabled.

## Code Samples

The following example shows how to read a revenue recognition schedule record:

**Read:**

```
1 | GET {{REST_SERVICES}}/record/v1/RevRecSchedule/805?expandSubResources=true
```

## Revenue Recognition Template

A revenue recognition template indicates how revenue from associated items should be posted. For each template: you can select from a choice of standard terms or define your own custom terms, set the time period over which recognition occurs, define an offset to delay the start of recognition, and set up an initial amount to be recognized. This record is available when the Revenue Recognition feature is enabled.

For help working with this record in the UI, see the help topic [Creating Revenue Recognition Templates](#).

The REST API Browser includes information about the field names and field types of the revenue recognition template record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [revRecTemplate](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a revenue recognition template REST record is **revrectemplate**.

## Prerequisites

You must enable the following features or preferences to access this record:

- Revenue Recognition

For more information about using record, see the help topic [Creating Revenue Recognition Templates](#).

## Limitations

All fields of the revenue recognition template record are read-only except for the name and recurrence fields. To modify these two fields, you must enable the Allow Users to Modify Revenue Recognition Schedule preference.

The **isAmortization**, **isPublic**, and **isSchedule** fields on this record are not available in REST.

## Code Samples

The following example shows how to read a revenue recognition template record:

**Read:**

```
1 | GET {{REST_SERVICES}}/record/v1/revrectemplate/104?expandSubResources=true
```

## Sales Order

A sales order record exposes a sales order to REST web services. This record has one subrecord: salesOrderItem.

To access this record in NetSuite, go to Transactions > Sales > Enter Sales Order.



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the sales order record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [sales order](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a sales order REST record is **salesorder**.

## Usage Notes

To use the **Choose Team** and **Update Customer** fields in the Sales Team subtab, the Team Selling feature must be enabled. To enable Team Selling, go to Setup > Company > Enable Features > CRM. Check the Team Selling box, and then click Save.

The Multi-Partner Management Feature must also be enabled to use the **Update Customer** field in the Relationships subtab. An administrator can enable the Multi-Partner Management feature at Setup > Company > Setup Tasks > Enable Features. Click the CRM subtab, and then under Partners, check the Multi-Partner Management box. Click Save.

When you transform a sales order into an item fulfillment, you must include the `inventoryLocation` field in the request body. For more information, see the help topic [Transforming Sales Orders with the Intercompany Cross-Subsidiary Feature](#)

```

1 POST /record/v1/salesOrder/<source Sales Order id>/!transform/itemFulfillment
2 {
3     "inventoryLocation": {
4         "id" : <location id>
5     }
6     ...
7 }
```

You can create a sales order with an ItemGroup item but it is not possible to change or update the quantity, rate, and amount value of the member items using the create request. You can only update the quantity, rate, and amount values using the PATCH request.

## Code Samples

These samples show common use cases for sales orders. The example ID is 1504.

### A Basic GET Request of a Sales Order

```

1 GET: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/salesorder/1504
```

### Updating a Sales Order

```

1 PATCH: https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/salesorder/1504
2 {"istaxable":false}
```

### Updating the Amount of a Line Item Within a Sales Order

```

1 {
2   "item": {
3     "items": [{
4       "line": 1,
5       "item": {
6         "id": 328
7       },
8       "amount": 54.0
9     }]
10  }
11 }
```

## Setting the Sales Order's Sales Team Members to Match Sales Group 164

```
1 | "salesGroup": {
2 |   "id": 164 }
```

## Update the Customer's Partner Team to Match the Transaction Partners

```
1 | "syncPartnerTeams": true
```

## Update the Customer's Sales Team to Match the Transaction Sales Team

```
1 | "syncSalesTeams": true
```

## Sales Role

A sales role record exposes a sales role to REST web services.

For more information about sales roles and how they function in the user interface, see the help topic [NetSuite Roles Overview](#).

To access this record in NetSuite, go to Setup > Users/Roles > Manage Roles.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the sales role record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [sales role](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the sales role REST record is **salesrole**.

## Prerequisites

You must enable the Team Selling feature before you can use this record through REST web services. For more information, see the help topic [Team Selling](#).

## Code Sample

The following samples show common use cases for a sales role record. The example ID is 6.

## Retrieving Sales Role Metadata

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/metadata-catalog/salesrole
```

## Deleting a Sales Role Using a DELETE Request

```
1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/salesrole/6
```

## Sales Tax Item

For help working with this record in the UI, see the help topic [Sales Taxes](#).

This record is not a subrecord.

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the sales tax item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [salesTaxItem](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the sales tax item REST record is **salesTaxItem**.

## Prerequisites

The SuiteTax feature must be enabled.

## Actions

The sales tax item record has the following actions through REST web services:

- Submit new
- Submit

## Code Samples

This code sample shows how to post the sales tax item record:

```
1 | POST /services/rest/record/v1/salesTaxItem { "name": "Test Tax Code", "description": "Test Tax Code", "taxType": "13"
2 | }
```

This code sample shows how to get the sales tax item record:

```
1 GET /services/rest/record/v1/salesTaxItem/91 { "links": [ { "rel": "self", "href": "https://barochpatrik-runbox-parity-deuf1-001.en
g.netsuite.com/services/rest/record/v1/salesTaxItem/91" } ], "description": "Test Tax Code", "id": "91", "isInactive": false, "item
Type": "TaxItem", "name": "Test Tax Code", "taxType": { "links": [ { "rel": "self", "href": "https://barochpatrik-runbox-parity-
deuf1-001.eng.netsuite.com/services/rest/record/v1/taxtype/13" } ], "id": "13", "refName": "US Tax Type" }
2 }
```

## Service Purchase Item

A service purchase item record exposes a service purchase item report to REST web services. Service purchase item records are used to track charges for a service your business pays for that is not billed to a customer.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the service purchase item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [service purchase item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a service purchase item REST record is **servicepurchaseitem**.

## Limitations

Tax schedule-related fields are not supported in REST.

## Code Sample

This sample shows how to create the record:

```
1 POST /services/rest/record/v1/servicepurchaseitem
2 {
3   "itemId": "YourItemName"
4 }
```

## Service Resale Item

A service resale item record exposes a service resale item report to REST web services. Service resale item records are used to track charges for a service your business pays for initially and then bills to a customer.

This record:

- is not a subrecord

- has no subrecords

The REST API Browser includes information about the field names and field types of the service resale item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [service resale item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a service resale item REST record is **serviceresaleitem**.

## Limitations

Tax schedule-related fields, billing rate-related and item task template-related sublists are not supported in REST.

## Code Sample

This sample shows how to create the record:

```
1 POST /services/rest/record/v1/serviceresaleitem
2 {
3   "itemId": "YourItemName"
4 }
```

## Service Sale Item

A service sale item record exposes a service sale item report to REST web services. Service sale item records are used to track charges for a service your business performs that is billed to a customer.

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the service sale item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [service sale item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a service sale item REST record is **servicesaleitem**.

## Limitations

Tax schedule-related fields, billing rate-related and item task template-related sublists are not supported in REST.

## Code Sample

This sample shows how to create the record:

```
1 POST /services/rest/record/v1/servicesaleitem
2 {
3   "itemId": "YourItemName"
4 }
```

## Ship Item

A ship item record exposes a ship item to REST web services. This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the ship item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [ship item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a ship item REST record is **shipitem**.

## Taxation

**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The ship item record includes the following fields related to taxation features:

- Charge Tax on This Shipping Portion of Item
- Charge Tax on This Handling Portion of Item

## Code Samples

The following samples show common use cases for ship items. The example ID is 4.

### Creating a Ship Item Using a POST Request

This sample shows how to create a \$10.00 flat-rate ship item.

```
1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/shipitem
2 {
3   "itemId": "Flat Rate Priority Shipping", "displayname": "Flat Rate Priority Shipping", "description": "Only to be used for prior
ity orders - flat rate 10.00", "subsidiary": "1", "isonline": false, "account": "54", "costbasis": "fr", "shippingflatrateamoun
t": 10.00, "accounthandling": "238", "taxschedule": "1", "taxschedulehandling": "1"
3 }
```

## Retrieving a Ship Item Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/shipitem/4
```

## Updating a Ship Item Using a PATCH Request

This sample shows how to update fields on a ship item.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/shipitem/4
2 | { "itemid": "Flat Rate Priority Shipping - $15", "displayname": "Flat Rate Priority Shipping - $15", "description": "Only to be
   |   used for priority orders - flat rate 15.00", "shippingflatrateamount": 15.00
3 | }
```

## Deleting a Ship Item Using a DELETE Request

```
1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/shipitem/4
```

## Subscription

A subscription record exposes a SuiteBilling subscription to REST web services. This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the subscription record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [subscription](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a subscription REST record is **subscription**.

## Prerequisites

You must [Enabling SuiteBilling Features](#) before you can use this record through REST web services.

## Limitations

A subscription REST record does not show change orders, related records, or system information.

## Code Samples

These samples show common use cases for creating subscriptions.

## Create with all Header Fields

```
1 { "name": "REST-Subscription", "customer": 4, "billingAccount": 1, "subscriptionPlan": 8, "priceBook": 1, "initialTerm": {"id" : -102}, "startDate": "2019-2-20", "billingSchedule": 1, "billingFrequency": "MONTHLY", "defaultRenewalTerm": {"id" : 2}, "autoRenewal": true, "advanceRenewalPeriodNumber": 5, "advanceRenewalPeriodUnit": "DAYS", "defaultRenewalMethod": "CREATE_NEW_SUBSCRIPTION", "defaultRenewalPriceBook": {"id" : 103}, "defaultRenewalPlan": {"id" : 8}, "defaultRenewalTranType": "SalesOrd", "endDate": "2022-3-5"
2 }
```

## Create with Different Pricing Values

```
1 { "name": "REST-Subscription_with_pricing", "customer": 4, "billingAccount": 1, "subscriptionPlan": 8, "priceBook": 1, "initialTerm": {"id" : 1}, "startDate": "2019-2-20", "billingFrequency": "MONTHLY", "priceinterval": { "items": [{ "subscriptionplanlinenum ber": 2, "quantity": 3, "repeatEvery": 2, "pricePlan": {"id": 820}, "frequency": "ANNUALLY" }] }
2 }
```

## Create a New Price Interval

```
1 { "name": "REST-Subscription_with_pricing", "customer": 4, "billingAccount": 1, "subscriptionPlan": 8, "priceBook": 1, "initialTerm": {"id" : 1}, "startDate": "2019-2-20", "billingFrequency": "MONTHLY", "priceinterval": { "items": [{ "subscriptionplanlinenum ber": 2, "linenum": 2, "quantity": 2, "startDate": "2019-4-20", "pricePlan": {"id": 1020}, "frequency": "MONTHLY", "repeatEvery": 1 }] }
2 }
```

## Create with Different Subline Values

```
1 { "name": "REST-Subscription_with_pricing", "customer": 4, "billingAccount": 1, "subscriptionPlan": 8, "priceBook": 1, "initialTerm": {"id" : 1}, "startDate": "2019-2-20", "billingFrequency": "MONTHLY", "subscriptionline": { "items": [{ "linenum": 1, "includeinrenewal": true, "revrecoption": "OVER_TERM" }, { "linenum": 2, "proratestartdate": false, "prorateenddate": false, "includeinrenewal": false }] }
2 }
```

## Create with an Add-On Item

```
1 { "name": "REST-Subscription_with_pricing", "customer": 4, "billingAccount": 1, "subscriptionPlan": 8, "priceBook": 1, "initialTerm": {"id" : 1}, "startDate": "2019-2-20", "billingFrequency": "MONTHLY", "subscriptionline": { "items": [{ "linenum": 4, "includeinrenewal": true, "item": 5, "subscriptionlinetype": 2, "billingMode": "IN_ADVANCE", "proratestartdate": false, "prorateenddate": true, "isincluded": false }] }, "priceinterval": { "items": [{ "subscriptionplanlinenum": 4, "linenum": 4, "quantity": 3, "repeatEvery": 2, "pricePlan": {"id": 1217}, "frequency": "ANNUALLY" }] }
2 }
```

## Subscription Line

A subscription line record exposes a SuiteBilling subscription line to REST web services.

This record is not a subrecord, but is always tied to a subscription.

The REST API Browser includes information about the field names and field types of the subscription line record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [subscription line](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a subscription line REST record is **subscriptionline**.

## Prerequisites

You must [Enabling SuiteBilling Features](#) before you can use this record through REST web services.

## Limitations

A subscription line REST record does not show system notes.

You cannot directly create or delete the exposed subscription line record, but you can update some of the fields.

## Code Sample

```
1 | Sample Update:
2 | { "subscriptionlinestatus": "PENDING_ACTIVATION", "proratestartdate": false, "prorateenddate": false, "includeinrenewal": false
3 | }
```

## Subscription Plan

A subscription plan is a stand-alone record that exposes a SuiteBilling subscription plan to REST web services.

A subscription requires a subscription plan, but the subscription plan is not always tied to a subscription.

The REST API Browser includes information about the field names and field types of the subscription plan record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [subscription plan](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a subscription plan REST record is **subscriptionplan**.

The record ID for a subscription plan REST subrecord is **subscriptionplanmember**.

## Prerequisites

You must [Enabling SuiteBilling Features](#) before you can use this record through REST web services.

## Limitations

A subscription plan REST record does not show price books or related records.

When you access this record through REST web services, there is no link available for creating price books.

## Code Sample

This sample shows a common use case for creating a subscription plan.

### Create with all Header Fields

```
1 { "itemid": "NotRested-2", "initialterm" : { "id" : 1 }, "member": { "items": [{ "item": { "id": 5 }, "isrequired": true, "subscriptionlinetype": "2", "renewaloption": "DIFFERENT_PLAN" }] }
2 }
```

## Subscription Term

A subscription term is a stand-alone record that exposes a SuiteBilling subscription term to REST web services.

The REST API Browser includes information about the field names and field types of the subscription term record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [subscription term](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a subscription term REST record is **subscriptionterm**.

## Prerequisites

You must [Enabling SuiteBilling Features](#) before you can use this record through REST web services.

## Code Sample

This sample shows a common use case for creating a subscription plan.

### Create a Six-Month Subscription Term

```
1 POST services/rest/record/v1/subscriptionterm
2 { "name": "Six Months", "subscriptionTermDuration": 6, "subscriptionTermUnit": { "id": "MONTHS" }
3 }
```

## Statistical Journal Entry

The statistical journal entry record lets you increase or reduce the balance of statistical accounts.

To use this record, the Statistical Accounts feature must be enabled at Setup > Company > Setup Tasks > Enable Features (Administrator), on the Accounting subtab. Also, you must have already created at least one statistical account. In the UI, you access this record at Transactions > Financial > Make Journal Entries > List (Administrator).

For help working with this record in the UI, see the help topic [Working with Statistical Journal Entries](#).

The REST API Browser includes information about the field names and field types of the statistical journal entry record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [statistical journal entry](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the statistical journal entry REST record is **statisticaljournalentry**.

The statistical journal entry record does not include any subrecords.

## Limitations

Read-only sublists, including system notes, are not supported in REST.

## Code Sample

```
1 { "approved": true, "createdDate": "2023-08-03T06:27:00Z", "customForm": { "id": "30" }, "lastModified
  Date": "2023-08-03T06:27:00Z", "line": { "items": [ { "account": { "id": "122" }, "debit": 10 } ] }, "memo": "memo test", "posting
  Period": { "id": "267" }, "reversalDefer": false, "subsidiary": { "id": "4" }, "tranDate": "2023-08-02", "unitsType": { "id": 1
  }, "tranId": "1", "unit": 1
2 }
```

## Subsidiary

NetSuite exposes the subsidiary record to REST web services. The subsidiary record enables you to manage data for a hierarchical structure of separate legal entities. See the help topic [Subsidiaries in OneWorld](#).



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the subsidiary record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [subsidiary](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the subsidiary REST record is **subsidiary**.

The record IDs for the subsidiary REST subrecords are:

- **mainaddress**
- **shippingaddress**
- **returnaddress**

## Prerequisites

You must either use NetSuite OneWorld or have the Subsidiaries hidden feature enabled before you can use this record through REST web services.

## Exposed Elements

The following subsidiary record elements are exposed to REST web services:

- (internalid) - internal id
- (externalid) - external id
- (isinactive) - subsidiary is inactive
- (legalname) - legal name
- (name) - name
- (fullname) - (search-only field)
- (parent) - subsubsidiary of
- (currency) - currency
- (iselimination) - elimination
- (country) - country
- (state) - state/province
- (url) - web site
- (email) - return email address
- (fax) - fax

The following subsidiary subrecord elements are exposed to REST web services:

- (mainaddress) - main address
- (shippingaddress) - shipping address
- (returnaddress) - return address

## Limitations

Only GET (Read, Search) operation is supported for REST web services. POST (create), PATCH (update), and DELETE are not supported. Tax-related fields are not supported in REST.

## Code Samples

The following code samples show a common use case for getting supported elements from the subsidiary REST record.

```
1 | {GET /services/rest/record/v1/subsidiary/<id>}
```

```
1 | {GET /services/rest/record/v1/subsidiary?q=fullName CONTAIN "parent"}
```

## Subtotal Item

A subtotal item record exposes a subtotal item to REST web services. This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the subtotal item record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [subtotal item](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a subtotal item REST record is **subtotalItem**.

## Code Samples

The following samples show common use cases for subtotal items. The example ID is 4.

### Creating a Subtotal Item Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/subtotalItem
2 | { "itemId": " Subtotal Item A", "displayName": "Subtotal Item A DisplayName"
3 | }
```

### Retrieving a Subtotal Item Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/subtotalItem/4
```

### Updating a Subtotal Item Using a PATCH Request

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/subtotalItem/4
2 | { "itemId": "Payment Item B", "displayName": "Payment Item B DisplayName", "description": "description"
3 | }
```

### Deleting a Subtotal Item Using a DELETE Request

```
1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/subtotalItem/4
```

## Support Case

Support cases are support issues logged for specific companies and used for case management.

Use the support case record to create new support cases and expose them to REST web services.

To learn more, see the help topic [Support Management](#).

This record:

- is not a subrecord
- has one subrecord: supportcaseescalatehist

The REST API Browser includes information about the field names and field types of the support case record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [support case](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a support case REST record is **supportcase**.

## Prerequisites

You must enable the Lists -> Case permission before you can use this record through REST web services.

## Elements Not Accessible Through REST Web Services

The following fields are present in the UI but not accessible using REST web services:

- initialresponsetime
- timeelapsed
- timeopen
- timetoassign

The following fields are not accessible through REST web services:

- autoname
- customform
- emailform
- emailemployees
- externalid
- escalationmessage
- firstissueremoved
- helpdesk
- htmlmessage
- incomingmessage
- internalonly
- mediaitem
- messagenew
- messagesave
- origbinactive
- outgoingmessage

- transactionid
- timeitem
- timeonhold
- timetoclose
- solutions
- serialnumber
- useemployeetemplate

## Additional Details

The support case record is complex because it is integrated with many features.

Some elements of the support case form behave differently when accessed through REST web services.

## Code Sample

The following samples show common use cases for support cases.

### Creating a Support Case

```
1 POST https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/supportCase
2 With body: { "title": "some subject", "company": { "id": 931 }
3 }
```



**Note:** To create a support case, you need to have the permission of lists > customers with at least a value of view.

### Reading a Support Case

```
1 GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/supportCase/{id}
```

### Updating a Support Case

```
1 PATCH https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/supportCase/{id}
2 Body:
3 { "title": "different subject"
4 }
```

### Deleting a Support Case

```
1 DELETE https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/supportCase/{id}
```

## Task

A task record exposes a task to REST web services.

This record:

- is not a subrecord
- has the following subrecords
  - EventFile
  - TaskCompanyMap
  - TaskContactMap

The REST API Browser includes information about the field names and field types of the task record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [task](#) reference page.

For more information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the task REST record is **task**.

## Exposed Elements

The following task record elements are exposed to REST web services:

- accesslevel
- assigned
- bom (Requires Advanced Bill of Materials feature)
- bomrevision ( Requires Advanced Bill of Materials feature)
- company
- contact
- createdate (Read only)
- customform
- duedate
- endtime
- externalid
- id
- nessage
- order
- owner
- percentcomplete
- priority
- relateditem
- reminderminutes
- sendemail
- startdate
- starttime
- status

- supportcase
- timedevent
- title
- transaction
- usernotes

## Code Sample

In the following example, <accountID> represents your account ID.

HTTP request: `https://<accountID>/services/rest/record/v1/task`

## Request Body

```
1 { "title": "Prepare sales slides", "message": "Charts are required", "priority": "HIGH", "duedate": "2022-11-03", "timedevent":
  true
2 }
```

## To Obtain Previously-Created Task Record (where ID is 100526)

HTTP Method: GET

URL: `https://<accountID>/services/rest/record/v1/task`

## Response Body

```
1 { "links": [ { "rel": "self", "href": "http://<accountID>/services/rest/record/v1/task/100526" } ], "accessLevel": false, "as
  signed": { "links": [ { "rel": "self", "href": "http://<account>/service/rest/record/v1/customer/-5" } ], "id": "-5", "refName":
    "A Wolfe-admin", "createdDate": "2020-11-25T17:13:00Z", "links": [], "id": "-120", "refName": "Standard Task Form", "dueDate":
    "2022-11-03", "endTime": "10:00", "id": "100526", "lastModifiedDate": "2020-11-25T17:13:00Z", "message": "Charts are required",
    "owner": { "links": [ { "rel": "self", "href": "http://<accountID>/services/rest/record/v1/customer/-5" } ], "id": "-5",
    "refName": "-5", "percentTimeComplete": 0.0, "priority": { "links": [], "id": "HIGH", "refName": "High", "sendEmail": false,
    "startDate": "2020-11-25", "startTime": "09:00", "status": { "links": [], "id": "NOTSTART", "refName": "Not Started", "timedE
    vent": true, "timeRemaining": "0:00", "timeRemaining": "0:00", "timezone": "America/Los_Angeles",
2 "title": "Prepare sales slides"
3 }
```

## Tax Type

A tax type is associated with a nexus, and tells the system where to post the tax amounts on the balance sheet. For each nexus, you must assign at least one account in your general ledger for posting tax.

For more information about tax types in SuiteTax, see the help topic [Understanding Tax Types and Tax Codes in SuiteTax](#).



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the tax type record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [tax type](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a taxtype REST record is **taxType**.

## Subrecord

This record has the **taxTypeNexusAccounts** subrecord.

## Prerequisites

The SuiteTax features must be enabled.

## Actions

The tax type record has the following actions through REST web services:

- Submit new
- Submit
- Reset
- Delete

## Code Samples

This code sample shows how to post the tax type record:

```
1 POST /services/rest/record/v1/taxType { "country": "US", "name": "US Tax Type", "description": "US Tax Type", "doesNotAddToTotal":
  true, "nexusAccounts": { "items": [ { "nexus": { "id": "1" }, "payablesAccount": { "id": "24" }, "receivablesAccount": { "id": "6"
  } } ] }
2 }
```

This code sample shows how to get the tax type records:

```
1 GET /services/rest/record/v1/taxType/13
2 { "links": [ { "rel": "self", "href": "https://barochpatrik-runbox-parity-deuf1-001.eng.netsuite.com/services/rest/record/v1/
  taxType/13" } ], "country": { "id": "US", "refName": "United States" }, "description": "US Tax Type", "doesNotAddToTotal":
  true, "id": "13", "isInactive": false, "name": "US Tax Type", "nexusAccounts": { "links": [ { "rel": "self", "href": "http
  s://barochpatrik-runbox-parity-deuf1-001.eng.netsuite.com/services/rest/record/v1/taxtype/13/nexusAccounts" } ] }, "postToItem
  Cost": false, "reverseCharge": false, "taxInNetAmount": false
3 }
```

## Term

NetSuite exposes the term record to REST web services. Terms are used to specify when payment is due on your customers' invoices. Define the specific requirements of a term of payment by creating a term record. You can create different payment terms for different customers. In the UI, this is a user defined

list at Setup > Accounting > Setup Tasks > Accounting Lists > New. For more information about payment terms, see the help topic [Creating Terms of Payment](#).

## Record ID

The record ID for the term REST record is **term**.

## Subrecord

The term record has the CustomInstallmentPercent subrecord.

Note that certain fields become available only after you enable the Installments feature.

There are two types of terms: standard and date driven. The type is determined by the dateDriven field. Certain fields are valid only for one of the term types.

The REST API Browser includes information about the field names and field types of the estimate record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [term](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Code Samples

The code samples in this section show common use cases for standard and date driven terms.

### Creating a Standard Term

```
1 POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/term
2 { "dateDriven": false, "name": "St Term name", "daysuntilnetdue": 15, "discountpercent": 10, "daysuntilexpiry": 5
3 }
```

### Creating a Date Driven Term

```
1 POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/term
2 { "dateDriven": true, "name": "DD Term name", "dayofmonthnetdue": 15, "duenextmonthifwithindays": 10, "discountpercentdatedriv
3   en": 5, "daydiscountexpires": 5
  }
```

### Updating a Standard Term

```
1 PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/term/{id}
2 { "name": "Updated St Term", "daysuntilnetdue": 17, "discountpercent": 17, "daysuntilexpiry": 7 }
```

### Updating a Date Driven Term

```
1 PATCH https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/term/{id}
2 { "name": "Updated DD Term", "dayofmonthnetdue": 17, "duenextmonthifwithindays": 7, "discountpercentdatedriv
3   en": 7, "daydiscountexpires": 7
  }
```

## Time Bill (Track Time)

A time transaction, also known as TimeBill, records the hours worked by an employee. You can use this transaction to record billable hours and invoice customers. This transaction is available when the Time Tracking feature is enabled.



**Important:** The TimeBill record is labelled as the Track Time record in the UI.

The REST API Browser includes information about the field names and field types of the time bill record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [time bill](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a time bill REST record is **timebill**.

## Prerequisites

You must enable the Time Tracking feature before you can use this record through REST web services.

## Code Sample

The following sample shows how to update a timebill record with id=28, where you want to change the number of hours to 4:00 and set a memo explaining what occurred.

```
1 PATCH https://3553905.suitetalk.api.netsuite.com/services/rest/record/v1/timebill/28Request headers
2 Content-Type: application/json
3 Authorization: OAuth realm="3553905",oauth_consumer_key="secret",oauth_token="secret",oauth_signature_method="H
  MAC-SHA256",oauth_timestamp="1643201657",oauth_nonce="iuZQtDG47ET",oauth_version="1.0",oauth_signature="uW43ATVUbQM
  CFD6TE%2FLoGHUqkQqiZ59x00YnU0tJAo%3D"
4 User-Agent: PostmanRuntime/7.29.0
5 Accept: */*
6 Postman-Token: c559cac0-1f7a-4275-9037-4c97ff85cb57
7 Host: 3553905.suitetalk.api.netsuite.com
8 Accept-Encoding: gzip, deflate, br
9 Connection: keep-alive
10 Content-Length: 71
11 Cookie: NS_ROUTING_VERSION=LAGGING
12 Request Body
13 { "memo": "I updated the hours to reflect reality", "hours": "4:00" }
```

## Topic

Knowledge base solutions are organized by a hierarchy of topics and subtopics. When you publish a topic, you also publish its subtopics and the solutions attached to these subtopics.

Use the topic record to create new topics and expose them to REST web services.

To learn more, see [Support Case](#).

This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the topic record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [topic](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a topic REST record is **topic**.

## Prerequisites

You must enable Enable Features > CRM > Support > Knowledge Base feature and Permissions > Lists > Knowledge Base before you can use this record through REST web services.

## Elements Not Accessible Through REST Web Services

The following elements are not accessible through REST web services:

- subtopics
- solutions
- notes
- publish

## Usage Notes

Review the following notes on working with the topic REST record:

- title is required, all other fields are optional

## Code Sample

The following samples show common use cases for topic.

### Creating a Topic

```
1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/topic
2 Body: { "title": "REST title", "description": "<p>A Description</p>", "isInactive": false, "longDescription": "<p>Long Descrip
tion</p>", "parentTopic": 2
3 }
```

### Retrieving a Topic

```
1 GET
```

```
2 | https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/topic/4
```

## Updating a Topic

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/topic/4
2 | Body:{ "title": "Updated REST title", "description": "<p>Updated Description</p>", "isInactive": true, "longDescription": "<p>Up
3 | dated Description</p>", "parentTopic": 3
  | }
```

## Deleting a Topic

```
1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/paymentItem/4
```

## Transfer Order

A transfer order transaction records the change in location of inventory items within the same subsidiary. It also records the quantity and shipping details.

This transaction is available when the Locations feature and the Multi-Location Inventory (MLI) feature are enabled. A transfer order can initialize item fulfillment and item receipt transactions. It has one subrecord: inventory detail. See the help topics [Item Fulfillment](#) and [Item Receipt](#). For more information about transfer orders, see the help topic [Inventory Transfer Orders](#).

The REST API Browser includes information about the field names and field types of the transfer order record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [transfer order](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a transfer order REST record is **transferorder**.

The record ID for an inventory detail REST subrecord is **inventorydetail**.

## Prerequisites

There are no prerequisites for using the record in REST.

## Code Samples

The following sample shows you how to update a transfer order:

```
1 | PATCH {{REST_SERVICES}}/record/v1/transferorder/118 { "memo": "test", "currency": { "id": "1", "refName": "British pound"
  | }, "incoTerm": { "id": "1", "refName": "DAP" }, "item": { "items": [ { "amount": 0.0, "inventoryDetail": { "inventoryAssignment":
  | { "items": [], "totalResults": 0 } }, "isClosed": true, "item": { "id": "20" }, "itemType": { "id": "InvPart" }, "line": 1, "quan
  | tity": 10.0, "rate": 0.0 }, { "amount": 0.0, "inventoryDetail": { "inventoryAssignment": { "items": [], "totalResults": 0
  | } }, "isClosed": true, "item": { "id": "21" }, "itemType": { "id": "InvPart" }, "line": 4, "quantity": 5.0, "rate": 0.0
  | } ] }, "location": { "id": "1", "refName": "WMS Warehouse 1" }, "orderStatus": { "id": "B", "refName": "B" }, "subsidiary":
```

```
{ "id": "1", "refName": "WMS" }, "transferLocation": { "id": "2", "refName": "WMS Warehouse 2" }, "status": { "id": "Closed", "refName": "Closed" } }
```

## Unit of Measure

A unit of measure record exposes a unit of measure to REST web services. The unit of measure is also known as a unit type. This record is not a subrecord.

The REST API Browser includes information about the field names and field types of the unit type record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [units type](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a unit of measure REST record is **unitsType**.

## Code Samples

The following samples show common use cases for unit of measure. The example ID is 4.

### Creating a Unit of Measure Using a POST Request

```
1 POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/unitsType
2 { "name": "unit A", "uom": { "items": [ { "abbreviation": "UA", "baseUnit": true, "conversionRate": 1.0, "inUse": false, "pluralAbbreviation": "UsA", "pluralName": "units A", "unitName": "unit A" } ] }
3 }
```

### Retrieving a Unit of Measure Using a GET Request

This sample shows how to retrieve an existing units type.

```
1 GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/unitsType/4
```

This sample shows how to retrieve an existing units type unit of measure with the internal id of 567.

```
1 GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/unitsType/4/uom/567
```

### Updating a Unit of Measure Using a PATCH Request

This sample shows how to update an existing units type.

```
1 PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/unitsType/4
2 {
3   "name": "unit B",
4   "description": "Unit description",
5 }
```

This sample shows how to update an existing unit of measure.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/unitsType/4
2 | { "abbreviation": "UBx", "baseUnit": true, "conversionRate": 1.0, "pluralAbbreviation": "UsBx", "pluralName": "units Bx", "unit
  |   Name": "unit Bx"
3 | }
```

## Deleting a Unit of Measure Using a DELETE Request

```
1 | DELETE https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/unitsType/4
```

## Usage

A usage record exposes SuiteBilling usage to REST web services. This record:

- is not a subrecord
- has no subrecords

The REST API Browser includes information about the field names and field types of the usage record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [usage](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a usage REST record is **usage**.

## Prerequisites

You must [Enabling SuiteBilling Features](#) before you can use this record through REST web services.

This record also requires, at a minimum, a customer, subscription, subscription plan, and subscription line.

## Code Sample

This sample shows a common usage case.

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/usage
2 | { "memo": "Record of usage", "usageQuantity": 33, "usageDate": "2020-07-14", "customer": 5, "subscriptionPlan": 8, "usageSubscrip
  |   tion": 208, "usageSubscriptionLine": 235
3 | }
```

## Vendor

A vendor is a company or person you purchase goods and services from. Vendor records track information about your vendors and enable you to view past transactions and communications with

them. For information about working with vendor records in the UI, see the help topic [Vendor Records Overview](#).

NetSuite exposes the vendor record to REST web services. This record enables you to manage a specific vendor record.

All elements on this record are accessible through REST web services.

The REST API Browser includes information about the field names and field types of the vendor record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [vendor](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a vendor REST record is **vendor**.

## Code Samples

The following code sample shows how to retrieve information about a specific vendor record with the id 94:

```
1 | GET https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/vendor/94
```

The following code sample shows how to create a new vendor record:

```
1 | POST https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/vendor
2 | { "id": "95", "legalName": "AU Vendor"
3 | }
```

## Vendor Bill

Vendor bills are records which you can use to track payables as they arrive from vendors.

For information about working with vendor bills in the UI, see the help topic [Vendor Bills](#).

NetSuite exposes the vendor bill record to REST web services. This record enables you to manage a specific vendor bill record.

The REST API Browser includes information about the field names and field types of the vendor bill record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [vendorBill](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

The vendor bill record contains the following subrecord:

- billingAddress

The following elements are not accessible through REST web services:

- expensePlanMessage
- accountingBookDetail

## ■ taxDetails

## Record ID

The record ID for a vendor bill REST record is **vendorBill**.

## Code Samples

The following samples show some common actions for a vendor bill.

### Creating a Vendor Bill

```
1 | POST {{COMPANY_URL}}/services/rest/record/v1/vendorBill
2 | { "entity":{"id": {{vendorId}} }, "item": { "items": [{ "item": {"id": {{ItemId}} }, "rate": 10 }] }
3 | }
```

### Reading a Vendor Bill

```
1 | GET {{COMPANY_URL}}/services/rest/record/v1/vendorBill/{{id}}?expandSubResources=true
```

### Updating a Vendor Bill

```
1 | PATCH {{COMPANY_URL}}/services/rest/record/v1/vendorBill/{{id}}
2 | { "memo": "This is a test"
3 | }
```

### Deleting a Vendor Bill

```
1 | DELETE {{COMPANY_URL}}/services/rest/record/v1/vendorBill/{{id}}
```

## Vendor Category

A vendor category is a record that is used to group vendors. For information about creating vendor categories in the UI, see the help topic [Setting Up Accounting Lists](#).

NetSuite exposes the vendor category record to REST web services. This record is not a subrecord and does not contain subrecords.

The REST API Browser includes information about the field names and field types of the vendor category record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [vendor category](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a vendor category REST record is **vendorcategory**.

## Code Sample

The following code sample shows how to create a vendor category:

```
1 POST {{REST_SERVICES}}/record/v1/vendorCategory Body:
2 { "name": "name"
3 }
```

## Vendor Credit

A vendor credit transaction creates a credit from a vendor that can be applied to a payables account. For example, a vendor credit transaction may occur when items are returned to a vendor or when a discount is negotiated with a vendor.

For information about working with vendor credits in the UI, see the help topic [Vendor Credits](#).

NetSuite exposes the vendor credit record to REST web services. This record contains the subrecord **billingAddress**.

The REST API Browser includes information about the field names and field types of the vendor credit record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [vendor credit](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a vendor credit REST record is **vendorcredit**.

## Code Sample

The following code sample shows how to create a vendor credit record:

```
1 POST {{REST_SERVICES}}/record/v1/vendorCredit Body:
2 { "entity": { "id": {{vendorId}} }, "item": { "items": [{ "item": { "id": {{itemId}} }, "rate": 10, "location": { "id":
3   {{locationId}} } ] } }
```

## Vendor Payment

A vendor payment transaction posts to the general ledger as an expense and the amount of the payment is deducted from your accounts payable total. A vendor payment can be applied to one or more vendor bills. Vendor payments can help to track expenditures and total payables due.

For help working with this record in the UI, see the help topic [Vendor Payments Overview](#).

The vendor payment record type includes the following actions from the Actions Framework:

- New
- Fax

- Show activity
- Go to register
- GL Impact

The REST API Browser includes information about the field names and field types of the vendor payment record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [vendor payment](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a vendor payment REST record is **vendorPayment**. Vendor payment has one subrecord, with the record ID **payeeAddress**.

## Prerequisites

There are no prerequisites to using this record through REST web services.

## Code Samples

The following examples show how to read, create, update, or delete a vendor payment:

### Read:

```
1 | GET {{REST_SERVICES}}/record/v1/vendorPayment/${transaction-id}
```

### Create:

```
1 | POST {{REST_SERVICES}}/record/v1/vendorPayment { "entity": { "id": "${entity-id}" }, "apply": { "items": [ { "doc":
  { "id": "${bill-id}" }, "apply": true, "amount": ${amount} } ] }
2 | }
```

### Update:

```
1 | PATCH {{REST_SERVICES}}/record/v1/vendorPayment/${transaction-id}
2 |
3 | {
4 |   "externalId" : "AAA"
5 | }
```

### Delete:

```
1 | DELETE {{REST_SERVICES}}/record/v1/vendorPayment/${transaction-id}
```

## Vendor Prepayment

A vendor prepayment is a posting transaction that impacts the general ledger without offsetting the accounts payables account. When the vendor prepayment is applied, the accounts payables account is

offset. Vendor prepayments record and track payments to vendors before they accept a purchase order for a good or service. You can apply these prepayment amounts against open bills for the vendor.

For information about working with vendor prepayments in the UI, see the help topic [Entering Vendor Prepayments](#).

NetSuite exposes the vendor prepayment record to REST web services. This record is not a subrecord and does not contain subrecords.

The REST API Browser includes information about the field names and field types of the vendor prepayment record. It also includes information about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [vendor prepayment](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).



**Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

## Record ID

The record ID for a vendor prepayment REST record is **vendorprepayment**.

## Prerequisites

To use the vendor prepayment REST record, you must have the Vendor Prepayments feature enabled, and a default vendor prepayment account set. The account can be a new or existing account that is an Other Current Asset type. This is the account that NetSuite selects by default when entering a new vendor prepayment transaction.

## Code Sample

The following code sample shows how to create a vendor prepayment record:

```
1 POST {{REST_SERVICES}}/record/v1/vendorPrepayment Body:
2 { "entity": { "id": {{entityId}} }, "account": { "id": {{accountId}} }, "payment": amount
3 }
```

## Vendor Prepayment Application

After you make a prepayment to a vendor, you must apply the prepayment against the vendor bills. A vendor prepayment application is a posting transaction that impacts the general ledger. The Prepayment account is credited, offsetting the Accounts Payable account. The application decreases the total amount you must pay to your vendor.

For information about applying vendor prepayments in the UI, see the help topic [Vendor Prepayment Application](#).

NetSuite exposes the vendor prepayment application record to REST web services. This record is not a subrecord and does not contain subrecords.

The REST API Browser includes information about the field names and field types of the vendor prepayment application record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [vendorPrepaymentApplication](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for a vendor prepayment application REST record is **vendorprepaymentapplication**.

## Prerequisite

To use the vendor prepayment application REST record, you must have the Vendor Prepayments feature enabled, and a default vendor prepayment account set. This account is what NetSuite selects by default when entering a new vendor prepayment transaction.

## Exception

On the vendor prepayment application record, the Bills subtab is not accessible through REST web services.

## Code Sample

The following example shows how to create a vendor prepayment application by transforming a vendor prepayment into a vendor prepayment application. This transformation process applies the vendor prepayment.

```
1 POST {{REST_SERVICES}}/record/v1/vendorPrepayment/{{vendorPrepaymentId}}/!transform/vendorPrepaymentApplication Body:
2 { "bill": { "bills": [{ "line": {{lineId}}, "apply": true }} }
3 }
```

## Vendor Return Authorization

When you track vendor returns using authorizations, you know the status of each return and you can track accurate information about each return you process. The vendor return workflow includes obtaining Return Materials Authorization (RMA) from the vendor, then processing the return in NetSuite to track information about each return.

For information about working with vendor return authorizations in the UI, see the help topic [Vendor Return Authorization Overview](#).

NetSuite exposes the vendor return authorization record to REST web services. This record enables you to manage a specific vendor return authorization record.

The REST API Browser includes information about the field names and field types of the vendor return authorization record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [vendor return authorization](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

The vendor return authorization record contains the following subrecords:

- itemSublist
- expenseSublist
- billingAddress

The vendor return authorization record contains fields that are related to taxation features.

REST web services do not support legacy tax features. To work with taxation through REST web services you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

## Record ID

The record ID for a vendor return authorization REST record is **vendorReturnAuthorization**. To use the vendor return authorization REST record, you must have the Vendor Return Authorization feature enabled.

## Code Samples

The following samples show some common actions for a vendor return authorization.

### Creating a Vendor Return Authorization

```
1 { "entity": { "id": "36", "refName": "EU Vendor" }, "item": { "items": [{ "item": { "id": "89" }, "quantity": 5 } ] }
2 }
```

### Updating a Vendor Return Authorization

```
1 { "entity": { "id": "38" }, "currency": { "id": 1 }
2 }
```

## Vendor Subsidiary Relationship

NetSuite exposes the vendor subsidiary relationship record to REST web services. This record enables you to manage a specific vendorSubsidiaryRelationship record. You can also get a list of vendor subsidiary relationship records. This record is not accessible through the user interface.

The REST API Browser includes information about the field names and field types of the vendor subsidiary relationship record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [vendor subsidiary relationship](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record ID

The record ID for the vendor subsidiary relationship REST record is **vendorSubsidiaryRelationship**.

## Prerequisites

You must either use NetSuite OneWorld or have the Subsidiaries hidden feature enabled before you can use this record through REST web services.

## Updatable Fields on the vendorSubsidiaryRelationship Record

You can update the following fields on the vendorSubsidiaryRelationship record:

- credit limit (available on the vendor record, Subsidiary subtab)
- tax item (available on the vendor record, Subsidiary subtab)
- custom fields (available on the vendor record, Subsidiary subtab)
- (externalid) - external id

## Supported Operations

The following operations are supported for REST web services:

- GET (Read, Search for a list of vendor subsidiary relationship records)
- POST (insert a record)
- DELETE (a specific record)
- GET (Read, Search a specific record)
- PATCH (update a specific record)
- PUT (insert or update a specific record)

Tax-related fields are not supported in REST.

## Code Samples

The following code sample shows how to get a specific vendorSubsidiaryRelationship record:

```
1 | GET https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/vendorsubsidiaryrelationship?q=entity
2 | EQUAL 13 AND subsidiary EQUAL 5
```

The following code sample shows how to get the ID of a specific vendorSubsidiaryRelationship record:

```
1 | POST https://123456.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql?limit=5
```

```
1 | Header:
2 | 'prefer: transient'
3 | 'Content-Type: application/json'
4 | Body:
5 | '{ "q": "SELECT id FROM vendorSubsidiaryRelationship WHERE entity = 13 AND subsidiary = 5"
6 | }'
```

The following code sample shows how to set the externalId and creditLimit for a specific vendorSubsidiaryRelationship record:

```
1 | PATCH 'https://123456.suitetalk.api.netsuite.com/services/rest/record/v1/vendorsubsidaryrelationship/1'
```

```
1 | Header:
2 | 'Content-Type: application/json'
3 | Body:
4 | '{ "externalId": "VSR1", "creditLimit": 150
5 | }'
```

## Website

A website record exposes a website to REST web services.

This record has the following subrecords:

- imagesize
- siteentrypoint
- websitefieldset

 **Note:** REST web services do not support legacy tax features. To work with taxation through REST web services, you must have the SuiteTax feature enabled. For more information about using SuiteTax, see the help topic [SuiteTax](#).

The REST API Browser includes information about the field names and field types of the website record. It also includes the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [website](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for the website REST record is **website**.

## Prerequisites

You must enable the Website feature to be able to use the website record.

## Limitations

The following sublists are not accessible through REST web services:

- Storetab
- Sitecurrency
- Sitelanguage
- Cartcolumn
- Imagisize

Additionally, a number of fields are also not accessible. For the list of accessible fields, see the REST API browser.

The website record does not work with external ID.

## Code Samples

The following code sample shows how to load website data from a website with the ID 1.

```
1 | GET {{COMPANY_URL}}/services/rest/record/v1/website/1
```

The following code sample shows how to find a website by name.

```
1 | GET {{COMPANY_URL}}/services/rest/record/v1/website?q=displayName START_WITH "My website"
```

## Weekly Timesheet

With weekly time tracking, you can track the hours you and other employees work for a week at a time.

Weekly timesheets work in conjunction with the existing Time Tracking feature to help you customize how you capture time entries in a weekly format.

For help working with this record in the UI, see the help topics [Managing Time Tracking](#) and [Weekly Time Tracking](#).

The timesheet record type includes the following actions from the Actions Framework:

- Approve
- Reject
- Submit
- Retract

The REST API Browser includes information about the field names and field types of the timesheet record and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [time sheet](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a timesheet REST record is **timesheet**.

## Prerequisites

This record is available only when the Time Tracking and Weekly Timesheets features are enabled. For more information about how to enable these features, see the help topics [Managing Time Tracking](#) and [Weekly Time Tracking](#).

## Limitations

The following fields on this record are read-only:

- total hours
- hours total
- approval status

- submitted hours
- rejected hours
- allocated hours
- work calendar hours
- lock status

This record does not support the Update operation through REST.

## Code Samples

The following examples show how to create a new timesheet and approve an existing timesheet.

### Create a new timesheet:

```
1 | POST /services/rest/record/v1/timesheet HTTP/1.1
2 | Host: yourServer
3 | Content-Type: application/json
4 | Authorization: yourAuthorizationToken
5 | Content-Length: 46
6 | {
7 |   "employee": "Your_ID", "tranDate" : "YYYY-MM-DD"
8 | }
```

### Approve an existing timesheet

```
1 | POST /services/rest/record/v1/timesheet/346/@approve HTTP/1.1
2 | Host: burespctr-runbox-ml-deuf1-001.eng.netsuite.com
3 | Content-Type: application/json
4 | Authorization: yourAuthorizationToken
```

## Work Order

A work order record exposes a work order to REST web services. This record is not a subrecord.

This record:

- is not a subrecord
- has one subrecord: item

The REST API Browser includes information about the field names and field types of the work order record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [work order](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a work order REST record is **workOrder**.

## Prerequisites

You must enable the Work Orders feature.

## Code Samples

The following samples show common use cases for work orders. The example ID is 4.

### Creating a Work Order Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrder
2 | { "assemblyItem": {"id": "66"}, "subsidiary": {"id": "5"}, "location": {"id": "5"}, "quantity": 5, "isWip": true
3 | }
```

### Converting Work Orders Using PATCH Requests

The following code sample shows how to convert a work order to a work order issue.

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/record/v1/workOrder/4!transform/workOrderIssue
```

The following code sample shows how to convert a work order to a work order completion.

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/record/v1/workOrder/4!transform/workOrderCompletion
2 | { "startOperation": {"refName": "10"}, "endOperation": {"refName": "20"}
3 | }
```

The following code sample shows how to convert a work order to a work order completion with backflush.

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/record/v1/workOrder/4!transform/workOrderCompletion
2 | { "isBackflush": true, "startOperation": {"refName": "10"}, "endOperation": {"refName": "40"}, "component": { "items":
3 |   [{ "item": "59", "quantity": 1 }] } }
```

The following code sample shows how to convert a work order to a work order close.

```
1 | https://demo123.suitetalk.api.snap.netsuite.com/services/record/v1/workOrder/4!transform/workOrderClose
```

The following code sample shows how to create an assembly build from a non-WIP work order.

```
1 | https://demo123.suitetalk.api.snap.netsuite.com/services/record/v1/workOrder/4!transform/assemblyBuild
2 | { "location": {"id": 5}
3 | }
```

### Creating a Work Order With Items Using a POST Request

```
1 | POST https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrder
2 | { "subsidiary": { "id": "5" }, "assemblyItem": { "id": "67" }, "quantity": 1.0, "item": { "items": [{ "item": { "id": 63 } },
3 |   { "item": { "id": 65 } }] } }
```

### Retrieving a Work Order Using a GET Request

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrder/4
```

## Updating Work Orders Using a PATCH Request

The following code sample shows how to add an item to the item members of a work order.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrder/4
2 | { "item": { "items": [ { "item": { "id": 63 }, "quantity": 6 } ] } }
3 | }
```

The following code sample shows how to update a member item of a work order. The item ID is 7.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrder/4/item/7
2 | { "quantity": 55
3 | }
```

## Work Order Close

A work order close record exposes a work order close to REST web services.

This record:

- is not a subrecord
- has the following subrecords:
  - component
  - routingItem

The REST API Browser includes information about the field names and field types of the work order close record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [workOrderClose](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a work order close REST record is **workOrderClose**.

## Prerequisites

You must enable the Manufacturing Work In Process feature before you can use this record through REST web services.

## Code Samples

The following samples show common use cases for work order closes. The example ID is 4.

## Retrieving a Work Order Close Using a GET Request

A work order close is created by doing a transformation on a work order.

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrderClose/4
```

# Work Order Completion

A work order completion record exposes a work order completion to REST web services.

This record:

- is not a subrecord
- has the following subrecords:
  - component
  - operation

The REST API Browser includes information about the field names and field types of the work order completion record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [work order completion](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a work order completion REST record is **workOrderCompletion**.

## Prerequisites

You must enable the Manufacturing Work In Process feature before you can use this record through REST web services.

## Code Samples

The following samples show common use cases for work order completion. The example ID is 4.

### Retrieving a Work Order Completion Using a GET Request

A work order completion is created by doing a transformation on a work order.

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrderCompletion/4
```

### Updating a Work Order Completion Using a PATCH Request

The following code sample shows how to edit the setup time in the operation sequence. The ID for the operation is 7.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrderCompletion/4/operation/7
2 | { "laborSetupTime": 20.0
3 | }
```

The following code sample shows how to edit the component with backflush. The component ID is 7.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrderCompletion/4/component/lineNumber=7
2 | { "quantity": 3
```

3 | }

## Work Order Issue

A work order issue record exposes a work order issue to REST web services.

This record:

- is not a subrecord
- has one subrecord: component

The REST API Browser includes information about the field names and field types of the work order issue record, and about the HTTP methods, request parameters, and operations available to this record. For details, see the REST API Browser's [work order issue](#) reference page.

For information about using the REST API Browser, see the help topic [The REST API Browser](#).

## Record IDs

The record ID for a work order issue REST record is **workOrderIssue**.

## Prerequisites

You must enable the Manufacturing Work In Process feature before you can use this record through REST web services.

## Code Samples

The following samples show common use cases for work order issues. The example ID is 4.

### Retrieving a Work Order Issue Using a GET Request

A work order issue is created by doing a transformation on a work order.

```
1 | GET https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrderIssue/4
```

### Updating a Work Order Issue Using a PATCH Request

The following code sample shows how to edit the quantity of the component member item. The ID for the item is 7.

```
1 | PATCH https://demo123.suitetalk.api.snap.netsuite.com/services/rest/record/v1/workOrderIssue/4/component/item=7
2 | { "quantity": 5
3 | }
```

# REST Web Services Tutorials

The following sections contain end-to-end guidelines for specific business scenarios.

- [Sales Order Use Cases](#)
- [SuiteBilling Use Cases](#)
- [Inventory Item Use Cases](#)
- [Customer Use Cases](#)
- [Employee Use Cases](#)
- [Invoice Use Cases](#)
- [Journal Entry Use Cases](#)
- [Event Use Cases](#)
- [Credit Memo Use Cases](#)
- [Vendor Use Cases](#)
- [Vendor Bill Use Cases](#)
- [Cash Sale Use Cases](#)

## Sales Order Use Cases

This section includes the following sample use cases for managing sales order transactions:

- [Use Case For Creating Your Sales Order](#)
- [Use Case For Applying a Promotion to Your Sales Order](#)
- [Use Case For Retrieving Your Sales Order](#)
- [Use Case For Updating Your Sales Order](#)
- [Use Case For Approving Your Sales Order](#)
- [Use Case For Fulfilling Your Sales Order](#)
- [Use Case For Creating Invoices or Cash Sales from Your Sales Order](#)
- [Use Case For Creating a Progress Sales Order](#)
- [Use Case For Deleting a Sales Order](#)

A sales order is a transaction that records a commitment to sell items or services to a customer. Sales orders have no accounting impact until items ship or services are completed. For more information on NetSuite sales orders, see the help topic [Creating Sales Orders](#).

## About This Tutorial

The following `{{ VARIABLE }}` expressions are variables that you need to customize when reusing the code samples.

| Variable                       | Definition                                                                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>{{REST_SERVICES}}</code> | Indicates your access to the NetSuite REST Service (for example, <code>https://testaccount.corp.netsuite.com/services/rest</code> ). |

| Variable | Definition                                                                                          |
|----------|-----------------------------------------------------------------------------------------------------|
| {{*_ID}} | These variables refer to the internal ID of the record. For example, ESTIMATE_ID or SALES_ORDER_ID. |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case For Creating Your Sales Order

There are two ways you can create a sales order:

- [Create a Sales Order from an Estimate](#)
- [Create a New Stand-Alone Sales Order](#)

### Create a Sales Order from an Estimate

To create a sales order from an existing estimate, use the transform function in the REST API:

```
1 | POST {{REST_SERVICES}}/record/v1/estimate/{{ESTIMATE_ID}}/!transform/salesOrder
```

This function takes an estimate with an ID of {{ESTIMATE\_ID}} and transforms it into a sales order. You can also specify in the body of the request which fields you want to change during the transformation.

```
1 | POST {{REST_SERVICES}}/record/v1/estimate/{{ESTIMATE_ID}}/!transform/salesOrder
2 |
3 | {
4 |   "memo": "Transformed Estimate to Sales Order!"
5 | }
```

### Create a New Stand-Alone Sales Order

In the Basic Sales Order example, you can create a sales order without filling in the details of the other fields.

The system automatically calculates many fields in the sales order based on the value of the entity field.

The transaction Date (or tranDate in REST) is the creation date of the sales order. If you leave the field blank, it defaults to the current date.

For a complete list of fields you can populate on a sales order, see the [Sales Order Records Browser](#).

Use the Custom Form field (customform in REST) to determine what type of record the system creates after you bill a sales order. For more information about the custom form field, see the help topic [Creating Sales Orders](#).

Custom Form Types include:

- Standard Sales Order
- Standard Sales Order - Cash Sale

- Standard Sales Order - Invoice
- Standard Sales Order - Progress Billing

```
1 | POST {{REST_SERVICES}}/record/v1/salesOrder
```

### Basic Sales Order

```
1 | {
2 |   "entity": {
3 |     "id": "110"
4 |   },
5 |   "item": {
6 |     "items": [
7 |       {
8 |         "item": {
9 |           "id": 98
10 |         },
11 |         "quantity": 1
12 |       }
13 |     ]
14 |   }
15 | }
```

### More Complicated Sales Order

```
1 | {
2 |   "entity": {
3 |     "id": "110"
4 |   },
5 |   "customForm": "68",
6 |   "tranDate": "2020-6-15",
7 |   "otherrefnum": "11037",
8 |   "memo": "This is a test memo!",
9 |   "item": {
10 |     "items": [
11 |       {
12 |         "item": {
13 |           "id": 98
14 |         },
15 |         "quantity": 1,
16 |         "Description": "Test Item"
17 |       }
18 |     ]
19 |   }
20 | }
```

## Use Case For Applying a Promotion to Your Sales Order

To apply a promotion to a sales order, provide the ID value to the `promocode` field.

You can do this when you create a new sales order or when you make an update to an existing sales order.

### Applying a Promotion to a Sales Order

```
1 | POST {{REST_SERVICES}}/record/v1/salesOrder
2 |
3 | {
4 |   "entity": {
5 |     "id": "110"
```

```

6   },
7   "promocode": { "id": 1 },
8   "item": {
9     "items": [
10      {
11        "item": {
12          "id": 98
13        },
14        "quantity": 1
15      }
16    ]
17  }
18 }

```

## Apply a Discount Item to a Sales Order

If you have a discount item you want to apply to a sales order, you can either add in the discount item when you create a new sales order or when you make an update to an existing sales order. For the `discountitem` `id`, you specify the internal ID of the discount item.

### Applying a Discount Item When Creating a New Sales Order

```

1  POST {{REST_SERVICES}}/record/v1/salesOrder
2
3  {
4    "entity": {
5      "id": "110"
6    },
7    "discountitem": { "id": 99 };
8    "item": {
9      "items": [
10       {
11         "item": {
12           "id": 98
13         },
14         "quantity": 1
15       }
16     ]
17   }
18 }

```

### Applying a Discount Item When Updating an Existing Sales Order

```

1  PATCH {{REST_SERVICES}}/record/v1/salesOrder/{{SALES_ORDER_ID}}
2
3  {
4    "discountitem": { "id": 99 }
5  }

```

## Use Case For Retrieving Your Sales Order

Use a GET request with the Sales Order's ID number to retrieve the sales order data.

```

1  GET {{REST_SERVICES}}/record/v1/salesOrder/{{ID}}

```

In the response body, you can see all the data pertaining to the sales order. The data includes the specific field values that you set, as well as default values for the fields that you did not set.

Each of the sublists has its own endpoint for you to gather field data for each line.

## GET Request Results

```

1  {
2    "links": [
3      {
4        "rel": "self",
5        "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/salesOrder/18"
6      }
7    ],
8    "altSalesTotal": 0.0,
9    "balance": 0.0,
10   "balreadyrefunded": "F",
11   "billingaddress": {
12     "links": [
13       {
14         "rel": "self",
15         "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/salesorder/18/billingaddress"
16       }
17     ]
18   },
19   "canBeUnapproved": true,
20   "carrier": "nonups",
21   "ccApproved": false,
22   "ccProcessAsPurchaseCard": false,
23   "checkCommitted": false,
24   "createdDate": "2020-07-29T08:34:00Z",
25   "credholdentity": 110,
26   "credholdoverride": "F",
27   "currency": {
28     "links": [
29       {
30         "rel": "self",
31         "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/currency/1"
32       }
33     ],
34     "id": "1",
35     "refName": "USA"
36   },
37   "currencyName": "USA",
38   "currencyPrecision": 2,
39   "currencysymbol": "USD",
40   "customForm": "68",
41   "defaultCatchUp": 92,
42   "deferredrevenue": 0.0,
43   "discountTotal": 0.0,
44   "edition": "US",
45   "entity": {
46     "links": [
47       {
48         "rel": "self",
49         "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/customer/110"
50       }
51     ],
52     "id": "110",
53     "refName": "Anonymous Customer"
54   },
55   "entity_nexus_country": "US",
56   "entityfieldname": "entity",
57   "entityNexus": {
58     "links": [
59       {
60         "rel": "self",
61         "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/nexus/1"
62       }
63     ],
64     "id": "1",
65     "refName": "US-CA"
66   },
67   "exchangeRate": 1.0,
68   "excludecommission": false,
69   "getAuth": false,
70   "giftCertApplied": 0.0,
71   "giftCertRedemption": {

```

```

72     "links": [
73         {
74             "rel": "self",
75             "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/salesorder/18/giftCertRedemption"
76         }
77     ],
78     "hasanydelayedrevrec": false,
79     "hasFedexFreightService": false,
80     "hasprecalcs": false,
81     "id": "18",
82     "ignoreAvs": false,
83     "ignoreAvsVis": false,
84     "ignoreCsc": false,
85     "ignoreCscVis": false,
86     "installmentcount": 0,
87     "invertrevrecschedule": false,
88     "iseitf81on": false,
89     "isOnlineTransaction": "F",
90     "isRecurringPayment": false,
91     "item": {
92         "links": [
93             {
94                 "rel": "self",
95                 "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/salesorder/18/item"
96             }
97         ]
98     },
99     "itemShippingCostFxRate": 1,
100    "lastModifiedDate": "2020-07-29T08:34:00Z",
101    "linkedTrackingNumberList": {
102        "links": [
103            {
104                "rel": "self",
105                "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/salesorder/18/linkedTrackingNumberList"
106            }
107        ]
108    },
109    "manualCreditHold": "F",
110    "needsbill": "F",
111    "needsPick": true,
112    "nextbill": "2020-07-29",
113    "nexus": {
114        "links": [
115            {
116                "rel": "self",
117                "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/nexus/1"
118            }
119        ]
120    },
121    "id": "1",
122    "refName": "US-CA"
123 },
124 "nexus_country": "US",
125 "oldrevenuecommitment": false,
126 "ordbilled": "F",
127 "orderStatus": "A",
128 "ordpicked": false,
129 "ordrecvd": false,
130 "originalNexus": {
131     "links": [
132         {
133             "rel": "self",
134             "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/nexus/1"
135         }
136     ],
137     "id": "1",
138     "refName": "US-CA"
139 },
140 "originalNexusCountry": "US",
141 "overallbalance": 0.0,
142 "overallunbilledorders": 39.95,
143 "paymentSessionAmount": 39.95,
144 "paypalOverride": false,

```

```

145     "paypalProcess": false,
146     "prevDate": "2020-07-29",
147     "primarycurrency": 1.0,
148     "primarycurrencyfxrate": 1.0,
149     "recognizedrevenue": 0.0,
150     "revenuestatus": "A",
151     "saleeffectivedate": "2020-07-29",
152     "salesTeam": {
153       "links": [
154         {
155           "rel": "self",
156           "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/salesorder/18/salesTeam"
157         }
158       ]
159     },
160     "shipComplete": false,
161     "shipDate": "2020-07-29",
162     "shipIsResidential": false,
163     "shipItemHasFreeShippingItems": false,
164     "shipOverride": false,
165     "shippingAddress": {
166       "links": [
167         {
168           "rel": "self",
169           "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/salesorder/18/shippingAddress"
170         }
171       ]
172     },
173     "shippingCostOverridden": false,
174     "status": "A",
175     "storeorder": "F",
176     "subsidiary": {
177       "links": [
178         {
179           "rel": "self",
180           "href": "https://runbox3.corp.netsuite.com/services/rest/record/v1/subsidiary/1"
181         }
182       ]
183     },
184     "id": "1",
185     "refName": "Parent Company"
186   },
187   "subtotal": 39.95,
188   "taxTotal": 0.0,
189   "threedStatusCodeVis": false,
190   "toBeEmailed": false,
191   "toBeFaxed": false,
192   "toBePrinted": false,
193   "total": 39.95,
194   "tranDate": "2020-07-29",
195   "tranId": "1",
196   "transvsoebundle": false,
197   "unbilledOrders": 39.95,
198   "usingVSOEFldSet": false,
199   "vsoeautocalc": false,
200   "warnNexusChange": false,
201   "webstore": "F",
202   "weekendpreference": "ASIS"
203 }

```

## Use Case For Updating Your Sales Order

Some common updates to an existing sales order include the following options:

- [Update Field Values](#)
- [Update a Field for a Specific Line](#)
- [Add a New Transaction Line Without Updating a Field Value](#)
- [Remove an Item](#)

## Update Field Values

To update a specific field on the sales order, send a PATCH request with the changed field in the body.

```
1 | PATCH {{REST_SERVICES}}/record/v1/salesOrder/{{SALES_ORDER_ID}}
2 |
3 | {
   |   "orderStatus": "B" }
```

## Update a Field for a Specific Line

Updating a transaction line item is similar to updating a field by sending a PATCH request with the changed value in the body.

```
1 | PATCH {{REST_SERVICES}}/record/v1/salesOrder/{{SALES_ORDER_ID}}
```

To update a field for a specific line, specify the line number in the body of your request.

### Updating an Existing Item

```
1 | {
2 |   "item": {
3 |     "items": [
4 |       {
5 |         "line": 1,
6 |         "isclosed": true
7 |       }
8 |     ]
9 |   }
10 | }
```

## Add a New Transaction Line Without Updating a Field Value

To add a new transaction line without updating a field value, leave out the line number. The request creates a new transaction line for the sales order.

```
1 | PATCH {{REST_SERVICES}}/record/v1/salesOrder/{{SALES_ORDER_ID}}
```

### Creating a New Line Item

```
1 | {
2 |   "item": {
3 |     "items": [
4 |       {
5 |         "item": { "id": 98 },
6 |         "quantity": 2
7 |       }
8 |     ]
9 |   }
10 | }
```

## Remove an Item

After saving a sales order, you can use the sales order's ID to remove an item by using the following PATCH call. Using an existing sales order, update {{salesOrderId}} with the ID of the sales order you wish to modify and specify the new items. When specifying the new items in the sales order, make sure the item you wish to remove is no longer listed.

```
1 | PATCH {{REST_SERVICES}}/record/v1/salesOrder/{{salesOrderId}}?replace=item
```

**Note:** Currently, there is no option to individually delete a specific line through REST, so you can make the lines optional and let the sales person enforce this rule.

### Removing a Line Item (Future)

```
1 | {
2 |   "item": {
3 |     "items": [{
4 |       "item": { "id": {{itemId}} },
5 |       "rate": 40
6 |     }, {
7 |       "item": { "id": {{itemId}} },
8 |       "rate": 50
9 |     }]
10 |   }
11 | }
```

## Use Case For Approving Your Sales Order

After you create a sales order, the `orderStatus` field of the sales order is initially set to value "A", and later changes to value "B". These letters denote the state of the sales order in the approval workflow as follows:

| orderStatus | Approval Status     |
|-------------|---------------------|
| A           | Pending Approval    |
| B           | Pending Fulfillment |

To create a preapproved sales order, set the `orderStatus` in the POST request to "B".

### Setting Sales Order Approval Status

```
1 | POST {{REST_SERVICES}}/record/v1/salesOrder
2 |
3 | {
4 |   "entity": {
5 |     "id": "110"
6 |   },
7 |   "orderStatus": "B",
8 |   "item": {
9 |     "items": [
10 |       {
11 |         "item": {
12 |           "id": 98
13 |         },
14 |         "quantity": 1
15 |       }
16 |     ]
17 |   }
18 | }
```

If you want to create a sales order that requires approval after you create it, send in a PATCH request with `orderStatus` "B" in the body.

```
1 | PATCH {{REST_SERVICES}}/record/v1/salesOrder/{{SALES_ORDER_ID}}
2 |
3 | {
4 |   "orderStatus": "B"
5 | }
```

## Reset Approval Status

If you already approved the sales order but need to reset the approval status, send a PATCH request with the updated `orderStatus` in the body.

```
1 PATCH {{REST_SERVICES}}/record/v1/salesOrder/{{SALES_ORDER_ID}}
2
3 {
4   "orderStatus": "A"
5 }
```

## Close a Sales Order

To close the sales order, close individual line items on the sales order using the following PATCH call and body.

### Patch Call for Closing a Sales Order

```
1 PATCH {{REST_SERVICES}}/record/v1/salesOrder/{{SALES_ORDER_ID}}
2
3 {
4   "item": {
5     "items": [
6       {
7         "line": 1,
8         "isclosed": true
9       },
10      {
11        "line": 2,
12        "isclosed": true
13      },
14      {
15        "line": 3,
16        "isclosed": true
17      }
18    ]
19  }
20 }
```

## Use Case For Fulfilling Your Sales Order

To start the sales order fulfillment process, create a fulfillment from your sales order. You can create a fulfillment by sending a POST request with the ID of the sales order you want to fulfill.

```
1 POST {{REST_SERVICE}}/record/v1/salesorder/{{SALES_ORDER_ID}}/!transform/itemfulfillment
```

The following code sample contains two `orderLine` groupings:

- In the first grouping, you partially fulfill an order by specifying a quantity of less than the full order (**"quantity": 1**).
- In the second grouping, you leave the order unfulfilled by setting the `itemreceive` parameter to false (**"itemreceive": false**).

```
1 {
2   "item": {
3     "items": [
4       {
5         "orderLine": 1,
6         "location": 6,
7         "quantity": 1
```

```

8      },
9      {
10         "orderLine": 4,
11
12         "location": 6,
13         "itemreceive": false
14     }
15 ]
16 }
17 }

```



**Note:** Like the provided code sample, your sales order may not have sequential order lines. For example, "orderLine": 4 follows "orderLine": 1 in the code sample.

To check the correct orderLine number for an item, use a GET call to retrieve the sales order item information through REST as show here:

```
1 | GET {{REST_SERVICES}}/record/v1/salesOrder/{{ID}}/item
```

## Use Case For Creating Invoices or Cash Sales from Your Sales Order

This use case shows you how to create an invoice or cash sale for all fulfilled lines on the sales order.

### Create an Invoice

To create an invoice from a sales order, you need an approved sales order with the Custom Form field set to Standard sales order - Invoice and some fulfilled items.

```
1 | POST {{REST_SERVICES}}/record/v1/salesorder/{{SALES_ORDER_ID}}/!transform/invoice
```

If you leave the request body empty, NetSuite creates an Invoice for all fulfilled items and quantities. However, if you plan to fulfill only part of the order, you need to be able to create a matching partial invoice. As shown in the example, you can create a partial invoice by specifying the quantity for each item line.

#### Invoice From a Sales Order

```

1  {
2      "item": {
3          "items": [
4              {
5                  "orderLine": 1,
6                  "quantity": 3
7              },
8              {
9                  "orderLine": 2,
10                 "quantity": 0
11             }
12         ]
13     }
14 }

```

### Create a Cash Sale

To create a cash sale from a sales order, you need an approved sales order with the Custom Form field set to Standard Sales Order - Cash Sale and some fulfilled items.

```
1 | POST {{REST_SERVICES}}/record/v1/salesorder/{{SALES_ORDER_ID}}/!transform/cashsale
```

If you leave the request body empty, NetSuite creates a cash sale for all fulfilled items and quantities. You can create a partial cash sale by specifying the quantity for each item line as shown in the following example.

### Cash Sale From a Sales Order

```
1 | {
2 |   "item": {
3 |     "items": [
4 |       {
5 |         "orderLine": 1,
6 |         "quantity": 0
7 |       },
8 |       {
9 |         "orderLine": 2,
10 |        "quantity": 4
11 |       }
12 |     ]
13 |   }
14 | }
```

## Use Case For Creating a Progress Sales Order

Creating a progress sales order is similar to creating a new sales order. For a progress sales order, change the custom form to Standard Sales Order - Progress Billing. You do this through REST by setting the custom form value when sending your REST request.

| Custom Form Name                        | Custom Form ID |
|-----------------------------------------|----------------|
| Standard Sales Order (default)          | 68             |
| Standard Sales Order - Progress Billing | 86             |

To complete the sales order, you need to add:

- a value for the entity field that corresponds to the customer on the sales order
- at least one item

You can also modify the following optional fields:

| Field on the Sales Order | Corresponding Field on the Custom Form | Valid Values                                    |
|--------------------------|----------------------------------------|-------------------------------------------------|
| date                     | trandate                               | YYYY-MM-DD                                      |
| status                   | orderstatus                            | A - Pending Approval<br>B - Pending Fulfillment |
| billing terms            | terms                                  | see the terms table in the database             |

For more information, see the help topic [Creating Progress Sales Orders](#).

### Progress Sales Order

```
1 | POST {{REST_SERVICES}}/record/v1/salesorder
2 |
3 | {
4 |   "customform": 86, //For custom form "Standard Sales Order - Progress Billing"
```

```

5      "entity": {
6        "id": "110"
7      },
8      "trandate": "2020-07-30", //optional field
9      "orderstatus": "B", //optional field
10     "terms": 3, //optional field
11     "item": {
12       "items": [
13         {
14           "item": {
15             "id": 98
16           },
17           "quantity": 1
18         },
19         {
20           "item": {
21             "id": 98
22           },
23           "quantity": 2
24         },
25         {
26           "item": {
27             "id": 98
28           },
29           "quantity": 3
30         }
31       ]
32     }
33   }

```

## Use Case For Deleting a Sales Order

To delete a sales order, send the DELETE call to delete the specific record with the specified ID.

```
1 | DELETE {{REST_SERVICES}}/record/v1/salesOrder/{{ID}}
```

## SuiteBilling Use Cases

This section includes the following sample use cases for managing subscriptions using REST web services:

- [Use Case For Managing Your Subscription Catalog](#)
- [Use Case For Managing Your Subscription Sales](#)

## Use Case For Managing Your Subscription Catalog

REST services provide a convenient channel to manage your subscription catalog.

In this use case, you create the necessary records using REST for your fictitious company, CED Cyber Solutions.

This use case demonstrates how to:

1. [Create Service Items](#)
2. [Create a Subscription Plan](#)
3. [Create a Price Plan](#)
4. [Create a Price Book](#)
5. [Retrieve Records](#)

## Create Service Items

Before you begin, you need to create service items for your subscription plan. The security subscription plan that you want to sell includes the following items:

- Setup
- License
- Seats
- Bronze Support
- Silver Support
- Gold Support

You need to create a subscription term because you want your subscription plan to have an initial term of three years.

Although the subscription plan depends on these records, this use case does not cover how to create them through REST. For now, you can create them through the user interface.

After you create each record, make note of its internal ID so that you can reference it. You can find the internal ID of each record by navigating to the record's list page.

The following table lists prerequisite records along with the record type and example internal ID for each:

| Record Type                   | Record Name    | Example Internal ID |
|-------------------------------|----------------|---------------------|
| <a href="#">Service Items</a> | License        | 1                   |
| <a href="#">Service Items</a> | Setup          | 2                   |
| <a href="#">Service Items</a> | Seats          | 3                   |
| <a href="#">Service Items</a> | Bronze Support | 4                   |
| <a href="#">Service Items</a> | Silver Support | 5                   |
| <a href="#">Service Items</a> | Gold Support   | 6                   |
| Subscription Term             | Three Years    | 1                   |

## Create a Subscription Plan

To create records, you send POST requests to the endpoint for the record type. The request body should contain information for all required fields and any optional fields.

Your requirements are as follows:

- The plan name is "DEFEND Package"
- Setup is a one-time line
- Licensing and the number of users (seats) are recurring lines
- Customers must choose one of the three support tiers



**Note:** At present time, there is no way to enforce this behavior through NetSuite, so you can make the lines optional and let the sales person enforce this rule.

- All recurring lines should be prorated except for seats
- All lines are billed in advance

To set the `initialTerm` to the subscription term that you have created, you need to reference an existing record. Because you created a custom subscription term, wrap the ID of that term in an object with the `ID` property. Wrapping the ID in an object is commonly how records are referenced in REST.

Lines on subscription plans are specified in the member sublist. The member sublist is an object with an `items` property. The `items` property is an array of objects that each represent a line. The lines have their own fields that must be defined. Although the `lineNumber` can be inferred in some cases, it is best to always specify it. Lines are 1-indexed. This is the common structure for sublists in requests.

Some fields on the lines in the member sublist require enumeration values. Usually, the possible values are clearly defined in the Metadata Catalog. However, sometimes they can be unclear. For example, `subscriptionLineType` has an enumeration value of integer strings. Those strings map to subscription line types:

- **1** – One-Time
- **2** – Recurring
- **3** – Usage

A successful request returns a response with a HTTP status of **204 No Content**. In the response headers, there is a `Location` key with an associated value representing the endpoint at which you can access your newly created record. Make note of the ID so that you can reference your subscription plan later. The ID for your example subscription plan is 1.

### Example Subscription Plan

```

1 {
2   "itemId": "DEFEND Package",
3   "initialTerm": { "id": "1" },
4   "member": {
5     "items": [
6       {
7         "lineNumber": 1,
8         "item": { "id": "1" },
9         "isRequired": true,
10        "subscriptionLineType": "1",
11        "billingMode": "IN_ADVANCE",
12        "renewalOption": "DIFFERENT_PLAN"
13      },
14      {
15        "lineNumber": 2,
16        "item": { "id": "2" },
17        "isRequired": true,
18        "subscriptionLineType": "2",
19        "prorateStartDate": true,
20        "prorateEndDate": true,
21        "billingMode": "IN_ADVANCE",
22        "renewalOption": "ALWAYS"
23      },
24      {
25        "lineNumber": 3,
26        "item": { "id": "3" },
27        "isRequired": true,
28        "subscriptionLineType": "2",
29        "prorateStartDate": false,
30        "prorateEndDate": false,
31        "billingMode": "IN_ADVANCE",
32        "renewalOption": "ALWAYS"
33      },
34      {
35        "lineNumber": 4,
36        "item": { "id": "4" },
37        "isRequired": false,
38        "subscriptionLineType": "2",
39        "prorateStartDate": true,
40        "prorateEndDate": true,
41        "billingMode": "IN_ADVANCE",

```

```

42         "renewalOption": "ALWAYS"
43     },
44     {
45         "lineNumber": 5,
46         "item": { "id": "5" },
47         "isRequired": false,
48         "subscriptionLineType": "2",
49         "prorateStartDate": true,
50         "prorateEndDate": true,
51         "billingMode": "IN_ADVANCE",
52         "renewalOption": "ALWAYS"
53     },
54     {
55         "lineNumber": 6,
56         "item": { "id": "6" },
57         "isRequired": false,
58         "subscriptionLineType": "2",
59         "prorateStartDate": true,
60         "prorateEndDate": true,
61         "billingMode": "IN_ADVANCE",
62         "renewalOption": "ALWAYS"
63     }
64 ]
65 }
66 }

```

## Create a Price Plan

Your pricing requirements are as follows:

- Setup costs \$100.00
- A license costs \$1000.00
- Seats have a volume-based cost that decreases as your customer adds more users
  - For 0 - 20 seats, each seat costs \$7.00
  - For 21- 50 seats, each seat costs \$6.00
  - For 51+ seats, each seat costs \$5.00
- Support prices change over time
  - Bronze starts at \$30.00 and drops to \$20.00 after the first year
  - Silver starts at \$60.00 and drops to \$40.00 after the first year
  - Gold starts at \$90.00 and drops to \$60.00 after the first year

There are six lines on your subscription plan, which means that you need at least six price plans. You need three more price plans to account for the price drop after the first year on your support lines. To satisfy these requirements, you need nine price plans.

Each price plan requires its own request.

### Price Plan 1

```

1  {
2      "currency": { "id": "1" },
3      "pricePlanType": "2",
4      "priceTiers": {
5          "items": [
6              {
7                  "fromVal": 0,
8                  "pricingOption": { "id": "-102" },
9                  "value": 100.00
10             }
11         ]
12     }

```

```
13 | }
```

## Price Plan 2

```
1 | {
2 |   "currency": { "id": "1" },
3 |   "pricePlanType": "2",
4 |   "priceTiers": {
5 |     "items": [
6 |       {
7 |         "fromVal": 0,
8 |         "pricingOption": { "id": "-102" },
9 |         "value": 250.00
10 |       }
11 |     ]
12 |   }
13 | }
```

## Price Plan 3

```
1 | {
2 |   "currency": { "id": "1" },
3 |   "pricePlanType": "4",
4 |   "priceTiers": {
5 |     "items": [
6 |       {
7 |         "fromVal": 0,
8 |         "pricingOption": { "id": "-101" },
9 |         "value": 7.00
10 |       },
11 |       {
12 |         "fromVal": 20,
13 |         "pricingOption": { "id": "-101" },
14 |         "value": 6.00
15 |       },
16 |       {
17 |         "fromVal": 50,
18 |         "pricingOption": { "id": "-101" },
19 |         "value": 5.00
20 |       }
21 |     ]
22 |   }
23 | }
```

## Price Plan 4

```
1 | {
2 |   "currency": { "id": "1" },
3 |   "pricePlanType": "2",
4 |   "priceTiers": {
5 |     "items": [
6 |       {
7 |         "fromVal": 0,
8 |         "pricingOption": { "id": "-102" },
9 |         "value": 30.00
10 |       }
11 |     ]
12 |   }
13 | }
```

## Price Plan 5

```
1 | {
2 |   "currency": { "id": "1" },
3 |   "pricePlanType": "2",
4 |   "priceTiers": {
5 |     "items": [
```

```

6      {
7          "fromVal": 0,
8          "pricingOption": { "id": "-102" },
9          "value": 20.00
10     }
11 ]
12 }
13 }

```

### Price Plan 6

```

1 {
2     "currency": { "id": 1 },
3     "pricePlanType": "2",
4     "priceTiers": {
5         "items": [
6             {
7                 "fromVal": 0,
8                 "pricingOption": { "id": "-102" },
9                 "value": 60.00
10            }
11        ]
12    }
13 }

```

### Price Plan 7

```

1 {
2     "currency": { "id": "1" },
3     "pricePlanType": "2",
4     "priceTiers": {
5         "items": [
6             {
7                 "fromVal": 0,
8                 "pricingOption": { "id": "-102" },
9                 "value": 40.00
10            }
11        ]
12    }
13 }

```

### Price Plan 8

```

1 {
2     "currency": { "id": "1" },
3     "pricePlanType": "2",
4     "priceTiers": {
5         "items": [
6             {
7                 "fromVal": 0,
8                 "pricingOption": { "id": "-102" },
9                 "value": 90.00
10            }
11        ]
12    }
13 }

```

### Price Plan 9

```

1 {
2     "currency": { "id": "1" },
3     "pricePlanType": "2",
4     "priceTiers": {
5         "items": [
6             {
7                 "fromVal": 0,
8                 "pricingOption": { "id": "-102" },
9                 "value": 60.00

```

```

10 |         }
11 |     ]
12 | }
13 | }

```

## Create a Price Book

Your requirements are as follows:

- Setup is a one-time cost
- License cost is billed annually and prorated by month
- Seats are billed monthly and prorated by day
- Support is billed monthly, prorated by day, and prices decrease after the first year

The subscriptionPlanLineNumber has an integer value and is the number of the line to which the price interval should apply. The sublist line number cannot be inferred by the system because the relationship between subscription plan lines and price intervals is one-to-many.

The repeatEvery is connected to the frequency and determines how often the line should be billed. It has an enum value instead of an integer. The values map directly to their integer values.

For one-time lines, you must specify that repeatEvery is 0 and prorateBy is an empty string.

The startOffsetUnit and startOffsetValue are related to time-based pricing. Combined, the two fields determine when the pricing of a certain line should follow the tiers in the referenced price plan. For example, the support lines (4, 5, and 6) have two price intervals each. The first interval defines the pricing for the first year as startOffsetUnit = YEAR and startOffsetValue = 1. The second interval defines the pricing for the second year as startOffsetUnit = YEAR and startOffsetValue = 2. This results in a reduced recurring price for support in every year after the first.

On a successful response, you have completed the process for creating a subscription plan and price plans linked by a single price book.

### Price Book

```

1 | {
2 |   "subscriptionPlan": { "id": "1" },
3 |   "currency": { "id": "1" },
4 |   "name": "TEST Pricing",
5 |   "priceInterval": {
6 |     "items": [
7 |       {
8 |         "subscriptionPlanLineNumber": 1,
9 |         "pricePlan": { "id": "10" },
10 |        "frequency": "ONETIME",
11 |        "repeatEvery": "0",
12 |        "startOffsetUnit": "MONTH",
13 |        "startOffsetValue": 1,
14 |        "prorateBy": ""
15 |      },
16 |      {
17 |        "subscriptionPlanLineNumber": 2,
18 |        "pricePlan": { "id": "11" },
19 |        "frequency": "ANNUALLY",
20 |        "repeatEvery": "1",
21 |        "startOffsetUnit": "MONTH",
22 |        "startOffsetValue": 1,
23 |        "prorateBy": "MONTH"
24 |      },
25 |      {
26 |        "subscriptionPlanLineNumber": 3,
27 |        "pricePlan": { "id": "12" },
28 |        "frequency": "MONTHLY",
29 |        "repeatEvery": "1",

```

```

30         "startOffsetUnit": "MONTH",
31         "startOffsetValue": 1,
32         "prorateBy": "DAY"
33     },
34     {
35         "subscriptionPlanLineNumber": 4,
36         "pricePlan": { "id": "13" },
37         "frequency": "MONTHLY",
38         "repeatEvery": "1",
39         "startOffsetUnit": "YEAR",
40         "startOffsetValue": 1,
41         "prorateBy": "DAY"
42     },
43     {
44         "subscriptionPlanLineNumber": 4,
45         "pricePlan": { "id": "14" },
46         "frequency": "MONTHLY",
47         "repeatEvery": "1",
48         "startOffsetUnit": "YEAR",
49         "startOffsetValue": 2,
50         "prorateBy": "DAY"
51     },
52     {
53         "subscriptionPlanLineNumber": 5,
54         "pricePlan": { "id": "15" },
55         "frequency": "MONTHLY",
56         "repeatEvery": "1",
57         "startOffsetUnit": "YEAR",
58         "startOffsetValue": 1,
59         "prorateBy": "DAY"
60     },
61     {
62         "subscriptionPlanLineNumber": 5,
63         "pricePlan": { "id": "16" },
64         "frequency": "MONTHLY",
65         "repeatEvery": "1",
66         "startOffsetUnit": "YEAR",
67         "startOffsetValue": 2,
68         "prorateBy": "DAY"
69     },
70     {
71         "subscriptionPlanLineNumber": 6,
72         "pricePlan": { "id": "17" },
73         "frequency": "MONTHLY",
74         "repeatEvery": "1",
75         "startOffsetUnit": "YEAR",
76         "startOffsetValue": 1,
77         "prorateBy": "DAY"
78     },
79     {
80         "subscriptionPlanLineNumber": 6,
81         "pricePlan": { "id": "18" },
82         "frequency": "MONTHLY",
83         "repeatEvery": "1",
84         "startOffsetUnit": "YEAR",
85         "startOffsetValue": 2,
86         "prorateBy": "DAY"
87     }
88 ]
89 }
90 }

```

## Retrieve Records

You can retrieve records using a GET request. You can also use a GET request for validating records after creation. The endpoint is similar to the POST requests. The only difference is that the ID of the record is appended to the end.

**Example:** In this example, the GET request returns the response body that follows.

**Note:** The response body is slightly different from the request body because there are other optional fields. These fields were set to their default values internally. The member sublist has its own endpoint that you can access to gather field data for each line.

This GET request:

```
1 | GET http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/subscriptionplan/1
```

Returns this response body:

```
1 | {
2 |   "links": [
3 |     {
4 |       "rel": "self",
5 |       "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/subscriptionplan/1"
6 |     }
7 |   ],
8 |   "autoRenewal": false,
9 |   "createdDate": "2020-1-1T00:00:00Z",
10 |  "customForm": "-950",
11 |  "defaultRenewalTerm": {
12 |    "links": [],
13 |    "id": "1",
14 |    "refName": "Three Years"
15 |  },
16 |  "id": "1",
17 |  "includeChildren": false,
18 |  "incomeAccount": {
19 |    "links": [
20 |      {
21 |        "rel": "self",
22 |        "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/account/1"
23 |      }
24 |    ],
25 |    "id": "1",
26 |    "refName": "Sales"
27 |  },
28 |  "initialTerm": {
29 |    "links": [],
30 |    "id": "1",
31 |    "refName": "Three Years"
32 |  },
33 |  "isInactive": false,
34 |  "itemId": "DEFEND Package",
35 |  "lastModifiedDate": "2020-1-1T00:00:00Z",
36 |  "member": {
37 |    "links": [
38 |      {
39 |        "rel": "self",
40 |        "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/subscriptionplan/1/member"
41 |      }
42 |    ]
43 |  },
44 |  "subsidiary": {
45 |    "links": [
46 |      {
47 |        "rel": "self",
48 |        "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/subscriptionplan/1/subsidiary"
49 |      }
50 |    ]
51 |  },
52 |  "userNotes": {
53 |    "links": [
54 |      {
55 |        "rel": "self",
56 |        "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/subscriptionplan/1/userNotes"
57 |      }
58 |    ]
59 |  }
60 | }
```

```

60 | }

```

## Use Case For Managing Your Subscription Sales

You can use REST services to manage your subscription sales.

In this use case, you work through the process of using REST to sell a subscription plan for a fictitious company, CED Cyber Solutions.

This use case demonstrates how to:

1. [Create Prerequisite Records](#)
2. [Create a Billing Account](#)
3. [Create a Draft Subscription](#)
4. [Edit Your Draft Subscription](#)
5. [Add an Add-On Item](#)

### Create Prerequisite Records

Before you begin, you need to create the following records:

- [Subscription Plan](#) – If you have not created a subscription plan (or its associated pricing records) yet and would like to do so using REST, see [Use Case For Managing Your Subscription Catalog](#).
- [Price Plan](#) – See the help topic [Creating Price Plans](#)
- [Price Book](#) – See the help topic [Creating Price Books](#)
- [Customer](#) – You need a customer to sell your subscription plan to. In this example, your customer is called Clean Water Co.
- [Billing Schedule](#) – You need to create a charge-based billing schedule to determine how frequently we bill the customer. Your billing schedule is monthly, on the first of the month.

After creating these records, make note of their internal IDs. Throughout this example, use the IDs listed in the following table:

| Record Type                       | Name                           | ID |
|-----------------------------------|--------------------------------|----|
| <a href="#">Subscription Plan</a> | DEFEND Package                 | 1  |
| <a href="#">Price Book</a>        | Standard Pricing               | 1  |
| <a href="#">Customer</a>          | Clean Water Co.                | 1  |
| <a href="#">Subscription</a>      | Three Years                    | 1  |
| <a href="#">Billing Schedule</a>  | Monthly, 1st of the month      | 1  |
| <a href="#">Service Items</a>     | Managed Detection and Response | 7  |

### Create a Billing Account

Clean Water Co. wants to be billed on the first of the month. They use the same currency as your primary currency. Since they are starting their subscription on January 1, 2020, that is when billing starts.

The **startDate** must always be in the format YYYY-MM-DD. Use this format regardless of the date format preferences set.

A successful request returns a response with HTTP status **204 No Content**. The returned response headers contain the endpoint for the new record. Make note of the Billing Account ID. The example Billing Account internal ID is 1.

#### Example Billing Account

```
1 | {
2 |   "billingSchedule": { "id": "1" },
3 |   "currency": { "id": "1" },
4 |   "customer": { "id": "16" },
5 |   "name": "REST Billing Account",
6 |   "startDate": "2020-01-01"
7 | }
```

## Create a Draft Subscription

Your subscription references all records that you have created.

On a successful response, make note of the subscription ID. Use the ID to make changes to your draft subscription. The ID for your example subscription is 1.

#### Example Subscription

```
1 | {
2 |   "customer": { "id": "1" },
3 |   "billingAccount": { "id": "1" },
4 |   "subscriptionPlan": { "id": "1" },
5 |   "priceBook": { "id": "1" },
6 |   "initialTerm": { "id": "1" }
7 | }
```

## Edit Your Draft Subscription

When you created your subscription, you chose not to supply information about the line items included or the pricing of those line items. This means that the fields use the default values from the subscription plan.

If desired, you can change the defaults. To change the defaults, you send a PATCH request with the ID of the target record appended to the end of the patch request. The contents of the request body only need to include your changes. Make sure that all changes to dependent fields are included.

Clean Water Co. wants the silver support tier and has 25 users. Because they are one of your first customers, they receive a 50% discount on their setup costs.



**Note:** Leaving a field undefined leaves it unchanged. If you want to set the value of a field to an empty value, you must explicitly set the field to null in the request body.

A successful request returns a response with HTTP status **204 No Content**. The endpoint for the modified record is returned in the response headers.

#### Example Edited Subscription

```
1 | {
```

```

2      "subscriptionLine": {
3        "items": [
4          {
5            "lineNumber": 4,
6            "isIncluded": true
7          }
8        ]
9      },
10     "priceInterval": {
11       "items": [
12         {
13           "subscriptionPlanLineNumber": 1,
14           "discount": 50.0
15         },
16         {
17           "subscriptionPlanLineNumber": 3,
18           "quantity": 25
19         }
20       ]
21     }
22   }

```

## Add an Add-On Item

Before adding add-on items to any subscription plan, you must [Enabling SuiteBilling Features](#).

Clean Water Co. would like to add Managed Detection and Response to their subscription. The requirements for the line are as follows:

- Bill in advance
- Always add to renewal subscriptions
- Have a prorated start and end date
- Charge \$50.00 per month, prorated by day

Adding an add-on item to a draft subscription is an edit. You complete this edit by sending a PATCH request to the subscription endpoint with the ID appended to the end.

Because the add-on item is not tied to any subscription plan or price book, you must supply all of the subscription line and price book line information.

Because the add-on line requires new pricing information, you must create another price plan. If you are unfamiliar with this process, see [Use Case For Managing Your Subscription Catalog](#). You assign an ID of 19 to the new price plan.

If the customer decides that they no longer want the add-on item, you can PATCH the subscription to mark the line as not included.

### Example Add-On Items

```

1  {
2    "subscriptionLine": {
3      "items": [
4        {
5          "lineNumber": 7,
6          "item": { "id": "7" },
7          "subscriptionLineType": "2",
8          "billingMode": "IN_ADVANCE",
9          "renewalOption": "ALWAYS",
10         "prorateStartDate": true,
11         "prorateEndDate": true
12       }
13     ]
14   }

```

```

14 | },
15 |   "priceInterval": {
16 |     "items": [
17 |       {
18 |         "subscriptionPlanLineNumber": 7,
19 |         "pricePlan": { "id": "19" },
20 |         "frequency": "MONTHLY",
21 |         "repeatEvery": "1",
22 |         "startOffsetUnit": "MONTH",
23 |         "startOffsetValue": 1,
24 |         "prorateBy": "DAY"
25 |       }
26 |     ]
27 |   }
28 | }

```

## Inventory Item Use Cases

This section includes the following sample use cases for inventory items:

- Use Case for Changing the Location of an Inventory Item
- Use Case for Adding a Vendor to an Inventory Item
- Use Case for Finding Items that are Assigned a Pricing Group
- Use Case for Deleting an Inventory Item

Inventory item records are used to track information about items for which you maintain a stock. For more information, read the help topic [Inventory Items](#).

## About This Tutorial

The following `{{ VARIABLE }}` expressions are variables that you need to customize when reusing the code samples.

| Variable                       | Definition                                                                                                                            |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>{{REST_SERVICES}}</code> | Indicates your access to the NetSuite REST Service (for example, <code>https://testaccount.corp.netsuite.com/services/rest</code> ).  |
| <code>{{*_ID}}</code>          | These variables refer to the internal ID of the record (for example, <code>INVENTORY_ITEM_ID</code> or <code>SALES_ORDER_ID</code> ). |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case for Changing the Location of an Inventory Item

To change the location of the inventory item, enter the internal ID of the new location in the body.

```

1 | PATCH {{REST_SERVICES}}/record/v1/inventoryItem/{{INVENTORY_ITEM_ID}}
2 |
3 | {
4 |   "location" : 3

```

```
5 | }
```

## Use Case for Adding a Vendor to an Inventory Item

To add vendor information to the inventory item, send a PATCH request with the vendor ID and subsidiary ID in the body.

```
1 | PATCH {{REST_SERVICES}}/record/v1/inventoryItem/{{INVENTORY_ITEM_ID}}
2 | {
3 |   "itemvendor" : {
4 |     "items" : [{
5 |       "vendor" : {"id" : "1316"},
6 |       "subsidiary" : {"id" : "1"}
7 |     }
8 |   ]
9 | }
10 | }
```

## Use Case for Finding Items that are Assigned a Pricing Group

Use the GET command and query with internal ID of the pricing group to retrieve the list of items that are assigned a pricing group.

```
1 | GET {{REST_SERVICES}}/record/v1/inventoryitem/?q=pricinggroup EQUAL 1
```

In the response body, you will see all the inventory item records that have the pricing group selected.

```
1 | {
2 |   "links": [
3 |     {
4 |       "rel": "self",
5 |       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/inventoryitem/?q=pricinggroup+EQUAL+1"
6 |     }
7 |   ],
8 |   "count": 2,
9 |   "hasMore": false,
10 |  "items": [
11 |    {
12 |      "links": [
13 |        {
14 |          "rel": "self",
15 |          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/inventoryitem/86"
16 |        }
17 |      ],
18 |      "id": "86"
19 |    },
20 |    {
21 |      "links": [
22 |        {
23 |          "rel": "self",
24 |          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/inventoryitem/84"
25 |        }
26 |      ],
27 |      "id": "84"
28 |    }
29 |  ],
30 |  "offset": 0,
31 |  "totalResults": 2
32 | }
```

## Use Case for Deleting an Inventory Item

To delete an inventory item, send the DELETE command to delete the record with the specified ID.

```
1 | DELETE {{REST_SERVICES}}/record/v1/inventoryitem/{{INVENTORY_ITEM_ID}}
```

## Customer Use Cases

This section includes the following sample use cases for customer record:

- [Use Case for Finding Customers with Shipping Carrier Set to UPS](#)
- [Use Case for Updating Contact Details of a Customer](#)
- [Use Case for Creating a Customer from an External ID](#)
- [Use Case for Fetching a List of All Customers with Pagination](#)
- [Use Case for Creating an Invoice from a Customer](#)

Customer records track information about your customers and enable you to view past transactions and communications with them. For more information, read the help topic [Customers](#).

## About This Tutorial

The following `{{ VARIABLE }}` expressions are variables that you need to customize when reusing the code samples.

| Variable                       | Definition                                                                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>{{REST_SERVICES}}</code> | Indicates your access to the NetSuite REST Service (for example, <code>https://testaccount.corp.netsuite.com/services/rest</code> ). |
| <code>{{*_ID}}</code>          | These variables refer to the internal ID of the record (for example, <code>CUSTOMER_ID</code> or <code>SALES_ORDER_ID</code> ).      |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case for Finding Customers with Shipping Carrier Set to UPS

Use the GET command to retrieve the list of customers that have the shipping carrier set to UPS.

```
1 | GET {{REST_SERVICES}}/record/v1/customer/?q=shippingCarrier IS "ups"
```

In the response body, you will see all the customer records that have the shipping carrier set to UPS.

```
1 | {
2 |   "links": [
```

```

3      {
4        "rel": "self",
5        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer/?q=shippingCarrier+IS+%22ups
%22"
6      }
7    ],
8    "count": 3,
9    "hasMore": false,
10   "items": [
11     {
12       "links": [
13         {
14           "rel": "self",
15           "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer/315"
16         }
17       ],
18       "id": "315"
19     },
20     {
21       "links": [
22         {
23           "rel": "self",
24           "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer/614"
25         }
26       ],
27       "id": "614"
28     },
29     {
30       "links": [
31         {
32           "rel": "self",
33           "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer/1317"
34         }
35       ],
36       "id": "1317"
37     }
38   ],
39   "offset": 0,
40   "totalResults": 3
41 }

```

## Use Case for Updating Contact Details of a Customer

To update the contact details, enter the updated email and phone number in the body.

```

1 PATCH {{REST_SERVICES}}/record/v1/customer/{{CUSTOMER_ID}}/
2 {
3   "email" : "john.doe@abc.com",
4   "phone": "1234567890"
5 }

```

## Use Case for Creating a Customer from an External ID

Use the PUT command to create a customer using an external ID.

```

1 PUT {{REST_SERVICES}}/record/v1/customer/eid:{{customerExternalId}}
2 {
3   "companyName": "New Company External Id {{timestamp}}",
4   "email": "customer@example.com",
5   "location" : "1"
6 }

```

For more information about using external IDs [Using External IDs](#).

## Use Case for Fetching a List of All Customers with Pagination

Add the limit and offset criteria to your request to get the results in multiple pages. For more information about pagination, read the help topic [Collection Paging](#).

```
1 GET {{REST_SERVICES}}/record/v1/customer?q=dateCreated ON_OR_AFTER "{{thisYearStart}}" AND dateCreated BEFORE "{{thisYearEnd}}"&limit=3&offset=6
```

The following is an example of a response:

```
1 {
2   "links": [
3     {
4       "rel": "previous",
5       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=3&offset=3"
6     },
7     {
8       "rel": "first",
9       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=3&offset=0"
10    },
11    {
12      "rel": "next",
13      "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=3&offset=9"
14    },
15    {
16      "rel": "last",
17      "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=3&offset=12"
18    },
19    {
20      "rel": "self",
21      "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=3&offset=6&q=dateCreated+ON_OR_AFTER+%21%2F1%2F2022%22+AND+dateCreated+BEFORE+%21%2F1%2F2023%22"
22    }
23  ],
24  "count": 3,
25  "hasMore": true,
26  "items": [
27    {
28      "links": [
29        {
30          "rel": "self",
31          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer/514"
32        }
33      ],
34      "id": "514"
35    },
36    {
37      "links": [
38        {
39          "rel": "self",
40          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer/614"
41        }
42      ],
43      "id": "614"
44    },
45    {
46      "links": [
47        {
48          "rel": "self",
49          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer/716"
50        }
51      ],
52      "id": "716"
53    }
54  ],
55  "offset": 6,
56  "totalResults": 14
```

57 | }

## Use Case for Creating an Invoice from a Customer

To create an invoice from a customer, include the values of required fields such as posting period, location, and item details in the request body.

```
1 POST {{REST_SERVICES}}/record/v1/customer/{{CUSTOMER_ID}}/!transform/invoice
2 {
3   "postingperiod" : "15",
4   "location" : "1",
5   "item": {
6     "items": [
7       {
8         "item": { "id": "86" },
9         "amount": 10
10      }
11    ]
12  }
13 }
```

## Employee Use Cases

This section includes the following sample use cases for employee record:

- Use Case for Adding a Supervisor to an Employee
- Use Case for Adding Employee Address
- Use Case for Adding Subscriptions to an Employee
- Use Case for Creating an Expense Report from an Employee Record
- Use Case for Finding All Employee Records Under a Supervisor
- Use Case for Retrieving Permissions Assigned to a Role
- Use Case for Finding IDs of Roles Assigned to an Employee
- Use Case for Finding the Name of a Role

Employee records represent your employees. Use the employee record to store information for contact, login, payroll, and human resources purposes. For more information, see .

## About This Tutorial

The following {{ VARIABLE }} expressions are variables that you need to customize when reusing the code samples.

| Variable          | Definition                                                                                                             |
|-------------------|------------------------------------------------------------------------------------------------------------------------|
| {{REST_SERVICES}} | Indicates your access to the NetSuite REST Service (for example, https://testaccount.corp.netsuite.com/services/rest). |
| {{*_ID}}          | These variables refer to the internal ID of the record (for example, CUSTOMER_ID or EMPLOYEE_ID).                      |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case for Adding a Supervisor to an Employee

To add or update the supervisor of the employee, enter the internal ID of the supervisor in the request body.

```
1 PATCH {{REST_SERVICES}}/record/v1/employee/<id>/
2 {
3   "supervisor": "13"
4 }
```

## Use Case for Adding Employee Address

To add employee address, send a PATCH request with the address details in the request body.

```
1 PATCH {{REST_SERVICES}}/record/v1/employee/<id>/
2 {
3   "addressbook": {
4     "items": [
5       {
6         "label": "New York HQ",
7         "addressbookaddress": {
8           "country": "US",
9           "state": "NY",
10          "zip": "10001",
11          "addressee": "New company"
12        }
13      }
14    ]
15  }
16 }
```

## Use Case for Adding Subscriptions to an Employee

To add subscriptions to the employee, use the PATCH command and add the subscription details in the request body.

```
1 PATCH {{REST_SERVICES}}/record/v1/employee/1720/
2 {
3   "subscriptions": {
4     "items": [
5       {
6         "subscription": {
7           "id": "1"
8         },
9         "subscribed": true
10      }
11    ]
12  }
13 }
```

## Use Case for Creating an Expense Report from an Employee Record

To create an expense report from an employee record, include the values of required fields such as expense account, currency, and amount in the request body.

```

1 PATCH {{REST_SERVICES}}/record/v1/employee/<id>/!transform/ExpenseReport
2 {
3   "expense": {
4     "items": [
5       {
6         "expenseaccount": "58",
7         "currency": "1",
8         "amount": 100
9       }
10    ]
11  }
12 }

```

## Use Case for Finding All Employee Records Under a Supervisor

Use the GET command to fetch all employee records under a supervisor.

```

1 GET {{REST_SERVICES}}/record/v1/employee/?q=supervisor EQUAL 13

```

In the response body, you will see all the employee records under the supervisor.

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/employee/?q=supervisor+EQUAL+13"
6     }
7   ],
8   "count": 2,
9   "hasMore": false,
10  "items": [
11    {
12      "links": [
13        {
14          "rel": "self",
15          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/employee/1720"
16        }
17      ],
18      "id": "1720"
19    },
20    {
21      "links": [
22        {
23          "rel": "self",
24          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/employee/1519"
25        }
26      ],
27      "id": "1519"
28    }
29  ],
30  "offset": 0,
31  "totalResults": 2
32 }

```

## Use Case for Retrieving Permissions Assigned to a Role

To get permissions assigned to a role, send a SuiteQL query with the role id. Add the limit and offset criteria to your request to get the results in multiple pages. For more information about pagination, read the help topic [Collection Paging](#).

Note that Prefer: transient is a required header parameter.

```

1 POST {{REST_SERVICES}}/query/v1/suiteql?limit=5&offset=10
2 {
3   "q": "SELECT * FROM rolepermissions WHERE role = 1028"
4 }

```

The following is an example of a response:

```

1 {
2   "links": [
3     {
4       "rel": "previous",
5       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql?limit=5&offset=5"
6     },
7     {
8       "rel": "first",
9       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql?limit=5&offset=0"
10    },
11    {
12      "rel": "next",
13      "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql?limit=5&offset=15"
14    },
15    {
16      "rel": "last",
17      "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql?limit=5&offset=60"
18    },
19    {
20      "rel": "self",
21      "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql?limit=5&offset=10"
22    }
23  ],
24  "count": 5,
25  "hasMore": true,
26  "items": [
27    {
28      "links": [],
29      "name": "Custom Fields",
30      "permkey": "ADMI_CUSTFIELD",
31      "permlevel": "4",
32      "restriction": "-1",
33      "role": "1028"
34    },
35    {
36      "links": [],
37      "name": "Custom Item Fields",
38      "permkey": "ADMI_CUSTITEMFIELD",
39      "permlevel": "4",
40      "restriction": "-1",
41      "role": "1028"
42    },
43    {
44      "links": [],
45      "name": "Custom Lists",
46      "permkey": "ADMI_CUSTLIST",
47      "permlevel": "4",
48      "restriction": "-1",
49      "role": "1028"
50    },
51    {
52      "links": [],
53      "name": "Custom PDF Layouts",
54      "permkey": "ADMI_CUSTLAYOUT",
55      "permlevel": "4",
56      "restriction": "-1",
57      "role": "1028"
58    },
59    {
60      "links": [],
61      "name": "Custom Record Types",
62      "permkey": "ADMI_CUSTRECORD",

```

```

63     "permlevel": "4",
64     "restriction": "-1",
65     "role": "1028"
66   }
67 ],
68 "offset": 10,
69 "totalResults": 61
70 }

```

## Use Case for Finding IDs of Roles Assigned to an Employee

To find IDs of roles assigned to an employee, send a SuiteQL query with the employee ID. Note that Prefer: transient is a required header parameter.

```

1 POST {{REST_SERVICES}}/query/v1/suiteql?
2 {
3   "q": "SELECT rolesforsearch FROM employee WHERE id = 13"
4 }

```

The following is an example of the response:

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql"
6     }
7   ],
8   "count": 1,
9   "hasMore": false,
10  "items": [
11    {
12      "links": [],
13      "rolesforsearch": "1, 3"
14    }
15  ],
16  "offset": 0,
17  "totalResults": 1
18 }

```

## Use Case for Finding the Name of a Role

To find the name of a role, send a SuiteQL query with the role ID. Note that Prefer: transient is a required header parameter.

```

1 POST {{REST_SERVICES}}/query/v1/suiteql?
2 {
3   "q": "SELECT name FROM role where id = 7"
4 }

```

The following is an example of the response:

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql"
6     }
7   ],
8   "count": 1,

```

```

9      "hasMore": false,
10     "items": [
11       {
12         "links": [],
13         "name": "A/P Clerk"
14       }
15     ],
16     "offset": 0,
17     "totalResults": 1
18   }

```

## Invoice Use Cases

This section includes the following sample use cases for invoice record:

- [Use Case for Creating an Invoice](#)
- [Use Cases for Updating an Invoice](#)
- [Use Case or Retrieving an Invoice](#)
- [Use Case for Fetching Invoices from a Lead Source](#)
- [Use Case for Deleting an Invoice](#)

An invoice is a record of a sale to a customer. Invoicing is the process of creating bills for goods and services that customers receive. For more information, see the help topic [Invoices](#).

## About this Tutorial

The following `{{ VARIABLE }}` expressions are variables that you need to customize when reusing the code samples.

| Variable                       | Definition                                                                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>{{REST_SERVICES}}</code> | Indicates your access to the NetSuite REST Service (for example, <code>https://testaccount.corp.netsuite.com/services/rest</code> ). |
| <code>{{*_ID}}</code>          | These variables refer to the internal ID of the record (for example, <code>INVOICE_ID</code> or <code>EMPLOYEE_ID</code> ).          |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case for Creating an Invoice

To create an invoice with an item, include the values of required fields such as posting period, location, item ID, and amount in the request body.

```

1  POST {{REST_SERVICES}}/record/v1/invoice/
2  {
3    "entity": {
4      "id": "614"
5    },
6    "postingperiod": "21",
7    "location": 1,
8    "item": {

```

```

9      "items": [
10         {
11             "amount": 1000.0,
12             "item": {
13                 "id": "86"
14             }
15         }
16     ],
17 },
18 "subsidiary": {
19     "id": "1"
20 },
21 "terms": {
22     "id": "1"
23 }
24 }

```

For more options to create an invoice using REST API, see [Invoice](#).

## Use Cases for Updating an Invoice

Use the PATCH command to add a line item to an invoice:

```

1 PATCH {{REST_SERVICES}}/record/v1/invoice/<id>/
2 {
3     "item": {
4         "items": [
5             {
6                 "item": {
7                     "id": "13"
8                 },
9                 "quantity": 50,
10                "amount": 100
11            }
12        ]
13    }
14 }

```

To update a line item in an invoice, add the line item ID in the request.

```

1 PATCH {{REST_SERVICES}}/record/v1/invoice/<id>/
2 {
3     "item": {
4         "items": [
5             {
6                 "line": 5,
7                 "item": {
8                     "id": "10"
9                 },
10                "quantity": 75,
11                "amount": 90
12            }
13        ]
14    }
15 }

```

## Use Case or Retrieving an Invoice

Use the GET command to retrieve an invoice.

```

1 GET {{REST_SERVICES}}/record/v1/invoice/1215/

```

In the response body, you will get details of the requested invoice.

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/invoice/1215/"
6     }
7   ],
8   "amountPaid": 0.0,
9   "amountRemaining": 1000.0,
10  "amountRemainingTotalBox": 1000.0,
11  "billAddress": "Dwight Schrute\nNew York NY 10001\nUnited States",
12  "billAddressList": {
13    "links": [],
14    "id": "401",
15    "refName": "New York HQ"
16  },
17  "billingAddress": {
18    "links": [
19      {
20        "rel": "self",
21        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/invoice/1215/billingAddress"
22      }
23    ]
24  },
25  "billingAddress_text": "Dwight Schrute\nNew York NY 10001\nUnited States",
26  "createdDate": "2023-02-06T09:48:00Z",
27  "currency": {
28    "links": [
29      {
30        "rel": "self",
31        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/currency/1"
32      }
33    ],
34    "id": "1",
35    "refName": "USA"
36  },
37  "custbody_atlas_exist_cust_hdn": {
38    "links": [
39      {
40        "rel": "self",
41        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customlist_atlas_cust_type/2"
42      }
43    ],
44    "id": "2",
45    "refName": "Existing Customer"
46  },
47  "custbody_atlas_new_cust_hdn": {
48    "links": [
49      {
50        "rel": "self",
51        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customlist_atlas_cust_type/1"
52      }
53    ],
54    "id": "1",
55    "refName": "New Customer"
56  },
57  "custbody_atlas_no_hdn": {
58    "links": [
59      {
60        "rel": "self",
61        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customlist_atlas_appr_by_creator/2"
62      }
63    ],
64    "id": "2",
65    "refName": "No"
66  },
67  "custbody_atlas_yes_hdn": {
68    "links": [
69      {
70        "rel": "self",
71        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customlist_atlas_appr_by_creator/1"

```

```

72     }
73   ],
74   "id": "1",
75   "refName": "Yes"
76 },
77 "custbody_esc_created_date": "2023-02-06",
78 "custbody_esc_last_modified_date": "2023-02-06",
79 "customForm": {
80   "id": "91",
81   "refName": "Standard Service Invoice"
82 },
83 "dueDate": "2023-02-21",
84 "email": "another.customer@example.com",
85 "entity": {
86   "links": [
87     {
88       "rel": "self",
89       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer/614"
90     }
91   ],
92   "id": "614",
93   "refName": "8 Another Company 1661496973"
94 },
95 "estGrossProfit": 1000.0,
96 "estGrossProfitPercent": 100.0,
97 "exchangeRate": 1.0,
98 "id": "1215",
99 "item": {
100   "links": [
101     {
102       "rel": "self",
103       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/invoice/1215/item"
104     }
105   ]
106 },
107 "lastModifiedDate": "2023-02-06T09:48:00Z",
108 "location": {
109   "links": [
110     {
111       "rel": "self",
112       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/location/1"
113     }
114   ],
115   "id": "1",
116   "refName": "California"
117 },
118 "originator": "restWebServices",
119 "postingPeriod": {
120   "links": [
121     {
122       "rel": "self",
123       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/accountingperiod/21"
124     }
125   ],
126   "id": "21",
127   "refName": "Feb 2023"
128 },
129 "prevDate": "2023-02-06",
130 "salesEffectiveDate": "2023-02-06",
131 "shipAddress": "Dwight Schrute\nNew York NY 10001\nUnited States",
132 "shipAddressList": {
133   "links": [],
134   "id": "401",
135   "refName": "New York HQ"
136 },
137 "shipDate": "2023-02-06",
138 "shipIsResidential": false,
139 "shipOverride": false,
140 "shippingAddress": {
141   "links": [
142     {
143       "rel": "self",
144       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/invoice/1215/shippingAddress"

```

```

145     }
146   ]
147 },
148 "shippingAddress_text": "Dwight Schrute\nNew York NY 10001\nUnited States",
149 "source": {
150   "id": "REST Web Services",
151   "refName": "REST Web Services"
152 },
153 "status": {
154   "id": "Open",
155   "refName": "Open"
156 },
157 "subsidiary": {
158   "links": [
159     {
160       "rel": "self",
161       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/subsidiary/1"
162     }
163   ],
164   "id": "1",
165   "refName": "Parent Company"
166 },
167 "subtotal": 1000.0,
168 "terms": {
169   "links": [
170     {
171       "rel": "self",
172       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/term/1"
173     }
174   ],
175   "id": "1",
176   "refName": "Net 15"
177 },
178 "toBeEmailed": false,
179 "toBeFaxed": false,
180 "toBePrinted": false,
181 "total": 1000.0,
182 "totalCostEstimate": 0.0,
183 "tranDate": "2023-02-06",
184 "tranId": "INV07"
185 }

```

## Use Case for Fetching Invoices from a Lead Source

To fetch invoices from a lead source, query the lead source's ID in the request.

```
1 | GET {{REST_SERVICES}}/record/v1/invoice/?q=leadsource EQUAL "-4"
```

In the response body, you will get all the invoices from the requested lead source.

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/invoice/?q=leadsource+EQUAL+%22-4%22"
6     }
7   ],
8   "count": 2,
9   "hasMore": false,
10  "items": [
11    {
12      "links": [
13        {
14          "rel": "self",
15          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/invoice/1109"
16        }
17      ],
18      "id": "1109"
19    }

```

```

20     {
21         "links": [
22             {
23                 "rel": "self",
24                 "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/invoice/1115"
25             }
26         ],
27         "id": "1115"
28     }
29 ],
30 "offset": 0,
31 "totalResults": 2
32 }

```

## Use Case for Deleting an Invoice

To delete an invoice, send the DELETE command with the invoice ID.

```
1 | DELETE {{REST_SERVICES}}/record/v1/invoice/<id>/
```

## Journal Entry Use Cases

This section includes the following sample use cases for journal entry record:

- [Use Case for Creating a Journal Entry](#)
- [Use Cases for Approving a Journal Entry](#)
- [Use Case for Creating a Reverse Journal Entry](#)
- [Use Case for Updating Line Items in a Journal Entry](#)
- [Use Case for Retrieving Journal Entries Created After a Date](#)

You use the journal entry record to adjust balances in accounts. Journal entries let you change the value of any set of accounts without having to enter a posting transaction. For more information, read the help topic [Journal Entries](#).

## About This Tutorial

The following `{{ VARIABLE }}` expressions are variables that you need to customize when reusing the code samples.

| Variable                       | Definition                                                                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>{{REST_SERVICES}}</code> | Indicates your access to the NetSuite REST Service (for example, <code>https://testaccount.corp.netsuite.com/services/rest</code> ). |
| <code>{{*_ID}}</code>          | These variables refer to the internal ID of the record (for example, <code>JOURNAL_ENTRY_ID</code> or <code>EMPLOYEE_ID</code> ).    |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case for Creating a Journal Entry

To create a journal entry, include the values of required fields such as subsidiary, account, and debit and credit amounts in the request body. Make sure the debits and credits in the journal entry balance.

```

1 POST {{REST_SERVICES}}/record/v1/journalentry/
2 {
3   "subsidiary": {
4     "id": "1"
5   },
6   "line": {
7     "items": [
8       {
9         "account": {
10          "id": "10"
11        },
12        "debit": 300
13      },
14      {
15        "account": {
16          "id": "14"
17        },
18        "credit": 300
19      }
20    ]
21  }
22 }

```

## Use Cases for Approving a Journal Entry

To approve a journal entry, send a PATCH command in the request body.

```

1 PATCH {{REST_SERVICES}}/record/v1/journalentry/<id>/
2 {
3   "approved": true
4 }

```

## Resetting Approval Status

If you already approved a journal entry but need to reset the approval status, send a PATCH command with the updated status in the request body.

```

1 PATCH {{REST_SERVICES}}/record/v1/journalentry/<id>/
2 {
3   "approved": false
4 }

```

## Use Case for Creating a Reverse Journal Entry

To create a reverse journal entry, send a PATCH command with the reversal date in the request body.

```

1 PATCH {{REST_SERVICES}}/record/v1/journalentry/<id>/
2 {
3   "reversaldate": "2023-2-22"
4 }

```

## Use Case for Updating Line Items in a Journal Entry

To add line items to a journal entry, include the values of required fields such as account, debit and credit amounts in the request body. Make sure the debits and credits in the journal entry balance.

```

1 PATCH {{REST_SERVICES}}/record/v1/journalentry/<id>/

```

```

2  {
3    "line": {
4      "items": [
5        {
6          "account": {
7            "id": "15"
8          },
9          "debit": 250
10         },
11         {
12           "account": {
13             "id": "17"
14           },
15           "credit": 250
16         }
17       ]
18     }
19   }

```

To update an existing line item, mention the line number in the request body.

```

1  PATCH {{REST_SERVICES}}/record/v1/journalentry/<id>/
2  {
3    "line": {
4      "items": [
5        {
6          "line": 2,
7          "account": {
8            "id": "15"
9          },
10         "debit": 1150
11        },
12        {
13          "line": 3,
14          "account": {
15            "id": "17"
16          },
17          "credit": 1150
18        }
19      ]
20    }
21  }

```

## Use Case for Retrieving Journal Entries Created After a Date

To retrieve journal entries created after a date, enter the created date in the request body.

```
1 | {{REST_SERVICES}}/record/v1/journalentry/?q=createddate AFTER "2/14/23"
```

Following is a sample response:

```

1  {
2    "links": [
3      {
4        "rel": "self",
5        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/journalentry/?q=createddate+AFTER+
6        %22%2F14%2F23%22"
7      }
8    ],
9    "count": 7,
10   "hasMore": false,
11   "items": [

```

```

12     "links": [
13         {
14             "rel": "self",
15             "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/journalentry/1415"
16         }
17     ],
18     "id": "1415"
19 },
20 {
21     "links": [
22         {
23             "rel": "self",
24             "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/journalentry/1416"
25         }
26     ],
27     "id": "1416"
28 },
29 {
30     "links": [
31         {
32             "rel": "self",
33             "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/journalentry/1516"
34         }
35     ],
36     "id": "1516"
37 },
38 {
39     "links": [
40         {
41             "rel": "self",
42             "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/journalentry/1417"
43         }
44     ],
45     "id": "1417"
46 },
47 {
48     "links": [
49         {
50             "rel": "self",
51             "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/journalentry/1517"
52         }
53     ],
54     "id": "1517"
55 },
56 {
57     "links": [
58         {
59             "rel": "self",
60             "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/journalentry/1518"
61         }
62     ],
63     "id": "1518"
64 },
65 {
66     "links": [
67         {
68             "rel": "self",
69             "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/journalentry/1618"
70         }
71     ],
72     "id": "1618"
73 }
74 ],
75 "offset": 0,
76 "totalResults": 7
77 }

```

## Event Use Cases

This section includes the following sample use cases for event record:

- Use Case for Creating an Event
- Use Case for Adding an Attendee to an Event
- Use Case for Getting a List of Events with Confirmed Status and that are Scheduled After a Date
- Use Case for Fetching Details of Events at a Location
- Use Case for Deleting an Event

You use event records to reserve time on your calendar for appointments and meetings. For more information, read the help topic [Working with Events](#).

## About This Tutorial

The following `{{ VARIABLE }}` expressions are variables that you need to customize when reusing the code samples.

| Variable                       | Definition                                                                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>{{REST_SERVICES}}</code> | Indicates your access to the NetSuite REST Service (for example, <code>https://testaccount.corp.netsuite.com/services/rest</code> ). |
| <code>{{*_ID}}</code>          | These variables refer to the internal ID of the record (for example, <code>CALENDAREVENT_ID</code> or <code>EMPLOYEE_ID</code> ).    |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case for Creating an Event

To create an event, include values for fields such as title, status, and start date in the request body.

```

1 | POST {{REST_SERVICES}}/record/v1/calendarevent/
2 | {
3 |   "title": "Meeting with Tom",
4 |   "status": "CONFIRMED",
5 |   "message": "Let's discuss work",
6 |   "location": "Caffe West",
7 |   "startdate": "2023-11-04"
8 | }
```

## Use Case for Adding an Attendee to an Event

Use the PATCH command to add an attendee to an event.

```

1 | PATCH {{REST_SERVICES}}/record/v1/calendarevent/<id>/
2 | {
3 |   "attendee": {
4 |     "items": [
5 |       {
6 |         "attendee": {
7 |           "id": "5"
8 |         },
9 |         "response": "ACCEPTED"
10 |      }
11 |     ]
12 |   }
13 | }
```

```

11     ]
12   }
13 }

```

## Use Case for Getting a List of Events with Confirmed Status and that are Scheduled After a Date

Use the GET command and query with status and start date to retrieve the list of events.

```
1 | GET {{REST_SERVICES}}/record/v1/calendarevent/?q=status IS "CONFIRMED"&q=startdate AFTER "3/16/23"
```

The following is an example of the response:

```

1  {
2    "links": [
3      {
4        "rel": "self",
5        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/calendarevent/?q=startdate+AFTER+
%223%2F16%2F23%22"
6      }
7    ],
8    "count": 5,
9    "hasMore": false,
10   "items": [
11     {
12       "links": [
13         {
14           "rel": "self",
15           "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/calendarevent/103"
16         }
17       ],
18       "id": "103"
19     },
20     {
21       "links": [
22         {
23           "rel": "self",
24           "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/calendarevent/2"
25         }
26       ],
27       "id": "2"
28     },
29     {
30       "links": [
31         {
32           "rel": "self",
33           "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/calendarevent/203"
34         }
35       ],
36       "id": "203"
37     },
38     {
39       "links": [
40         {
41           "rel": "self",
42           "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/calendarevent/4"
43         }
44       ],
45       "id": "4"
46     },
47     {
48       "links": [
49         {
50           "rel": "self",
51           "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/calendarevent/3"

```

```

52         }
53     ],
54     "id": "3"
55 }
56 ],
57 "offset": 0,
58 "totalResults": 5
59 }

```

## Use Case for Fetching Details of Events at a Location

To get details of events at a location, send a SuiteQL query with the location. Note that Prefer: transient is a required header parameter.

```

1  {{REST_SERVICES}}/query/v1/suiteql?
2  {
3      "q": "SELECT title, startdate, FROM calendarevent WHERE location = 'Caffe West'"
4  }

```

The following is an example of the response:

```

1  {
2      "links": [
3          {
4              "rel": "self",
5              "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql"
6          }
7      ],
8      "count": 4,
9      "hasMore": false,
10     "items": [
11         {
12             "links": [],
13             "startdate": "11/4/2023",
14             "title": "All hands"
15         },
16         {
17             "links": [],
18             "startdate": "11/3/2023",
19             "title": "Customer Engagement"
20         },
21         {
22             "links": [],
23             "startdate": "11/2/2023",
24             "title": "Meeting with Tom"
25         },
26         {
27             "links": [],
28             "startdate": "11/6/2023",
29             "title": "Private Appointment"
30         }
31     ],
32     "offset": 0,
33     "totalResults": 4
34 }

```

## Use Case for Deleting an Event

To delete an event, send the DELETE command with the event ID.

```

1  DELETE {{REST_SERVICES}}/record/v1/calendarevent/<id>/

```

## Credit Memo Use Cases

This section includes the following sample use cases for credit memo record:

- [Use Case for Creating a Credit Memo](#)
- [Use Case for Adding Line Items to a Credit Memo](#)
- [Use Case for Fetching List of Credit Memos Created After a Date](#)
- [Use Case for Fetching Credit Memos from a Lead Source](#)
- [Use Case for Getting Details of a Credit Memo](#)

A credit memo is a transaction that decreases the amount a customer owes you. You can use a credit memo to reverse a charge you billed to a customer. For more information, read the help topic [Customer Credit Memos](#).

## About This Tutorial

The following `{{ VARIABLE }}` expressions are variables that you need to customize when reusing the code samples.

| Variable                       | Definition                                                                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>{{REST_SERVICES}}</code> | Indicates your access to the NetSuite REST Service (for example, <code>https://testaccount.corp.netsuite.com/services/rest</code> ). |
| <code>{{*_ID}}</code>          | These variables refer to the internal ID of the record (for example, <code>CREDITMEMO_ID</code> or <code>EMPLOYEE_ID</code> ).       |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case for Creating a Credit Memo

To create a credit memo, include values for fields such as customer, location, and item details in the request body.

```

1 POST {{REST_SERVICES}}/record/v1/creditmemo/
2
3 {
4   "entity": {
5     "id": "14"
6   },
7   "item": {
8     "items": [
9       {
10        "amount": 10.0,
11        "item": {
12          "id": "13"
13        }
14      }
15    ]
16  },
17  "location": "1"
18 }
```

## Use Case for Adding Line Items to a Credit Memo

Use the PATCH command to add line items to a credit memo.

```

1 PATCH {{REST_SERVICES}}/record/v1/creditmemo/<id>/
2
3 {
4   "item": {
5     "items": [
6       {
7         "item": {
8           "id": 10
9         },
10        "amount": 100,
11        "quantity": 25
12      }
13    ]
14  }
15 }
```

## Use Case for Fetching List of Credit Memos Created After a Date

Use the GET command and query with the createddate field to get a list of credit memos after the specified date.

```

1 GET {{REST_SERVICES}}/record/v1/creditmemo/?q=createddate AFTER "03/14/2023"
```

In response you will get the list of credit memos, something like this:

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/?q=createddate+AFTER+
6       %2203%2F14%2F2023%22"
7     }
8   ],
9   "count": 3,
10  "hasMore": false,
11  "items": [
12    {
13      "links": [
14        {
15          "rel": "self",
16          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/1818"
17        }
18      ],
19      "id": "1818"
20    },
21    {
22      "links": [
23        {
24          "rel": "self",
25          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/1819"
26        }
27      ],
28      "id": "1819"
29    },
30    {
31      "links": [
```

```

32         "rel": "self",
33         "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/1919"
34     },
35 ],
36     "id": "1919"
37 }
38 ],
39 "offset": 0,
40 "totalResults": 3
41 }

```

## Use Case for Fetching Credit Memos from a Lead Source

To fetch credit memos from a lead source, query the lead source's ID in the request.

```
1 | GET {{REST_SERVICES}}/record/v1/creditmemo/?q=leadsource EQUAL "-2"
```

In the response body, you will get all the credit memos from the requested lead source, something like this:

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/?q=leadsource+EQUAL+
%22-2%22"
6     }
7   ],
8   "count": 2,
9   "hasMore": false,
10  "items": [
11    {
12      "links": [
13        {
14          "rel": "self",
15          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/1818"
16        }
17      ],
18      "id": "1818"
19    },
20    {
21      "links": [
22        {
23          "rel": "self",
24          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/1919"
25        }
26      ],
27      "id": "1919"
28    }
29  ],
30  "offset": 0,
31  "totalResults": 2
32 }

```

## Use Case for Getting Details of a Credit Memo

Use the GET command and query with internal ID of the credit memo to retrieve the details. In the following example, the internal ID of the credit memo is 1819.

```
1 | GET {{REST_SERVICES}}/record/v1/creditmemo/1819/
```

The response will look something like this:

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/1819/"
6     }
7   ],
8   "amountPaid": 0.0,
9   "amountRemaining": 10.0,
10  "applied": 0.0,
11  "billingAddress": {
12    "links": [
13      {
14        "rel": "self",
15        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/1819/billingAddress"
16      }
17    ]
18  },
19  "createdDate": "2023-03-30T10:14:00Z",
20  "currency": {
21    "links": [
22      {
23        "rel": "self",
24        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/currency/1"
25      }
26    ],
27    "id": "1",
28    "refName": "USA"
29  },
30  "custbody_atlas_exist_cust_hdn": {
31    "links": [
32      {
33        "rel": "self",
34        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customlist_atlas_cust_type/2"
35      }
36    ],
37    "id": "2",
38    "refName": "Existing Customer"
39  },
40  "custbody_atlas_new_cust_hdn": {
41    "links": [
42      {
43        "rel": "self",
44        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customlist_atlas_cust_type/1"
45      }
46    ],
47    "id": "1",
48    "refName": "New Customer"
49  },
50  "custbody_atlas_no_hdn": {
51    "links": [
52      {
53        "rel": "self",
54        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customlist_atlas_appr_by_creator/2"
55      }
56    ],
57    "id": "2",
58    "refName": "No"
59  },
60  "custbody_atlas_yes_hdn": {
61    "links": [
62      {
63        "rel": "self",
64        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customlist_atlas_appr_by_creator/1"
65      }
66    ],
67    "id": "1",
68    "refName": "Yes"
69  },
70  "custbody_esc_created_date": "2023-03-30",
71  "custbody_esc_last_modified_date": "2023-03-31",

```

```

72   "customForm": {
73     "id": "94",
74     "refName": "Standard Credit Memo"
75   },
76   "email": "customer@example.com",
77   "entity": {
78     "links": [
79       {
80         "rel": "self",
81         "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/customer/14"
82       }
83     ],
84     "id": "14",
85     "refName": "1 Company 1660029525"
86   },
87   "estGrossProfit": 0.0,
88   "estGrossProfitPercent": 0.0,
89   "exchangeRate": 1.0,
90   "id": "1819",
91   "item": {
92     "links": [
93       {
94         "rel": "self",
95         "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/1819/item"
96       }
97     ]
98   },
99   "lastModifiedDate": "2023-03-31T08:57:00Z",
100  "location": {
101    "links": [
102      {
103        "rel": "self",
104        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/location/1"
105      }
106    ],
107    "id": "1",
108    "refName": "California"
109  },
110  "originator": "restWebServices",
111  "postingPeriod": {
112    "links": [
113      {
114        "rel": "self",
115        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/accountingperiod/22"
116      }
117    ],
118    "id": "22",
119    "refName": "Mar 2023"
120  },
121  "prevDate": "2023-03-30",
122  "salesEffectiveDate": "2023-03-30",
123  "shipIsResidential": false,
124  "shipOverride": false,
125  "shippingAddress": {
126    "links": [
127      {
128        "rel": "self",
129        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/creditmemo/1819/shippingAddress"
130      }
131    ]
132  },
133  "source": {
134    "id": "REST Web Services",
135    "refName": "REST Web Services"
136  },
137  "status": {
138    "id": "Open",
139    "refName": "Open"
140  },
141  "subsidiary": {
142    "links": [
143      {
144        "rel": "self",

```

```

145         "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/subsidiary/1"
146     },
147     ],
148     "id": "1",
149     "refName": "Parent Company"
150 },
151 "subtotal": 10.0,
152 "toBeEmailed": false,
153 "toBeFaxed": false,
154 "toBePrinted": false,
155 "total": 10.0,
156 "totalCostEstimate": 10.0,
157 "tranDate": "2023-03-30",
158 "tranId": "CM02",
159 "unapplied": 10.0
160 }

```

## Vendor Use Cases

This section includes the following sample use cases for vendor record:

- Use Case for Creating a Vendor
- Use Case for Creating a Tax Agency Vendor
- Use Case for Providing Role Access to a Vendor
- Use Case for Inactivating a Vendor
- Use Case for Setting Credit Limit for a Vendor
- Use Case for Defining the Preferred Transaction Delivery Methods for a Vendor
- Use Case for Finding Vendors that are Assigned an Expense Account
- Use Case for Getting Details of Vendors that are Assigned a Category
- Use Case for Getting List of Vendors that have Credit Limits Above a Value

A vendor is a company or person you purchase goods and services from. Vendor records track information about your vendors and enable you to view past transactions and communications with them. For information about working with vendor records in the UI, see the help topic [Vendor Records Overview](#).

## About This Tutorial

The following `{{ VARIABLE }}` expressions are variables that you need to customize when reusing the code samples.

| Variable                       | Definition                                                                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>{{REST_SERVICES}}</code> | Indicates your access to the NetSuite REST Service (for example, <code>https://testaccount.corp.netsuite.com/services/rest</code> ). |
| <code>{{*_ID}}</code>          | These variables refer to the internal ID of the record (for example, <code>VENDOR_ID</code> or <code>EMPLOYEE_ID</code> ).           |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case for Creating a Vendor

To create a vendor, include values for fields such as company name in the request body.

```

1 POST {{REST_SERVICES}}/record/v1/vendor/
2 {
3   "companyName": "NZ Vendor"
4 }

```

## Use Case for Creating a Tax Agency Vendor

To create a tax agency vendor, include the id of tax agency in the category field.

```

1 POST {{REST_SERVICES}}/record/v1/vendor/
2 {
3   "companyName": "BZ Vendor",
4   "category": "3"
5 }

```

## Use Case for Providing Role Access to a Vendor

To give a role access to a vendor, enable the **Give Access** box and add the role details in the request body.

```

1 PATCH {{REST_SERVICES}}/record/v1/vendor/<id>/
2 {
3   "giveaccess": true,
4   "roles": {
5     "items": [
6       {
7         "selectedRole": {
8           "id": "6"
9         }
10      }
11    ]
12  }
13 }

```

## Use Case for Inactivating a Vendor

Use the PATCH command and enable the **Inactive** box in the request body.

```

1 PATCH {{REST_SERVICES}}/record/v1/vendor/<id>/
2 {
3   "isinactive": true
4 }

```

## Use Case for Setting Credit Limit for a Vendor

Use the PATCH command and set the credit limit in the request body.

```

1 PATCH {{REST_SERVICES}}/record/v1/vendor/<id>/
2 {
3   "creditlimit": 10000
4 }

```

## Use Case for Defining the Preferred Transaction Delivery Methods for a Vendor

Use the PATCH command and enable the desired preferred transaction delivery methods in the request body. In the following example, email and print transactions are enabled.

```
1 | PATCH {{REST_SERVICES}}/record/v1/vendor/<id>/
2 | {
3 |   "emailtransactions": true,
4 |   "printtransactions": true
5 | }
```

## Use Case for Finding Vendors that are Assigned an Expense Account

Use the GET command and query with the id of the expense account to get the list of vendors that are assigned an expense account.

```
1 | GET {{REST_SERVICES}}/record/v1/vendor/?q=expenseaccount EQUAL "10"
```

Following is a sample response:

```
1 | {
2 |   "links": [
3 |     {
4 |       "rel": "self",
5 |       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/vendor/?q=expenseaccount+EQUAL+
6 |       %2210%22"
7 |     }
8 |   ],
9 |   "count": 3,
10 |  "hasMore": false,
11 |  "items": [
12 |    {
13 |      "links": [
14 |        {
15 |          "rel": "self",
16 |          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/vendor/1723"
17 |        }
18 |      ],
19 |      "id": "1723"
20 |    },
21 |    {
22 |      "links": [
23 |        {
24 |          "rel": "self",
25 |          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/vendor/1722"
26 |        }
27 |      ],
28 |      "id": "1722"
29 |    },
30 |    {
31 |      "links": [
32 |        {
33 |          "rel": "self",
34 |          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/vendor/1823"
35 |        }
36 |      ],
37 |      "id": "1823"
38 |    }
39 |  ],
40 |  "offset": 0,
```

```

40 |   "totalResults": 3
41 | }

```

## Use Case for Getting Details of Vendors that are Assigned a Category

To get details of vendors that are assigned a category, send a SuiteQL query with the category. Note that Prefer: transient is a required header parameter.

```

1 | POST {{REST_SERVICES}}/query/v1/suiteql?
2 | {
3 |   "q": "SELECT id, legalname, expenseaccount FROM vendor WHERE category = 3"
4 | }

```

Following is a sample response:

```

1 | {
2 |   "links": [
3 |     {
4 |       "rel": "self",
5 |       "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql"
6 |     }
7 |   ],
8 |   "count": 2,
9 |   "hasMore": false,
10 |  "items": [
11 |    {
12 |      "links": [],
13 |      "expenseaccount": "12",
14 |      "id": "1725",
15 |      "legalname": "BZ Vendor"
16 |    },
17 |    {
18 |      "links": [],
19 |      "expenseaccount": "10",
20 |      "id": "1823",
21 |      "legalname": "Test State Tax Agency"
22 |    }
23 |  ],
24 |  "offset": 0,
25 |  "totalResults": 2
26 | }

```

## Use Case for Getting List of Vendors that have Credit Limits Above a Value

To get vendors that have a credit limit above a certain value, send a SuiteQL query with the credit limit. Note that Prefer: transient is a required header parameter.

```

1 | POST {{REST_SERVICES}}/query/v1/suiteql?
2 | {
3 |   "q": "SELECT id, entityid AS 'vendor name', creditlimit FROM vendor WHERE creditlimit > 5000"
4 | }

```

Following is a sample response:

```

1 | {
2 |   "links": [

```

```

3      {
4        "rel": "self",
5        "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql"
6      }
7    ],
8    "count": 2,
9    "hasMore": false,
10   "items": [
11     {
12       "links": [],
13       "creditlimit": "8000",
14       "id": "1725",
15       "vendor name": "BZ Vendor"
16     },
17     {
18       "links": [],
19       "creditlimit": "10000",
20       "id": "1722",
21       "vendor name": "Test"
22     }
23   ],
24   "offset": 0,
25   "totalResults": 2
26 }

```

## Vendor Bill Use Cases

This section includes the following sample use cases for vendor bill record:

- Use Case for Creating a Vendor Bill
- Use Case for Approving a Vendor Bill
- Use Case for Adding Line Items to a Vendor Bill
- Use Case for Getting a List of Bills from a Vendor

Vendor bills are records which you can use to track payables as they arrive from vendors. For information about working with vendor bills in the UI, see the help topic [Vendor Bills](#).

## About This Tutorial

The following `{{ VARIABLE }}` expressions are variables that you need to customize when reusing the code samples.

| Variable                       | Definition                                                                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>{{REST_SERVICES}}</code> | Indicates your access to the NetSuite REST Service (for example, <code>https://testaccount.corp.netsuite.com/services/rest</code> ). |
| <code>{{*_ID}}</code>          | These variables refer to the internal ID of the record (for example, <code>VENDORBILL_ID</code> or <code>EMPLOYEE_ID</code> ).       |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case for Creating a Vendor Bill

To create a vendor bill, include values for fields such as vendor and item details in the request body.

```

1 POST {{REST_SERVICES}}/record/v1/vendorbill
2 {
3   "entity": {
4     "id": "1722"
5   },
6   "item": {
7     "items": [
8       {
9         "item": {
10          "id": 84
11        },
12        "quantity": 25,
13        "rate": 100
14      }
15    ]
16  }
17 }

```

## Use Case for Approving a Vendor Bill

To approve a vendor bill, use the PATCH command and include the approval status in the request body.

```

1 PATCH {{REST_SERVICES}}/record/v1/vendorbill/<id>
2 {
3   "approvalstatus": 2
4 }

```

## Use Case for Adding Line Items to a Vendor Bill

To add line items to a vendor bill, use the PATCH command and include the item details in the request body.

```

1 PATCH {{REST_SERVICES}}/record/v1/vendorbill/<id>
2 {
3   "item": {
4     "items": [
5       {
6         "item": {
7          "id": 84
8        },
9        "quantity": 30,
10        "rate": 50
11      }
12    ]
13  }
14 }

```

## Use Case for Getting a List of Bills from a Vendor

Use the GET command and query with the vendor id to get the list of bills from a vendor.

```

1 GET {{REST_SERVICES}}/record/v1/vendorbill/?q=entity EQUAL 1722

```

Following is an example of the response:

```

1 {

```

```
2      "links": [
3        {
4          "rel": "self",
5          "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/vendorbill/?q=entity+EQUAL+%221722%22"
6        }
7      ],
8      "count": 2,
9      "hasMore": false,
10     "items": [
11       {
12         "links": [
13           {
14             "rel": "self",
15             "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/vendorbill/1920"
16           }
17         ],
18         "id": "1920"
19       },
20       {
21         "links": [
22           {
23             "rel": "self",
24             "href": "https://<accountID>.suitetalk.api.netsuite.com/services/rest/record/v1/vendorbill/2020"
25           }
26         ],
27         "id": "2020"
28       }
29     ],
30     "offset": 0,
31     "totalResults": 2
32   }
```

# Cash Sale Use Cases

This section includes the following sample use cases for cash sale record:

- Use Case for Creating a Cash Sale
- Use Case for Updating a Cash Sale
- Use Case for Issuing a Return Authorization from a Cash Sale
- Use Case for Fetching List of Cash Sales for a Customer

A cash sale is a transaction that records the sale of goods or services for which you receive immediate payment. For more information, read the help topic [Cash Sales](#).

## About This Tutorial

The following {{ VARIABLE }} expressions are variables that you need to customize when reusing the code samples.

| Variable          | Definition                                                                                                             |
|-------------------|------------------------------------------------------------------------------------------------------------------------|
| {{REST_SERVICES}} | Indicates your access to the NetSuite REST Service (for example, https://testaccount.corp.netsuite.com/services/rest). |
| {{*_ID}}          | These variables refer to the internal ID of the record (for example, CASHSALE_ID or EMPLOYEE_ID).                      |

The IDs and the variables used in the following scenarios are examples only, and are included to show how to write REST commands.

## Use Case for Creating a Cash Sale

To create a cash sale, include values for fields such as customer ID, location, and item details in the request body.

```

1 POST {{REST_SERVICES}}/record/v1/cashsale
2 {
3   "entity": {
4     "id": "14"
5   },
6   "location": "1",
7   "item": {
8     "items": [
9       {
10        "item": {
11          "id": "13"
12        },
13        "quantity": 25,
14        "amount": 275
15      }
16    ]
17  }
18 }
```

## Use Case for Updating a Cash Sale

Use the PATCH command to update a cash sale.

```

1 PATCH {{REST_SERVICES}}/record/v1/cashsale/<id>
2 {
3   "item": {
4     "items": [
5       {
6         "item": {
7           "id": "14"
8         },
9         "quantity": 50,
10        "amount": 300
11      }
12    ]
13  }
14 }
```

## Use Case for Issuing a Return Authorization from a Cash Sale

To create a return authorization from a cash sale, use the transform function in the REST API:

```

1 POST {{REST_SERVICES}}/record/v1/cashsale/<id>!!transform/returnauthorization
```

## Use Case for Fetching List of Cash Sales for a Customer

Use the GET command and query with the customer id to get the list of cash sales from a customer.

```

1 GET {{REST_SERVICES}}/record/v1/cashsale/?q=entity EQUAL "14"
```

Following is an example of the response:

```

1  {
2    "links": [
3      {
4        "rel": "self",
5        "href": "https://5629979.suitetalk.api.netsuite.com/services/rest/record/v1/cashsale/?q=entity+EQUAL+%2214%22"
6      }
7    ],
8    "count": 3,
9    "hasMore": false,
10   "items": [
11     {
12       "links": [
13         {
14           "rel": "self",
15           "href": "https://5629979.suitetalk.api.netsuite.com/services/rest/record/v1/cashsale/2121"
16         }
17       ],
18       "id": "2121"
19     },
20     {
21       "links": [
22         {
23           "rel": "self",
24           "href": "https://5629979.suitetalk.api.netsuite.com/services/rest/record/v1/cashsale/2122"
25         }
26       ],
27       "id": "2122"
28     },
29     {
30       "links": [
31         {
32           "rel": "self",
33           "href": "https://5629979.suitetalk.api.netsuite.com/services/rest/record/v1/cashsale/2123"
34         }
35       ],
36       "id": "2123"
37     }
38   ],
39   "offset": 0,
40   "totalResults": 3
41 }

```